SKALA

KDT Cloud

쿠버네티스 이해 및 앱 배포 (판교5F)

2026.08

본 문서는 SK AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 AX



1. 왜 쿠버네티스인가
2. 컨테이너와 이미지
3. 쿠버네티스 아키텍처
4. 클러스터 내부 동작
5. 실습 환경과 kubectl
6. kubectl 실무
7. Pod
8. Deployment와 롤아웃
9. Service와 클러스터 네트워킹
10. 호출 흐름 추적
11. Ingress와 외부 노출
12. 설정과 스토리지
13. 리소스와 배포 실전



1. 왜 쿠버네티스인가

- 배포 방식의 변화 · 선언형 API · 매니페스트와 리소스 · 실습 환경

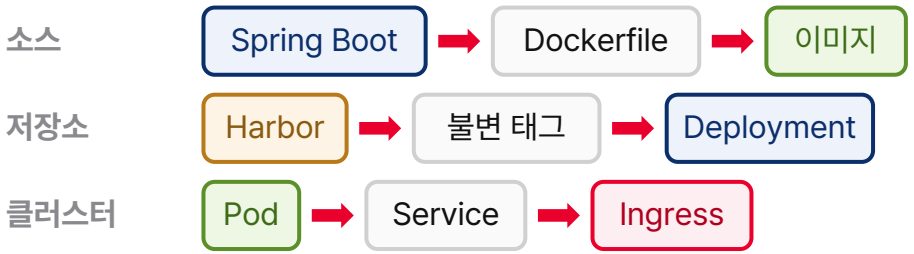
본 문서는 SK AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 AX

이 과정에서 만들어 볼 것

Spring Boot 앱 하나를 컨테이너로 만들어 EKS에 올리고, 무중단으로 고쳐 배포하는 데까지 간다

- 직전 과정에서 만든 **REST API를 그대로 가져다** 컨테이너로 감싼다
- **2일 안에 '내 앱이 도메인으로 접속된다'** 까지가 목표이다



Day 1

컨테이너와 이미지
클러스터 구조
kubectl · Pod

Day 2

Deployment · Service
Ingress · 설정 · 볼륨
배포 실전

[정리] 이 과정의 목표는 명령어 암기가 아니라 **내 앱을 내가 배포하고 고칠 수 있는 상태**이다

배포 방식은 세 번 바뀌었다

물리 서버 → 가상 머신 → 컨테이너. 매번 '한 대에 몇 개를 안전하게 올릴 수 있나'가 바뀌었다

- 격리 단위가 작아질수록 **부팅이 빨라지고 밀도가 올라간다**
- 컨테이너는 커널을 공유하므로 **OS 부팅 시간이 통째로 사라진다**

시대	격리 단위	기동 시간	한 대당 개수	약점
물리 서버	서버 1대 = 앱 1개	수 분~수십 분	1~2개	자원 낭비, 증설에 수 주
가상 머신	게스트 OS 통째로	30초~2분	10~20개	OS 중복, 이미지 수 GB
컨테이너	프로세스 + 네임스페이스	0.1~2초	50~200개	커널 공유, 격리가 약함

[참고] 컨테이너는 VM보다 **안 안전하다**. 커널을 공유하므로 커널 취약점 하나가 전체에 영향을 준다

컨테이너만으로는 부족하다

docker run 은 컨테이너 하나를 띄운다. 서비스를 운영한다는 것은 그다음 스무 가지를 하는 일이다

- 컨테이너 3개까지는 사람이 한다. **30개부터는 사람이 못 한다**
- 손으로 하던 일이 곧 **오케스트레이터의 기능 목록**이 된다

운영에서 실제로 필요한 것	docker run 만 있을 때	쿠버네티스
프로세스가 멈추면 재시작	--restart 옵션, 노드가 멈추면 끝	다른 노드에 다시 스케줄
트래픽을 여러 복제본에 분배	앞에 nginx를 직접 세우고 설정 수정	Service가 자동 분배
복제본 주소가 바뀌면 반영	설정 파일을 사람이 고치고 reload	Endpoint가 자동 갱신
새 버전으로 무중단 교체	직접 순서를 짜서 하나씩 교체	롤링 업데이트 내장
문제 생기면 이전 버전 복귀	이전 이미지 태그를 기억해야 함	rollout undo 한 줄
부하에 따라 개수 조절	모니터링 보고 사람이 실행	HPA가 지표 보고 자동
설정·비밀값 주입	-e 나열, 비밀값이 명령 이력에 남음	ConfigMap · Secret
어느 노드에 올릴지 결정	사람이 서버를 골라 SSH	스케줄러가 자원 보고 배치

[정리] 쿠버네티스는 새 개념이 아니라 **운영자가 손으로 하던 판단을 코드로 옮긴 것**이다

선언형 API와 조정 루프

'무엇을 하라'가 아니라 '어떤 상태여야 하는가'를 적으면, 컨트롤러가 계속 그 상태로 되돌린다

- 매니페스트는 명령이 아니라 **원하는 상태(desired state) 선언**이다
- 컨트롤러는 현재 상태와 비교해 차이가 사라질 때까지 반복한다. 이것이 조정 루프

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shop-api
spec:
  replicas: 3            # "3개여야 한다" — 만들라는 명령이 아니다
  selector:
    matchLabels:
      app: shop-api      # 이 라벨을 가진 Pod 를 내 것으로 센다
  template:              # 부족하면 이 틀로 새로 만든다
    metadata:
      labels: { app: shop-api }
    spec:
      containers:
        - name: app
          image: shop-api:1.0.0
```

[정리] Pod 를 사람이 지워도 3개로 되돌아온다. **지워지지 않는 게 아니라 다시 만들어지는 것**이다

명령형과 선언형, 무엇이 다른가

같은 결과를 만들지만, 두 번째 실행했을 때와 사고가 났을 때 완전히 달라진다

- 명령형은 **지금 무엇을 할지**, 선언형은 **최종 상태가 무엇인지**를 적는다
- 실습·디버깅은 명령형이 빠르고, **운영에 남기는 것은 반드시 선언형**이어야 한다

명령형은 kubectl run / create

`kubectl scale --replicas=5` 처럼 동작을 지시
두 번 실행하면 에러 (이미 있음)
누가 언제 무엇을 바꿨는지 기록이 안 남는다
클러스터를 새로 만들면 **복원 불가**
쓸 곳: 임시 확인, 디버깅용 Pod 띄우기

선언형은 kubectl apply -f

원하는 상태를 파일에 적고 적용
몇 번을 실행해도 결과가 같다 (멱등)
Git 이력이 곧 변경 이력이 된다
클러스터가 사라져도 **apply 로 재현**
쓸 곳: 운영에 올라가는 모든 것

[주의] 운영 클러스터에서 `kubectl edit` 으로 고치면 Git 과 실제가 어긋난다. 급하면 고치되 **반드시 파일에 반영**한다

Manifest 와 Resource

둘은 다른 것이다. 명세서를 내면 클러스터가 실물을 만들어 준다

- **Manifest 는 그릇, Resource 는 내용물이다. 파일을 지운다고 리소스가 사라지지 않는다**

	Manifest (명세서)	Resource (실물)
무엇	YAML 파일 · 요청서	클러스터 안에 만들어진 객체
어디에	내 노트북 · Git	etcd
누가 만드나	**사람**	**컨트롤러**
확인	`cat deploy.yaml`	`kubectl get deploy`
지우면	리소스는 그대로 남는다	클러스터에서 사라진다
예	`kind: Deployment` 를 적은 파일	이름·상태·이벤트를 가진 Deployment

[정리] 파일을 고쳤다고 클러스터가 바뀌지 않는다. `apply` 로 전달해야 비로소 리소스가 바뀐다

spec 과 status

내가 쓰는 것은 spec 뿐이다. status 는 클러스터가 채워 넣는 답장이다

- 조정 루프는 **status 를 spec 에 맞추는 일**을 끝없이 반복하는 것이다

spec 은 내가 작성한다

`replicas: 3`
`image: nginx:1.21.6`
`containerPort: 80`
원하는 상태를 적는다
손으로 편집하는 유일한 곳

status 는 클러스터가 채운다

`replicas: 3` (실제 생성됨)
`readyReplicas: 2` ← 하나가 아직
`availableReplicas: 2`
지금 상태가 적힌다
고쳐도 곧 덮어써진다

[체크] `kubectl get deploy` 의 READY 열이 `2/3` 이면 **status 가 spec 을 아직 못 따라잡은 것**이다

YAML 로 쓰고 JSON 으로 읽는다

API 서버는 JSON 으로 말한다. YAML 은 사람이 쓰기 좋으라고 있는 껍데기이다

- `-o json` 으로 보면 **API 가 실제로 주고받는 모양**이 보인다. jq 로 다루기 좋다

```
kubectl get pod my-pod -o yaml     # 사람이 읽기 좋다
kubectl get pod my-pod -o json     # 도구가 다루기 좋다

# 같은 데이터다. jq 를 쓰려면 json 이 편하다
kubectl get pod -o json | jq -r '.items[] | .metadata.name'

# YAML 을 쓸 때 자주 하는 실수
#   ① 탭 문자 — YAML 은 공백만 쓴다
#   ② 들여쓰기 2칸/4칸 혼용
#   ③ 문자열인데 따옴표를 안 씀 — port: "8080" 과 8080 은 다르다
#   ④ 목록 하이픈 뒤 공백 누락

kubectl apply --dry-run=server -f deploy.yaml   # 보내 보되 저장은 안 한다
```

[주의] `"100"` 과 `100` 은 다른 타입이다. **문자열 자리에 숫자를 넣으면** `cannot unmarshal` 로 거부된다

개발자가 알아야 할 8가지

리소스는 수십 가지지만, 앱을 올리는 데 실제로 사용하는 것은 여덟 개이다

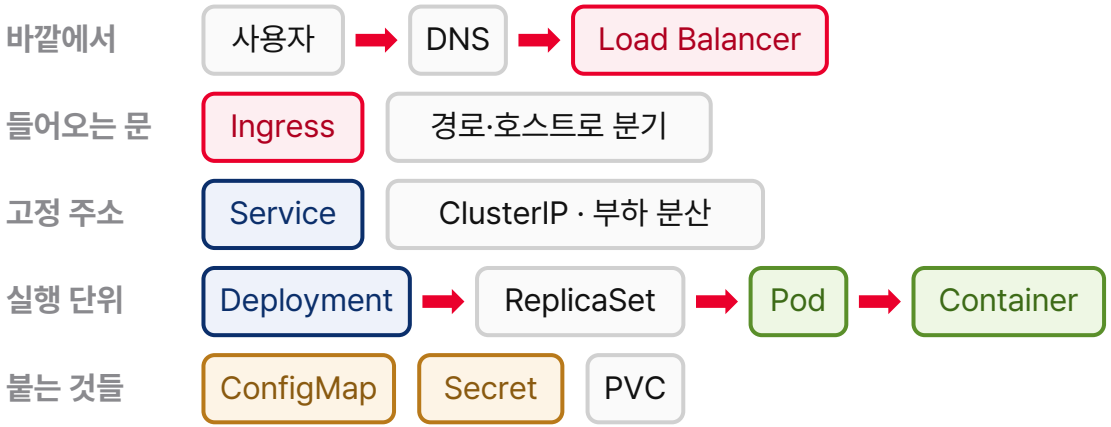
리소스	한 줄 정의	없으면	장
Pod	컨테이너를 담는 최소 실행 단위	아무것도 실행하지 못한다	7장
Deployment	Pod 를 몇 개, 어떻게 바꿀지	멈추면 안 살아난다	8장
Service	안 바뀌는 주소 + 부하 분산	Pod IP 를 쫓아다녀야	9장
Ingress	도메인·경로로 외부에서 들어오는 문	밖에서 못 붙는다	11장
ConfigMap	설정을 이미지 밖으로	환경마다 재빌드	12장
Secret	비밀값을 따로 (인코딩)	Git 에 비밀번호가 남는다	12장
PVC	사라지지 않는 저장소 요청	재시작하면 데이터가 증발	12장
StatefulSet	순서·이름이 고정된 Pod 집합	DB 같은 것을 못 올린다	심화

[정리] 앞의 넷이 '돌아가게' 하고, 뒤의 넷이 '제대로 돌아가게' 한다. 순서대로 배운다

핵심 리소스의 전체 그림

여덟 개가 따로 있는 게 아니라 하나의 요청 경로 위에 순서대로 놓여 있다

- 네트워크가 두 겹이다. Service 네트워크(클러스터)와 Pod 네트워크(오버레이)



[참고] Service 는 물리적 실체가 없다. 각 노드의 iptables 규칙일 뿐이다. 9장에서 직접 확인한다

노드 · 파드 · 컨테이너

세 겹이다. 각각이 무엇을 공유하고 무엇을 격리하는지가 다르다

- **같은 Pod 안의 컨테이너는 localhost 로 통신한다. 이것이 사이드카가 가능한 이유이다**

	노드	파드	컨테이너
정체	물리·가상 머신 (EC2)	**컨테이너 묶음**	프로세스 격리 단위
IP	노드 IP (VPC)	**Pod IP (하나)**	Pod IP 를 공유
네트워크	VPC 서브넷	Pod 마다 별도	**localhost 로 서로 통신**
저장소	노드 디스크	볼륨을 공유 가능	각자 파일시스템
수명	길다 (교체 가능)	짧다 (언제든 재생성)	Pod 와 함께
개수	노드당 **58 Pod** 상한	보통 컨테이너 1개	사이드카 시 2~3개

[체크] `kubectl get pod -o wide` 한 줄에 Pod 이름 · Pod IP · 어느 NODE 인지가 다 나온다

쿠버네티스의 경계선

어디까지가 내 책임이고 어디부터가 클러스터의 책임인지 선을 그어 둔다

- 경계를 모르면 **남의 영역을 고치려다 시간을 버린다**

경계	내 쪽	클러스터 쪽
이미지	무엇을 담을지 · Dockerfile	받아 오고 실행하기
개수	**몇 개 원하는지 (spec)**	그 개수를 유지하기
배치	제약만 준다 (requests)	**어느 노드에 둘지**
IP	관여하지 않는다	Pod IP · Service IP 할당
재시작	프로브로 판단 기준만	**실제 재시작 실행**
설정값	ConfigMap 내용	주입 시점과 방법
저장소	얼마나 · 어떤 모드	실제 볼륨 생성·연결
장애 복구	**설계로 대비**	노드 단위 재배치

[실무] 오른쪽 열을 손으로 하려 들면 반드시 충돌한다. 컨트롤러가 되돌린다. 원하는 게 있으면 spec 을 고친다

관리형 쿠버네티스와 EKS

컨트롤 플레인을 직접 운영하는 것과 맡기는 것의 차이. 이 과정은 EKS 기준으로 진행한다

- 직접 구축(kubeadm)은 **etcd 백업·인증서 갱신·HA 구성**까지 전부 우리 책임이다
- EKS는 컨트롤 플레인을 AWS가 운영하고 **우리는 노드와 워크로드만** 본다

항목	직접 구축 (kubeadm)	AWS EKS
컨트롤 플레인 서버	직접 3대 이상 구성·감시	AWS가 다중 AZ로 운영
etcd 백업	cron 으로 직접 스냅샷	AWS가 자동 백업
인증서 갱신	1년마다 수동 갱신 (놓치면 클러스터 정지)	자동
버전 업그레이드	컨트롤 플레인·노드 전부 직접	컨트롤 플레인은 콘솔에서, 노드는 우리가
비용	서버 값만 (인건비가 진짜 비용)	클러스터당 시간 $0.10 (월 약 $73)
장애 시 지원	우리 팀이 새벽에 대응	AWS 지원 티켓
AWS 연동	직접 컨트롤러 설치	IRSA · ALB · EBS CSI 를 표준으로 제공

[정리] 컨트롤 플레인은 맡기고 워크로드에 집중한다. 이 과정에서 우리가 배우는 것도 대부분 워크로드 쪽이다

이 과정의 실습 환경

EKS 클러스터 하나를 함께 쓴다. 네임스페이스는 반별로 하나이고, 반 안에서는 공유한다

- 네임스페이스가 1인 1개가 아니라 1반 1개이다. 이름 규칙이 곧 사고 방지 장치가 된다
- 실습 비용을 줄이려면 **실습 시간 외에는 노드 그룹을 0으로** 줄인다

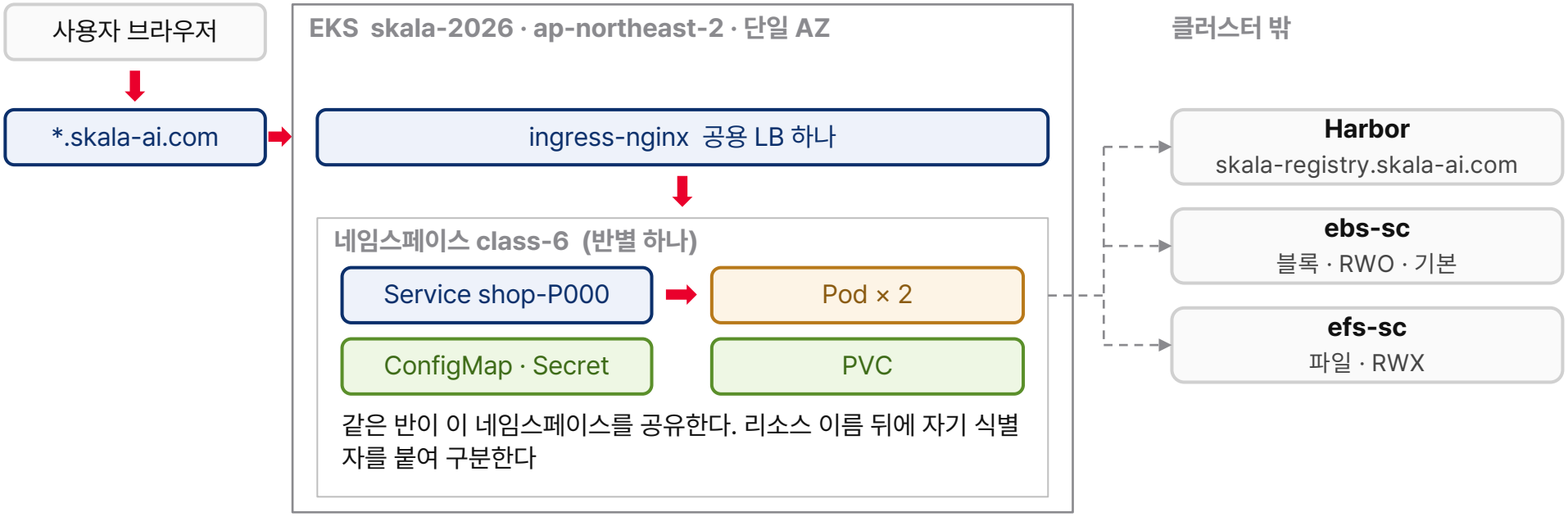
항목	값	메모
클러스터	**skala-2026** (ap-northeast-2)	Kubernetes 1.36
노드	m6i.xlarge (4vCPU · 16GiB)	훈련생용 0~6대. 노드당 **58 Pod**
네임스페이스	**class-6 ~ class-10**	반별 하나 — **같은 반은 공유**
레지스트리	**Harbor** skala-registry.skala-ai.com	프로젝트 = 네임스페이스 이름
Ingress	**ingress-nginx** (공용 LB 하나)	ALB 컨트롤러는 없다
스토리지	ebs-sc (기본) · efs-sc (RWX)	단일 AZ ap-northeast-2b
로컬 도구	kubectl · awscli v2 · docker	5장에서 설치 확인

[주의] 모든 리소스 이름 뒤에 **자기 식별자**를 붙인다 (`shop-P000`). 안 붙이면 옆 사람 것을 지우게 된다

내 앱이 놓이는 자리

클러스터 안에서 만드는 것과 밖에서 가져다 사용하는 것이 갈린다

요청은 도메인 하나로 들어와 공용 LB 를 거쳐 각 반 네임스페이스로 갈라진다. 이미지와 볼륨은 클러스터 밖에서 온다



빨간 화살표 = 요청 경로 · 점선 = 이미지와 볼륨을 가져오는 길 · 회색 상자 = 클러스터 밖

[정리] 매니페스트로 만드는 것은 네임스페이스 안쪽뿐이다. 레지스트리와 스토리지는 밖에 있고 이름으로만 참조한다

[핵심 요약] 왜 쿠버네티스인가

1장에서 남길 것은 개념 하나와 판단 기준 하나이다

항목	한 줄 정리	실무 포인트
배포의 변화	물리 → VM → 컨테이너, 격리 단위가 작아졌다	컨테이너가 VM보다 안전한 건 아니다
오케스트레이션	손으로 하던 운영 판단을 코드로 옮긴 것	재시작·분배·롤아웃·확장이 기본 기능
선언형 API	원하는 상태를 적으면 컨트롤러가 맞춘다	Pod 를 지워도 되살아나는 이유
조정 루프	현재 ↔ 목표를 계속 비교해 차이를 메운다	멈추려면 replicas=0 또는 삭제
명령형 vs 선언형	디버깅은 명령형, 운영은 선언형	edit 로 고쳤으면 그날 파일에 반영
범위 밖	로그·모니터링·CI 는 따로 붙여야 한다	도입 계획에 수집기가 없으면 첫 장애에서 막힌다
도입 판단	서비스 5개 이상 + 잦은 배포부터 이득	팀에 아는 사람 0명이면 PaaS 먼저
EKS	컨트롤 플레인은 AWS, 노드부터는 우리	버전 지원 14개월 — 연 1회 업그레이드 계획

[체크] 매니페스트에 `replicas: 3` 이라고 적으면 그다음 무슨 일이 일어나는지 **한 문장으로 말할 수 있는가?**

2. 컨테이너와 이미지

- 이미지와 레이어 · Dockerfile · JVM 메모리 · 다이제스트와 레지스트리

본 문서는 SK AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 AX

컨테이너는 무엇으로 만들어지나

컨테이너는 특별한 기술이 아니라 리눅스 커널 기능 세 가지를 조합한 '잘 격리된 프로세스'다

- 호스트에서 `ps` 를 치면 **컨테이너 프로세스가 그대로 보인다**
- 가상 머신이 아니므로 **부팅도 게스트 OS도 없다**

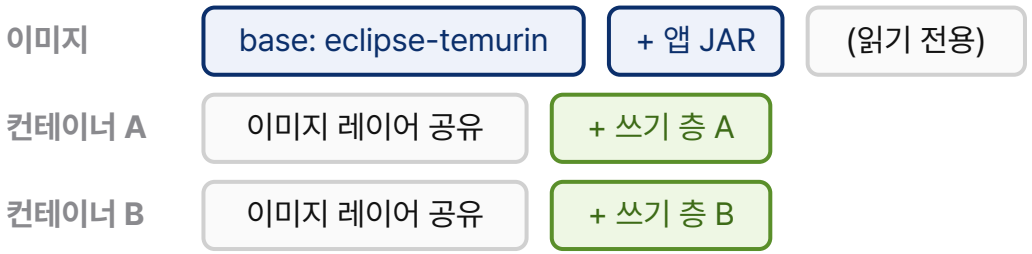
커널 기능	무엇을 하나	이것이 없으면
namespace	PID · 네트워크 · 마운트 · 사용자를 따로 보이게	옆 컨테이너의 프로세스가 다 보인다
cgroup	CPU · 메모리 사용량에 상한을 건다	한 컨테이너가 노드 메모리를 다 소모한다
union filesystem	읽기 전용 레이어를 겹쳐 하나로 보이게	이미지마다 OS 파일을 통째로 복사
capabilities	root 권한을 잘게 쪼개 일부만 준다	컨테이너 root = 호스트 root 위험
seccomp	쓸 수 있는 시스템 콜을 제한	커널 공격면이 그대로 열린다

[정리] 컨테이너 = **namespace(시야 격리) + cgroup(자원 제한) + 레이어 파일시스템**. 나머지는 편의 도구이다

이미지와 컨테이너의 관계

이미지는 읽기 전용 설계도, 컨테이너는 그 위에 쓰기 층을 하나 얹은 실행 인스턴스다

- 같은 이미지로 컨테이너를 100개 띄워도 **디스크는 이미지 한 벌만** 쓴다
- 컨테이너 안에서 만든 파일은 **컨테이너가 사라지면 같이 사라진다**



이미지
빌드 시점에 확정
읽기 전용
레지스트리에 저장

컨테이너
실행 시점에 생성
쓰기 층은 임시
지우면 데이터 소멸

남기려면
볼륨을 붙인다
12장 PV·PVC
로그는 stdout 으로

[주의] 컨테이너 안에 파일로 저장한 것은 **재시작하면 사라진다**. 로그를 파일로 쓰면 장애 원인이 같이 사라진다

이미지 레이어와 빌드 캐시

Dockerfile 한 줄이 레이어 하나이다. 바뀐 줄부터 아래는 전부 다시 빌드된다

- **자주 바뀌는 것을 아래로** 내리면 캐시가 살아 빌드가 몇 배 빨라진다
- 의존성 다운로드를 소스 복사보다 **먼저** 하는 것이 핵심이다

```
# 나쁜 예 — 소스가 한 글자만 바뀌어도 의존성을 다시 받는다
COPY . /app                    # 레이어 3  (소스가 자주 바뀜)
RUN ./gradlew build            # 레이어 4  → 매번 의존성 재다운로드 (5~10분)

# 좋은 예 — 빌드 파일이 안 바뀌면 의존성 레이어가 캐시에서 나온다
COPY gradle/ gradle/           # 레이어 3  (거의 안 바뀜)
COPY build.gradle settings.gradle ./
RUN ./gradlew dependencies     # 레이어 5  → 캐시 적중 (0초)
COPY src/ src/                 # 레이어 6  (자주 바뀜)
RUN ./gradlew bootJar          # 레이어 7  → 여기부터만 재빌드 (30초)
```

[정리] 레이어 순서 하나로 CI 빌드가 **8분에서 40초**가 된다. 커밋마다 도는 파이프라인에서는 큰 차이이다

Dockerfile 핵심 지시어

열 개 남짓이면 충분하다. 다만 CMD와 ENTRYPOINT, COPY와 ADD의 차이는 반드시 알아야 한다

- **ENTRYPOINT 는 exec 형식**으로 써야 시그널이 앱에 전달된다
- ADD 는 URL 다운로드·자동 압축해제까지 하므로 **COPY 를 기본**으로 쓴다

지시어	하는 일	실무 주의점
FROM	베이스 이미지 지정	태그를 고정한다. `:latest` 는 재현이 안 된다
WORKDIR	이후 명령의 작업 디렉터리	`RUN cd` 는 다음 줄에 안 남는다
COPY	빌드 컨텍스트에서 파일 복사	`.dockerignore` 로 불필요 파일 제외
ADD	COPY + URL · 자동 압축해제	의도치 않은 해제가 생긴다. 쓰지 않는다
RUN	빌드 시점에 명령 실행	`&&` 로 묶어 레이어 수를 줄인다
ENV	환경변수 설정	비밀값을 넣지 않는다. 이미지에 그대로 남는다
EXPOSE	문서용 포트 선언	실제로 열리지 않는다. 문서일 뿐
USER	실행 사용자 지정	**non-root 지정이 기본**
ENTRYPOINT	컨테이너의 주 명령	`["java","-jar",...]` exec 형식으로
CMD	ENTRYPOINT 의 기본 인자	docker run 뒤 인자로 덮인다

[주의] `ENTRYPOINT java -jar app.jar` (셸 형식)로 쓰면 앱이 PID 1이 아니라 **SIGTERM 을 못 받고 강제 종료**된다

Spring Boot 앱의 Dockerfile

멀티스테이지로 빌드 환경과 실행 환경을 분리한다. 이미지가 700MB에서 200MB로 줄어든다

- 빌드 단계의 **JDK·Gradle·소스는 최종 이미지에 남지 않는다**

```
# ── 1단계: 빌드 (JDK 필요) ──────────────────────────
FROM eclipse-temurin:21-jdk AS builder
WORKDIR /build
COPY gradlew settings.gradle build.gradle ./
COPY gradle/ gradle/
RUN ./gradlew dependencies --no-daemon      # 의존성만 먼저 (캐시 대상)
COPY src/ src/
RUN ./gradlew bootJar --no-daemon

# ── 2단계: 실행 (JRE 만 있으면 된다) ─────────────────
FROM eclipse-temurin:21-jre AS runtime
WORKDIR /app
RUN addgroup --system app && adduser --system --ingroup app app
COPY --from=builder /build/build/libs/*.jar app.jar
USER app                                    # non-root 로 실행
EXPOSE 8080
ENTRYPOINT ["java","-XX:MaxRAMPercentage=75","-jar","/app/app.jar"]
```

[정리] builder 단계의 결과물만 복사한다. 소스·빌드 캐시·JDK 가 최종 이미지에 남지 않아 공격면도 줄어든다

컨테이너 안의 JVM 메모리

JVM은 컨테이너의 메모리 제한을 인식한다. 다만 힙 외에도 메모리를 쓴다는 게 함정이다

- `-Xmx` 를 고정하면 **limit 을 바꿀 때마다 이미지를 다시 빌드**해야 한다
- `MaxRAMPercentage` 는 **컨테이너 limit 기준 비율**이라 limit 만 바꾸면 된다

메모리 영역	무엇인가?	대략 크기	메모
힙(Heap)	객체가 사는 곳	limit × 75%	MaxRAMPercentage 로 지정
메타스페이스	클래스 메타데이터	50~150MB	Spring 은 클래스가 많아 큰 편
스레드 스택	스레드당 1MB	200 스레드 = 200MB	톰캣 기본 스레드 수 확인
코드 캐시	JIT 컴파일 결과	50~100MB	장시간 구동 시 증가
직접 버퍼	NIO · Netty	수십 MB	누수가 나면 여기서 샌다
합계 여유	힙 외 오버헤드	**limit 의 20~25%**	그래서 75%가 권장값

[주의] 힙을 limit 의 90%로 잡으면 힙은 안 찼는데 컨테이너가 OOMKilled 된다. 가장 헷갈리는 장애 유형이다

이미지 태그 전략

latest 는 배포 도구가 아니다. 무엇이 돌고 있는지 모르게 만들고 롤백을 불가능하게 한다

- 태그는 불변(immutable) 이어야 한다. 같은 태그에 다른 내용을 덮어쓰지 않는다
- 롤백은 결국 **이전 태그로 되돌리는 일**이다. 태그가 없으면 롤백도 없다

태그 방식	예	쓸 곳	판단
latest	shop-api:latest	로컬 실습만	운영 금지. 무엇이 실행 중인지 모른다
시맨틱 버전	shop-api:1.4.2	릴리스	사람이 읽기 좋다. 수동 관리 부담
Git 커밋 SHA	shop-api:a3f9c21	CI 자동 배포	**가장 안전**. 코드와 1:1
빌드 번호	shop-api:build-482	CI	순서를 알기 쉽다
버전+SHA	shop-api:1.4.2-a3f9c21	운영 권장	읽기 좋고 추적도 된다
환경 태그	shop-api:prod	지양	가변 태그. 같은 문제 반복

[정리] 커밋 SHA 를 태그에 넣어라. 장애 났을 때 '지금 도는 게 어느 커밋인가'가 한 번에 나온다

태그는 움직이고 다이제스트는 안 움직인다

같은 태그가 어제와 오늘 다른 이미지일 수 있다. 다이제스트는 그럴 수 없다

- `sha256:...` 는 **이미지 매니페스트의 해시**이다. 내용이 1비트만 달라도 값이 바뀐다

  이미지 태그는 사람이 붙이는 이름이고, 이미지 다이제스트는 이미지 내용으로 계산된 고유한 지문(fingerprint)

```
# 태그로 받기 — 편하지만 같은 태그가 바뀔 수 있다
docker pull myapp:1.0.0

# 다이제스트로 받기 — 정확히 그 이미지
docker pull myapp@sha256:d59a7821cb7e3bb8b1b346a57a3d74d3...

# 지금 이미지의 다이제스트 확인
docker images --digests myapp
docker inspect myapp:1.0.0 --format '{{index .RepoDigests 0}}'

# 클러스터에서 실제로 무엇이 돌고 있나 — 태그가 아니라 이것을 본다
kubectl get pod my-pod -o jsonpath='{.status.containerStatuses[0].imageID}'
```

[실무] `latest` 가 위험한 진짜 이유가 이것이다. 롤백을 해도 같은 태그가 이미 다른 내용을 가리킬 수 있다

이미지 ID · 태그 · 다이제스트

셋 다 이미지를 가리키지만 서로 다른 문서의 해시이다. 재현을 보장하는 것은 하나뿐이다

- `docker images` 의 IMAGE ID 와 레지스트리의 다이제스트는 **다른 값**이다

	무엇의 값	언제 쓰나	예
태그	사람이 붙인 이름	일상적으로	`1.0.0` · `latest`
IMAGE ID	로컬 config 문서의 해시	로컬에서 구분	`d7c0cdd200be`
RepoDigest	**레지스트리 매니페스트의 해시**	재현·검증	`sha256:d59a78...`
imageID (Pod)	실제 돌고 있는 것	**배포 검증**	`…@sha256:d59a78`

다이제스트는 무엇의 해시인가

이미지 파일이 아니라 매니페스트 문서의 해시
매니페스트에 config 와 레이어 목록이 들어 있다
그래서 레이어가 같아도 값이 갈릴 수 있다

그래서 이런 일이 생긴다

같은 소스를 다시 빌드해도 값이 달라진다
레지스트리를 옮겨도 매니페스트가 같으면 값은 같다
태그를 지워도 다이제스트로는 계속 받는다

[주의] 로컬 IMAGE ID 는 **push 전후로 달라질 수 있다.** 재현성 확인은 반드시 RepoDigest 로 한다

다이제스트로 고정한 베이스

FROM 에 태그 대신 다이제스트(컨테이너 이미지의 내용을 기준으로 계산한 고유한 해시값)를 쓰면 빌드가 언제 돌아도 같은 결과가 나온다

- 재현 가능한 빌드의 첫걸음이다. 베이스 이미지가 바뀌면 내 앱도 바뀐다

```
# 태그 방식 — 베이스가 갱신되면 결과가 달라진다
FROM eclipse-temurin:21-jre

# 다이제스트 방식 — 언제 빌드해도 같은 베이스
FROM eclipse-temurin:21-jre@sha256:d59a7821cb7e3bb8b1b346a5...

COPY --from=build /app/build/libs/app.jar app.jar
ENTRYPOINT ["java", "-jar", "/app.jar"]

# 다이제스트 알아내기
#   docker pull eclipse-temurin:21-jre
#   docker inspect eclipse-temurin:21-jre \
#     --format '{{index .RepoDigests 0}}'
```

[주의] 다이제스트로 고정하면 **보안 패치도 자동으로 안 들어온다.** 주기적으로 올려 주는 절차가 함께 있어야 한다

레지스트리는 어떻게 인증하나

docker 는 설정 파일에, 클러스터는 Secret 에 자격증명을 둔다. 둘은 별개이다

- **내 노트북과 클러스터는 자격증명을 따로 갖는다**. 이 클러스터는 그 설정이 이미 되어 있다

	내 노트북	클러스터(Pod)
누가 받나	docker 데몬	노드의 kubelet
자격증명 위치	`~/.docker/config.json`	**Secret (dockerconfigjson)**
설정 방법	`docker login`	`kubectl create secret docker-registry`
연결	자동	**매니페스트에 `imagePullSecrets`**
범위	내 계정 전체	**네임스페이스 하나**
안 되면	`unauthorized`	`ImagePullBackOff`

[체크] 이 클러스터는 **풀 시크릿이 필요 없다**. 노드가 레지스트리에서 바로 받도록 구성되어 있다

로컬에서 먼저 돌려 본다

클러스터에 올리기 전에 로컬에서 컨테이너로 한 번 돌려 보면 문제의 절반이 여기서 걸러진다

- 컨테이너 안에서 `localhost` 는 **그 컨테이너 자신**이다. DB 연결이 여기서 자주 실패한다

```
# 빌드하고 바로 실행
docker build --platform linux/amd64 -t shop-api:dev .
docker run --rm -p 8080:8080 \
  -e SPRING_PROFILES_ACTIVE=local \
  -e JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=75" \
  --memory=1g --cpus=1 shop-api:dev      # limit 을 미리 흉내 낸다

# 헬스체크가 200 을 주는지 — K8s 프로브가 이걸 본다
curl -s localhost:8080/actuator/health | jq .

# 안 뜨면 안으로 들어가 본다
docker run --rm -it --entrypoint sh shop-api:dev
docker logs <컨테이너ID> --tail 100
```

[체크] `--memory=1g` 를 걸고도 뜨는가? 로컬에서 무제한으로만 돌려 보면 **클러스터에서 OOM 으로 처음 만난다**

[핵심 요약] 컨테이너와 Docker

2장에서 남길 것은 이미지를 만드는 사람의 판단 기준이다

항목	한 줄 정리	실무 포인트
컨테이너의 정체	namespace + cgroup + 레이어 FS 인 프로세스	VM 이 아니다. 커널을 공유한다
쓰기 층	컨테이너가 사라지면 같이 사라진다	로그는 파일이 아니라 stdout 으로
레이어 캐시	바뀐 줄부터 아래가 다시 빌드된다	의존성 → 소스 순서. 빌드 8분→40초
멀티스테이지	빌드 환경을 최종 이미지에서 뺀다	700MB → 200MB, 공격면도 축소
ENTRYPOINT	exec 형식으로 써야 SIGTERM 이 간다	셸 형식이면 graceful shutdown 실패
JVM 메모리	힙 외 오버헤드가 limit 의 20~25%	MaxRAMPercentage=75 가 기본값
플랫폼	맥 M시리즈는 arm64 로 빌드된다	`--platform linux/amd64` 를 습관으로
태그	latest 는 배포도 롤백도 못 한다	커밋 SHA 를 태그에 넣는다
보안	이미지에 넣은 것은 공개된 것	non-root · 스캔 · 비밀값 금지

[체크] 지금 우리 팀 Dockerfile 은 **non-root 인가, 태그가 불변인가, 의존성 캐시가 살아 있는가?**

3. 쿠버네티스 아키텍처

- 컨트롤 플레인 · 워커 노드 · 오브젝트 구조 · 라벨과 네임스페이스

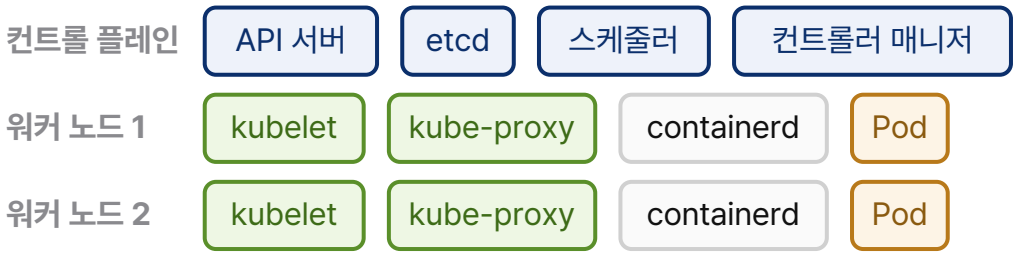
본 문서는 SK AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 AX

클러스터의 전체 그림

컨트롤 플레인은 결정하고, 워커 노드는 실행한다. 모든 대화는 API 서버를 거친다

- 컴포넌트끼리 **직접 대화하지 않는다**. 전부 API 서버를 통한다
- 그래서 API 서버가 멈추면 새 배포는 막히지만 실행 중이던 Pod 는 계속 동작한다



결정하는 쪽

무엇이 있어야 하는가
어느 노드에 둘 것인가
상태를 저장한다

실행하는 쪽

컨테이너를 실행한다
상태를 보고한다
트래픽을 넘긴다

연결하는 것

모든 통신은 API 서버
etcd 에만 상태 저장
kubelet 이 주기적으로 조회

[정리] API 서버가 유일한 관문이다. kubectl 도, kubelet 도, 컨트롤러도 전부 여기에만 말을 건다

컨트롤 플레인 구성요소

네 개이다. 각각이 무엇을 결정하는지 알면 장애 시 어디를 볼지가 정해진다

- EKS 에서는 이 넷이 **AWS 관리 영역**이라 직접 접근할 수 없다
- 대신 **컨트롤 플레인 로그를 CloudWatch 로** 내보내 간접 확인한다

컴포넌트	하는 일	이것이 멈추면
kube-apiserver	모든 요청의 관문. 인증·인가·검증 후 etcd 에 기록	kubectl 이 전부 실패. 돌던 Pod 는 유지
etcd	클러스터의 유일한 상태 저장소 (키-값 DB)	클러스터 전체 정지. 백업이 곧 생명줄
kube-scheduler	새 Pod 를 어느 노드에 둘지 결정	새 Pod 가 Pending 에서 멈춘다
kube-controller-manager	Deployment·ReplicaSet 등 컨트롤러 묶음. 조정 루프를 실행한다	자동 복구·스케일이 멈춘다
cloud-controller-manager	LoadBalancer·노드 등 클라우드 연동	Service 의 LB 가 안 만들어진다

[주의] 직접 구축한 클러스터라면 **etcd 백업이 최우선**이다. etcd 를 잃으면 클러스터를 잃는다

워커 노드 구성요소

kubelet이 컨테이너를 관리하고, kube-proxy가 트래픽 규칙을 만들고, 런타임이 실제로 실행한다

- kubelet 은 **API 서버를 폴링**하며 자기 노드에 배정된 Pod 를 맞춘다
- kube-proxy 는 트래픽을 직접 나르지 않고 **커널 규칙만 설치**한다

컴포넌트	하는 일	실무에서 만나는 지점
kubelet	Pod 스펙대로 컨테이너를 띄우고 프로브를 실행	프로브 실패·이미지 풀 실패가 여기 로그에
kube-proxy	Service 의 가상 IP → Pod IP 규칙을 커널에 설치	iptables/IPVS 모드. 규칙이 많으면 지연
컨테이너 런타임	containerd — 이미지 받고 컨테이너 실행	`crictl ps` 로 노드에서 직접 확인
CNI 플러그인	Pod 에 IP 를 할당하고 네트워크 연결	EKS 는 VPC CNI — Pod 가 VPC IP 를 받는다
CSI 드라이버	볼륨을 노드에 붙이고 마운트	EBS CSI — PVC 로 볼륨 자동 생성
노드 에이전트	로그 수집기·모니터링 등 DaemonSet	Fluent Bit · CloudWatch Agent

[참고] EKS 의 **VPC CNI 는 Pod 에 실제 VPC IP 를 준다.** 편리하지만 노드 타입마다 Pod 개수 상한이 생긴다

kubectl apply 한 줄의 여정

명령 하나가 컨테이너가 되기까지 여섯 단계를 거친다. 각 단계가 곧 장애 지점이다

- 각 단계에서 멈추면 **Pod 상태가 다르게 나타난다**

단계	누가	무엇을	여기서 멈추면
①	kubectl	YAML 을 API 서버로 POST	인증 실패 · RBAC 거부
②	API 서버	인증·인가·검증 후 etcd 에 저장	필드 오타 → 검증 에러
③	Deployment 컨트롤러	ReplicaSet 을 만든다	(거의 안 멈춤)
④	ReplicaSet 컨트롤러	부족한 수만큼 Pod 오브젝트 생성	쿼터 초과 → 생성 거부
⑤	스케줄러	노드를 골라 Pod 에 기록	**Pending** — 자원·taint·AZ
⑥	kubelet	이미지 받고 컨테이너 실행	**ImagePullBackOff** — 경로·태그
⑦	kubelet	프로브 확인 후 Ready 표시	**CrashLoopBackOff** — 앱 오류
⑧	Endpoint 컨트롤러	Ready 인 Pod 를 Service 에 등록	Endpoints 비어 있음 → 503

[정리] Pending 은 스케줄러, ImagePull 은 레지스트리, CrashLoop 은 앱. 상태 이름이 곧 단계를 알려 준다

Pod 하나가 뜨기까지

kubectl run 한 줄에 다섯 컴포넌트가 순서대로 관여한다. 멈춘 지점이 곧 원인이다

- 각 단계에서 멈췄을 때 보이는 상태 이름이 다르다. 그것이 진단의 출발점이다

#	누가	무엇을	여기서 멈추면
1	kubectl	API 서버로 요청	연결·인증 오류
2	API 서버	검증 후 etcd 저장	Forbidden · 검증 오류
3	**스케줄러**	노드 배정 (watch 로 감지)	**Pending**
4	kubelet	런타임에 생성 요청	**ContainerCreating**
5	런타임	이미지 받고 컨테이너 실행	**ImagePullBackOff**
6	kubelet	프로브 확인 후 Ready 보고	**Running 인데 0/1**
7	Endpoint 컨트롤러	Service 목록에 추가	**503**

[체크] `kubectl get pod` **의 STATUS 하나로 3~5 단계가 갈린다.** 그 다음 `describe` 의 Events 를 아래에서 위로 읽는다

모든 오브젝트의 공통 구조

Pod든 Service든 Ingress든 네 부분으로 되어 있다. 이 틀을 알면 처음 보는 리소스도 읽힌다

- 우리가 쓰는 것은 **spec** 이고, 쿠버네티스가 채우는 것은 **status** 이다

```yaml
apiVersion: apps/v1        # 어느 API 그룹·버전인가
kind: Deployment           # 무슨 종류인가
metadata:                  # 이름표 — 누구인가
  name: shop-api
  namespace: class-6
  labels: { app: shop-api, tier: backend }
spec:                      # 원하는 상태 — 우리가 쓴다
  replicas: 3
  selector:
    matchLabels: { app: shop-api }
  template:
    metadata:
      labels: { app: shop-api }
    spec:
      containers:
        - name: app
          image: shop-api:1.0.0
# status:                  # 현재 상태 — 쿠버네티스가 채운다 (직접 쓰지 않는다)
```

[정리] spec 은 소원, status 는 현실. 컨트롤러가 하는 일은 이 둘의 차이를 없애는 것이다

라벨과 셀렉터

쿠버네티스는 이름이 아니라 라벨로 대상을 찾는다. 느슨한 연결이 유연함을 만든다

- Service 는 'shop-api Pod' 가 아니라 **'app=shop-api 라벨이 붙은 것'** 을 찾는다
- 그래서 **Pod 이름이 매번 바뀌어도** 연결이 유지된다

```
# Deployment — 어떤 Pod 를 내 것으로 셀 것인가
spec:
  selector:
    matchLabels: { app: shop-api }      # ← 이 라벨을 가진 Pod 를 관리
  template:
    metadata:
      labels: { app: shop-api, ver: v1 }  # ← 만들 Pod 에 붙일 라벨
---
# Service — 어떤 Pod 로 트래픽을 보낼 것인가
spec:
  selector: { app: shop-api }           # ← 같은 라벨을 보고 찾아간다
---
# 명령줄에서도 같은 문법을 쓴다
# kubectl get pod -l app=shop-api
# kubectl get pod -l 'ver in (v1,v2)'
# kubectl delete pod -l app=shop-api,tier=backend
```

[주의] Deployment 의 `selector` 는 **만든 뒤 바꿀 수 없다.** 바꾸려면 지우고 다시 만들어야 한다

네임스페이스로 나누기

한 클러스터를 여러 논리 공간으로 나눈다. 이름 충돌을 막고 권한·쿼터의 경계가 된다

- 네임스페이스는 논리적 경계이지 보안 경계가 아니다. 네트워크는 기본적으로 뚫려 있다
- 노드·PV·StorageClass 처럼 **네임스페이스에 속하지 않는 리소스**도 있다

용도	어떻게 나누나	주의
환경 분리	dev / staging / prod	운영은 별도 클러스터가 더 안전하다
팀 분리	team-a / team-b	RBAC 으로 접근 권한을 함께 걸어야 의미가 있다
교육·실습	class-6 ~ class-10 (반별)	이름 규칙으로 반 안의 충돌을 막는다
시스템	kube-system	직접 만들지 않는다. 건드리면 클러스터가 흔들린다
기본	default	실무에서는 쓰지 않는 편이 좋다
속하지 않는 것	Node · PV · StorageClass · ClusterRole	`kubectl api-resources --namespaced=false`

[주의] 네임스페이스를 지우면 **그 안의 리소스가 전부 함께 지워진다.** 되돌릴 수 없다

기본에는 없어서 따로 넣는 것들

쿠버네티스를 설치했다고 다 되는 게 아니다. 몇 가지는 따로 넣어야 한다

- 우리 실습 클러스터에 **무엇이 설치되어 있는지** 알아야 무엇이 되는지 안다

애드온	없으면	우리 클러스터
CNI (VPC CNI)	**Pod 가 아예 안 뜬다**	설치됨 (정책 집행은 꺼짐)
CoreDNS	이름으로 통신 불가	설치됨
metrics-server	`kubectl top` · HPA 불가	**설치됨**
Ingress Controller	Ingress 가 무시된다	**ingress-nginx**
CSI 드라이버	PVC 가 Pending	EBS · EFS 설치됨
cert-manager	TLS 인증서 수동	설치됨 (letsencrypt)
로그 수집기	**Loki 가 비어 있다**	**없음**

[체크] kubectl get pods -n kube-system 으로 무엇이 실행 중인지 확인한다. 없는 기능을 사용하려다 시간을 버리지 않게 된다

[핵심 요약] 쿠버네티스 아키텍처

3장에서 남길 것은 장애가 났을 때 어디를 볼지 정하는 지도이다

항목	한 줄 정리	실무 포인트
통신 구조	모든 대화는 API 서버를 거친다	API 서버가 죽어도 돌던 Pod 는 산다
etcd	클러스터의 유일한 상태 저장소	잃으면 클러스터를 잃는다. 매니페스트를 Git 에
스케줄러	필터로 거르고 스코어로 고른다	Pending 이면 describe 의 Events 부터
kubelet	Pod 스펙대로 맞추고 프로브를 실행한다	ImagePull · CrashLoop 은 여기 로그
kube-proxy	규칙만 설치하고 패킷은 커널이	프로세스가 죽어도 기존 연결은 유지
apply 여정	8단계 — 상태 이름이 곧 단계	Pending/ImagePull/CrashLoop 분기
spec vs status	소원과 현실. 컨트롤러가 좁힌다	readyReplicas 를 replicas 와 비교
라벨	이름이 아니라 라벨로 찾는다	Deployment selector 는 불변
EKS 인증	IAM 이 인증, RBAC 이 인가	Unauthorized 와 forbidden 은 다른 문제

[체크] Pod 가 `Pending` 이다. **가장 먼저 칠 명령 한 줄은 무엇인가?**

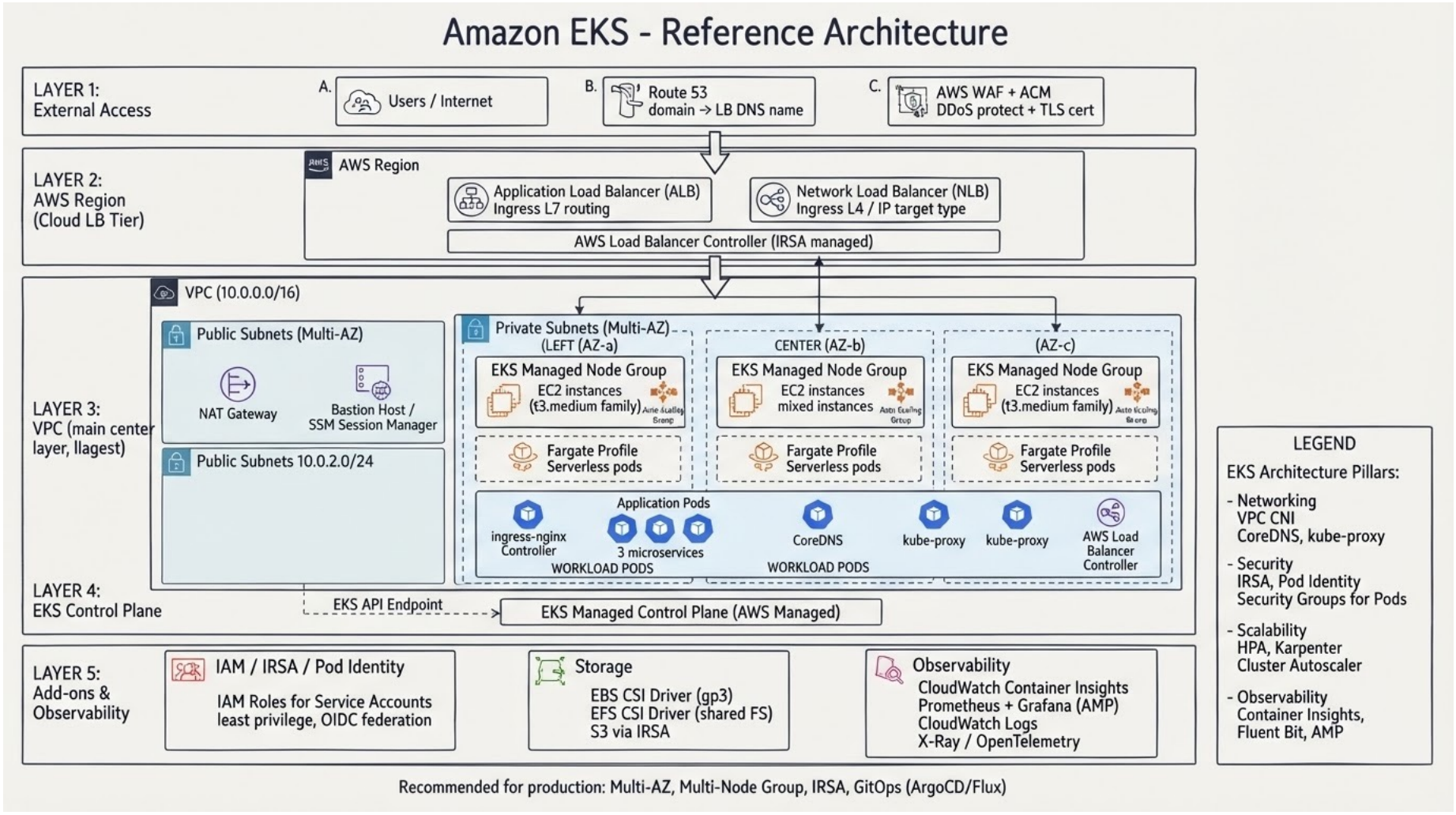
4. 클러스터 내부 동작

- 요청 처리 단계 · 컨트롤러와 스케줄러 · kubelet 과 런타임 · 장애 영향

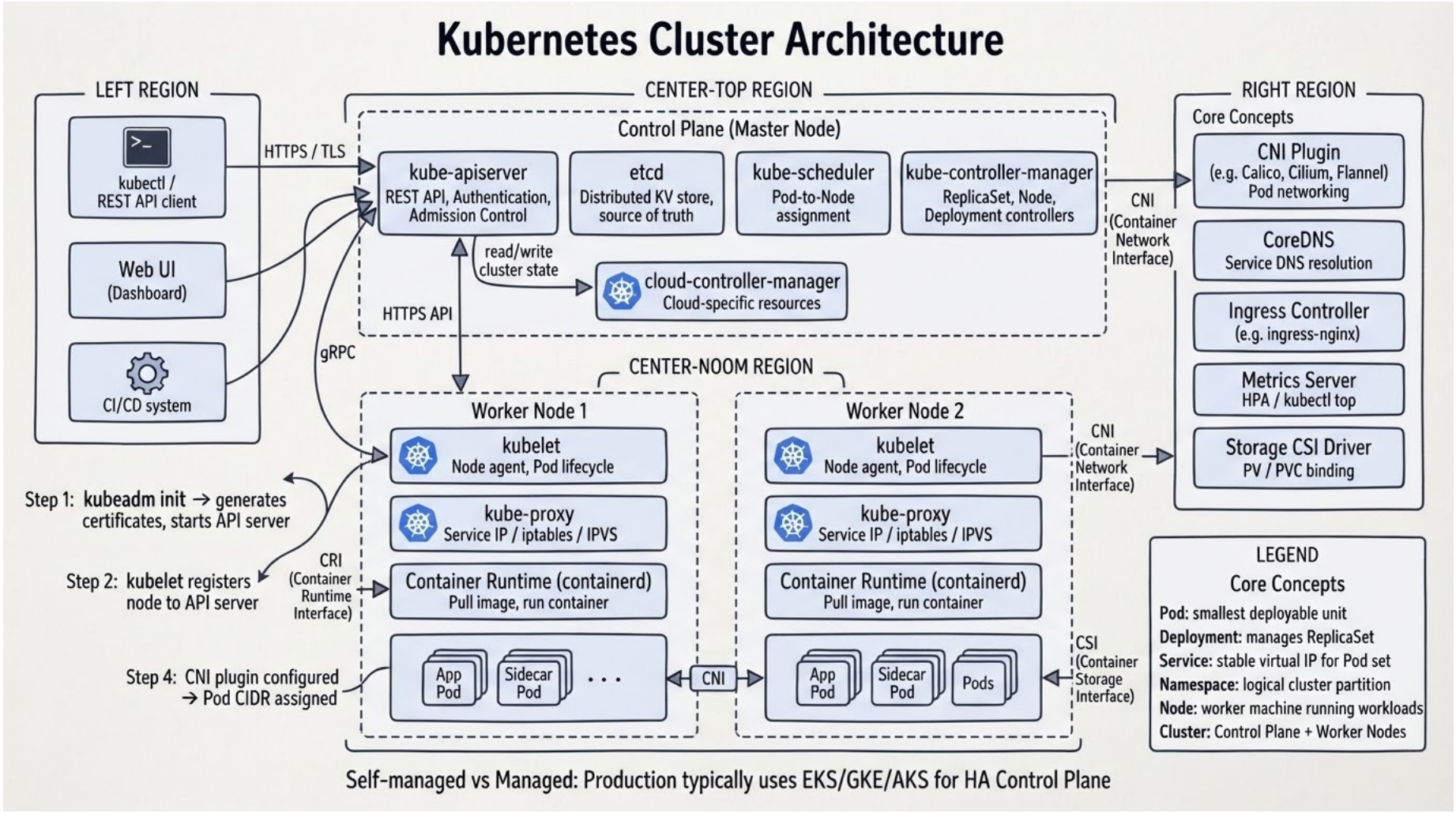
본 문서는 SK AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 AX

AWS EKS 아키텍처


Amazon EKS - Reference Architecture
LAYER 1:
External Access
A.
Users / Internet
B.
Route 53
domain → LB DNS name
C.
AWS WAF + ACM
DDoS protect + TLS cert
LAYER 2:
AWS Region
(Cloud LB Tier)
AWS Region
Application Load Balancer (ALB)
Ingress L7 routing
Network Load Balancer (NLB)
Ingress L4 / IP target type
AWS Load Balancer Controller (IRSA managed)
LAYER 3:
VPC (main center
layer, llagest)
VPC (10.0.0.0/16)
Public Subnets (Multi-AZ)
NAT Gateway
Bastion Host /
SSM Session Manager
Public Subnets 10.0.2.0/24
Private Subnets (Multi-AZ)
(LEFT (AZ-a)
CENTER (AZ-b)
(AZ-c)
EKS Managed Node Group
EC2 instances
(t3.medium family)
EKS Managed Node Group
EC2 instances
mixed instances
EKS Managed Node Group
EC2 instances
(t3.medium family)
Fargate Profile
Serverless pods
Fargate Profile
Serverless pods
Fargate Profile
Serverless pods
Application Pods
ingress-nginx
Controller
3 microservices
CoreDNS
kube-proxy
kube-proxy
AWS Load
Balancer
Controller
WORKLOAD PODS
WORKLOAD PODS
LAYER 4:
EKS Control Plane
EKS API Endpoint
EKS Managed Control Plane (AWS Managed)
LAYER 5:
Add-ons &
Observability
IAM / IRSA / Pod Identity
IAM Roles for Service Accounts
least privilege, OIDC federation
Storage
EBS CSI Driver (gp3)
EFS CSI Driver (shared FS)
S3 via IRSA
Observability
CloudWatch Container Insights
Prometheus + Grafana (AMP)
CloudWatch Logs
X-Ray / OpenTelemetry
LEGEND
EKS Architecture Pillars:
- Networking
VPC CNI
CoreDNS, kube-proxy
- Security
IRSA, Pod Identity
Security Groups for Pods
- Scalability
HPA, Karpenter
Cluster Autoscaler
- Observability
Container Insights,
Fluent Bit, AMP
Recommended for production: Multi-AZ, Multi-Node Group, IRSA, GitOps (ArgoCD/Flux)

쿠버네티스 아키텍처


Kubernetes Cluster Architecture
LEFT REGION
kubectl / REST API client
Web UI (Dashboard)
CI/CD system
HTTPS / TLS
gRPC
CENTER-TOP REGION
Control Plane (Master Node)
kube-apiserver
REST API, Authentication, Admission Control
etcd
Distributed KV store, source of truth
kube-scheduler
Pod-to-Node assignment
kube-controller-manager
ReplicaSet, Node, Deployment controllers
read/write cluster state
cloud-controller-manager
Cloud-specific resources
HTTPS API
CENTER-NOOM REGION
Worker Node 1
Worker Node 2
kubelet
Node agent, Pod lifecycle
kube-proxy
Service IP / iptables / IPVS
Container Runtime (containerd)
Pull image, run container
CRI (Container Runtime Interface)
App Pod
Sidecar Pod
Pods
CNI
CSI (Container Storage Interface)
RIGHT REGION
Core Concepts
CNI Plugin
(e.g. Calico, Cilium, Flannel)
Pod networking
CoreDNS
Service DNS resolution
Ingress Controller
(e.g. ingress-nginx)
Metrics Server
HPA / kubectl top
Storage CSI Driver
PV / PVC binding
Step 1: kubeadm init → generates certificates, starts API server
Step 2: kubelet registers node to API server
Step 4: CNI plugin configured → Pod CIDR assigned
LEGEND
Core Concepts
Pod: smallest deployable unit
Deployment: manages ReplicaSet
Service: stable virtual IP for Pod set
Namespace: logical cluster partition
Node: worker machine running workloads
Cluster: Control Plane + Worker Nodes
Self-managed vs Managed: Production typically uses EKS/GKE/AKS for HA Control Plane

API 서버가 요청을 받는 순서

apply 한 줄이 etcd 에 닿기까지 여섯 관문을 지난다. 어디서 막혔는지가 곧 원인이다

- **거부 메시지의 말투로 어느 관문인지 알 수 있다. 진단이 빨라진다**

#	관문	하는 일	여기서 막히면
1	HTTP Handler	요청을 API 객체로 변환	`400 Bad Request`
2	**인증 (authn)**	누구인가 — 토큰·인증서 확인	`Unauthorized`
3	**인가 (authz)**	할 수 있는가 — RBAC	`Forbidden`
4	Mutating Admission	값을 채워 넣음 (sidecar 주입 등)	webhook 오류
5	Schema 검증	필드·타입이 스펙에 맞는가	`cannot unmarshal`
6	**Validating Admission**	정책 검사 (Quota·보안)	정책 이름과 함께 거부
7	etcd 저장	여기까지 오면 성공	—

[정리] 3번(RBAC)과 6번(정책)을 헷갈리면 안 된다. 권한은 있는데 거부되는 것은 6번이다

컨트롤러 매니저 안의 컨트롤러들

하나의 프로세스 안에 수십 개의 컨트롤러가 각자 자기 리소스를 지켜본다

- 전부 같은 일을 한다. 지금 상태를 보고 원하는 상태와 다르면 움직인다

컨트롤러	무엇을 지켜보나	무슨 일을 하나
Deployment	Deployment 의 spec	ReplicaSet 을 만들고 교체한다
ReplicaSet	Pod 개수	**모자라면 만들고 남으면 지운다**
Node	노드의 하트비트	응답 없으면 NotReady · Pod 축출
Endpoint	Pod 의 Ready 상태	**Service 뒤 목록을 갱신**
Namespace	네임스페이스 삭제	안의 리소스를 전부 정리
PersistentVolume	PVC 요청	볼륨을 붙이고 뗀다
Job · CronJob	완료 여부 · 시각	실행하고 정리한다

[체크] Endpoint 컨트롤러가 이 과정에서 가장 자주 만나는 것이다. readiness 가 내려가면 이 컨트롤러가 목록에서 뺀다

스케줄러가 노드를 고르는 법

거르고, 점수를 매기고, 가장 높은 노드에 둔다. 두 단계다

- **하드 제약은 거르고 소프트 제약은 점수만 준다. 이 차이가 Pending 진단을 가른다**

단계	무엇을 보나	성격
① 거르기	requests 가 들어갈 자리가 있나	**하드** — 못 넣으면 탈락
	nodeSelector 라벨이 맞나	**하드**
	taint 를 toleration 하는지	**하드**
	노드당 Pod 상한(58)에 여유가 있나	**하드**
	PV 의 AZ 와 맞나	**하드**
② 점수	자원이 고르게 퍼지는가	소프트
	affinity 선호도	소프트
	topologySpread 균형	소프트

[주의] 모든 노드가 ① 에서 탈락하면 Pending 이다. `describe pod` 의 `0/N nodes are available` 뒤에 이유가 붙는다

상태는 etcd 에만 있다

모든 상태가 여기 있다. 다른 컴포넌트는 전부 이것을 보고 움직인다

- **etcd 가 멈추면 클러스터의 기억이 사라진다. 실행 중이던 Pod 는 살아 있지만 변경이 불가능해진다**

질문	답
무엇인가	분산 키-값 저장소 (Raft 합의)
무엇이 들어 있나	**모든 리소스의 spec 과 status**
누가 쓰나	**오직 kube-apiserver 만** — 다른 컴포넌트는 API 서버를 거친다
다른 컴포넌트는	API 서버에 **watch** 를 걸고 변경 알림을 받는다
EKS 에서는	AWS 가 관리한다. 접근도 백업도 우리 몫이 아니다
직접 만든 클러스터면	**백업이 곧 클러스터 백업이다. 최우선 운영 항목**

[참고] watch 구조라서 폴링이 없다. 변경이 생기면 API 서버가 알려 준다. 그래서 반응이 빠르다

클라우드 컨트롤러 매니저

쿠버네티스가 AWS 자원을 만들어 주는 통로이다. LoadBalancer Service 가 여기서 나온다

- 매니페스트 한 장이 **실제 AWS 요금이 되는 지점**이다

내가 만든 것	CCM 이 AWS 에 하는 일	요금
`type: LoadBalancer` **Service**	**ELB/NLB 생성**	시간당 + 트래픽
노드 추가	EC2 인스턴스를 노드로 등록	—
노드 삭제	**클러스터에서 제거**	—
PVC (EBS)	EBS 볼륨 생성·연결	GB 당
Pod 네트워크	VPC IP 할당 (VPC CNI)	—
Route	노드 간 경로 설정	—

[주의] Service 를 지워야 ELB 가 지워진다. 클러스터만 지우고 Service 를 안 지우면 로드밸런서가 남아 요금이 계속 나간다

kubelet 이 실제로 하는 일

노드마다 하나씩 있는 대리인이다. 클러스터의 결정을 이 노드에서 실행한다

- **kubelet 은 etcd 를 안 본다. API 서버에 watch 를 걸고 '내 노드 것'만 받는다**

하는 일	구체적으로	실패하면
Pod 실행	런타임(containerd)에 컨테이너 생성 요청	ContainerCreating 에서 멈춤
프로브 실행	liveness · readiness · startup 호출	재시작 또는 트래픽 차단
상태 보고	Pod 와 노드 상태를 API 서버로	**노드가 NotReady**
볼륨 연결	PV 를 노드에 마운트	Pod 가 Pending·마운트 오류
이미지 관리	pull 과 캐시·정리	ImagePullBackOff
자원 회수	디스크·메모리 부족 시 Pod 축출	Evicted

[정리] 프로브를 **kubelet 이 직접 호출**한다. 그래서 Service 나 Ingress 와 무관하게 동작하고, 네트워크 문제와 구분된다

컨테이너 런타임과 CRI

쿠버네티스는 Docker 를 쓰지 않는다. CRI 라는 규격으로 런타임과 대화한다

- 그래서 노드에서 `docker ps` 를 쳐도 **컨테이너가 안 보인다.** `crictl ps` 를 사용한다

옛날 kubelet ➡ dockershim ➡ Docker ➡ containerd

지금 kubelet ➡ CRI ➡ containerd

무엇이 빠졌나 Docker 는 사람이 쓰는 도구였다

이미지는 OCI 표준이라 그대로 호환된다

노드에서는 docker ps ➡ crictl ps, docker images ➡ crictl images, docker logs ➡ crictl logs

[주의] Docker 로 만든 이미지는 그대로 쓴다. 이미지 규격(OCI)과 실행 규격(CRI)은 다른 이야기다

kube-proxy 와 iptables

Service 라는 가상 IP 를 실제로 동작하게 만드는 것이 각 노드의 규칙표이다

- kube-proxy 는 트래픽을 중계하지 않는다. 규칙만 써 넣고 빠진다

```
# Service 를 만들면 각 노드에 이런 규칙이 생긴다 (개념)
#   ClusterIP 10.100.141.251:80 으로 온 것을
#   → 172.31.6.2:8080  (Pod A)   확률 1/3
#   → 172.31.6.7:8080  (Pod B)   확률 1/3
#   → 172.31.12.9:8080 (Pod C)   확률 1/3

kubectl get svc shop-api          # ClusterIP 확인
kubectl get endpoints shop-api    # 규칙의 오른쪽 목록

# 노드에서 (권한이 있다면)
#   iptables -t nat -L KUBE-SERVICES -n | grep 10.100.141.251
#   ipvsadm -Ln                   # IPVS 모드인 경우
```

[정리] Endpoints 가 곧 규칙의 오른쪽이다. 비어 있으면 규칙이 갈 곳이 없어 연결이 거부된다

컴포넌트가 멈추면 무엇이 함께 멈추나

전부 다 멈추지는 않는다. 무엇이 계속 동작하는지 아는 것이 장애 대응의 시작이다

- **이미 실행 중인 Pod 는 대부분의 장애에서 계속 동작한다. 사용자는 아무것도 못 느낀다**

멈춘 것	계속 되는 것	안 되는 것
API 서버	돌던 Pod · 기존 트래픽	조회·배포·모든 변경
etcd	돌던 Pod	**API 서버도 사실상 마비**
스케줄러	돌던 Pod · 기존 Pod 재시작	**새 Pod 배치 (Pending)**
컨트롤러 매니저	돌던 Pod	멈춘 Pod 복구 · 롤아웃
kubelet (한 노드)	다른 노드 Pod	그 노드가 NotReady → 축출
kube-proxy (한 노드)	돌던 Pod	**그 노드의 Service 연결**
CoreDNS	IP 로 하는 통신	**이름으로 하는 모든 통신**

[실무] 장애 신고를 받으면 **먼저 '사용자 트래픽이 실제로 끊겼나'** 를 확인한다. 컨트롤 플레인 장애는 대개 사용자에게 안 보인다

[핵심 요약] 클러스터 내부

컴포넌트 이름을 외우는 게 목적이 아니다. 증상에서 볼 곳을 찾는 것이 목적이다

- **모든 컴포넌트가 같은 방식으로 일한다. 지켜보다가 다르면 움직인다**

컴포넌트	한 줄로	이 증상이면 여기
API 서버	모든 요청의 유일한 입구	`Unauthorized` · `Forbidden`
etcd	클러스터의 유일한 기억	전체 조회 불가
스케줄러	Pod 를 어느 노드에 둘지	`Pending`
컨트롤러 매니저	status 를 spec 에 맞춘다	복구가 안 됨
CCM	AWS 자원을 만들고 지운다	LB · EBS 가 안 생김
kubelet	노드의 대리인 · 프로브 실행	`ContainerCreating`
런타임	컨테이너를 실제로 실행	`ImagePullBackOff`
kube-proxy	Service 규칙을 노드에 쓴다	연결 즉시 거부
CoreDNS	이름 → IP	이름 해석 실패

[체크] 요청 관문 여섯 단계와 **Pod 가 뜨는 일곱 단계**, 이 두 표가 이 장의 결론이다

5. 실습 환경과 kubectl

- 도구 설치 · 클러스터 접속 · 조회와 출력 · 권한 확인

본 문서는 SK AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 AX

필요한 도구와 버전 확인

네 가지면 된다. 버전이 안 맞아 생기는 문제가 의외로 많으니 먼저 확인하고 시작한다

- kubectl 은 클러스터와 **마이너 버전 ±1** 안이어야 안전하다

```
# 1) 설치 확인 — 네 개가 다 나와야 한다
kubectl version --client -o yaml | head -5   # v1.35 ~ v1.37
aws --version                                # aws-cli/2.x  (v1 은 안 된다)
docker --version                             # 이미지 빌드용
helm version --short                         # v3.x  (선택 · 심화 과정용)

# 2) 없으면 (macOS)
brew install kubectl awscli helm

# 3) AWS 자격증명 확인 — 이게 되어야 클러스터에 붙는다
aws sts get-caller-identity
# {  "Account": "123456789012",
#    "Arn": "arn:aws:iam::123456789012:user/class-6"  }
```

[주의] awscli v1 은 EKS 인증 토큰을 못 만든다. `aws --version` 이 `aws-cli/1.x` 면 v2 로 교체한다

EKS 클러스터에 접속하기

update-kubeconfig 한 번이면 kubectl이 클러스터를 바라보게 된다. 무엇이 만들어지는지 본다

- 이 명령은 클러스터를 만드는 게 아니라 **접속 설정을 로컬에 쓰는 것**이다
- 비밀번호가 저장되지 않는다. 매번 IAM 으로 토큰을 새로 만든다

```
# 1) 접속 설정 생성 (~/.kube/config 에 기록된다)
aws eks update-kubeconfig \
  --region ap-northeast-2 --name skala-2026 --profile skala-2026

# 2) 연결 확인
kubectl cluster-info
kubectl get nodes -o wide

# 3) 내 네임스페이스를 기본값으로 (매번 -n 을 안 써도 된다)
kubectl config set-context --current --namespace=class-6

# 4) 지금 내가 무엇을 할 수 있는지 확인
kubectl auth can-i create deployment        # yes
kubectl auth can-i delete node              # no
kubectl auth can-i get pods -n class-7    # no (다른 반)
```

[실무] `kubectl auth can-i` 는 **권한 문제를 30초에 끝내 준다.** RBAC 에러가 나면 추측하지 말고 이걸로 확인한다

컨텍스트와 KUBECONFIG

클러스터가 둘 이상이면 '지금 어디를 보고 있나'가 사고의 원인이 된다

- **운영 클러스터를 보면서 dev 명령을 치는 사고**가 실무에서 실제로 일어난다

```
# 지금 어디를 보고 있나 — 명령 치기 전에 확인하는 습관
kubectl config current-context
kubectl config get-contexts              # * 표시가 현재

# 네임스페이스 기본값 바꾸기 (매번 -n 안 쓰려고)
kubectl config set-context --current --namespace=class-6

# 컨텍스트 전환
kubectl config use-context <이름>

# 파일 단위로 격리 — 가장 안전한 방법
export KUBECONFIG=~/.kube/skala-2026.yaml
kubectl get nodes                        # 이 셸에서만 이 클러스터

# 프롬프트에 표시하기 (kube-ps1 등) — 눈으로 계속 보이게 한다
```

[실무] 클러스터가 둘 이상이면 KUBECONFIG 를 파일로 나눈다. 전역 `~/.kube/config` 하나에 몰아넣으면 언젠가 사고가 난다

kubectl 명령의 문법

동사 + 종류 + 이름 + 옵션. 이 네 칸이면 대부분의 명령이 만들어진다

- 명령을 외우지 말고 **문법을 외운다.** 나머지는 `--help` 와 탭 완성이 해 준다

칸	예	설명
동사	get · describe · logs · apply · delete	무엇을 할 것인가
	exec · edit · scale · rollout · port-forward	자주 쓰는 나머지
종류	pod · deployment · service · configmap	단수·복수·약어 모두 통한다
이름	shop-api-7d9f8-x2k4l	생략하면 전체 목록
옵션	`-n class-6`	네임스페이스
	`-l app=shop-api`	라벨로 고르기
	`-o wide` · `-o yaml` · `-o json`	출력 형식
	`-w`	변화를 실시간으로 지켜보기
	`--all-namespaces` (`-A`)	전체 네임스페이스

[정리] `kubectl <동사> <종류> <이름> <옵션>`. **이 한 줄이 kubectl 문법의 전부**이다

get 과 describe 는 보는 범위가 다르다

get은 목록을, describe는 한 리소스의 전체 사정을 보여 준다. 장애 때는 describe 부터 시작한다

- describe 의 **맨 아래 Events 가 원인의 90%** 를 알려 준다

```
# 목록 — 지금 무엇이 있나
kubectl get pod                       # 기본
kubectl get pod -o wide               # 노드·IP 까지 (아주 자주 쓴다)
kubectl get all                       # 주요 리소스 한 번에
kubectl get pod -w                    # 변화를 지켜본다

# 상세 — 이 하나에 무슨 일이 있었나
kubectl describe pod shop-api-7d9f8-x2k4l
#   Containers:  ... State: Waiting  Reason: ImagePullBackOff
#   Events:
#     Warning  Failed  2m  kubelet  Failed to pull image ...

# 최근 이벤트만 시간순으로 (클러스터 전체 진단에 유용)
kubectl get events --sort-by=.lastTimestamp | tail -20
```

[실무] `-o wide` 는 **어느 노드에 있고 IP 가 무엇인지**를 보여 준다. 분산 확인·네트워크 진단의 출발점이다

정렬하고 걸러서 조회하기

get 에 옵션 하나를 더하면 눈으로 훑을 것을 명령이 대신해 준다

- **재시작이 많은 Pod 찾기**는 장애 대응에서 가장 먼저 하는 일 중 하나이다

```
# 정렬 – 문제 있는 것을 위로 올린다
kubectl get pods --sort-by='.status.containerStatuses[0].restartCount'
kubectl get pods --sort-by=.metadata.creationTimestamp
kubectl get pv --sort-by=.spec.capacity.storage

# 상태로 거르기
kubectl get pods --field-selector=status.phase=Running
kubectl get pods --field-selector=status.phase!=Running   # 문제만
kubectl get events --field-selector=type=Warning

# 라벨로 거르기
kubectl get pods --show-labels
kubectl get pods -l app=shop-api
kubectl get pods -l 'app in (shop-api,worker)'
kubectl get pods -l app!=shop-api
```

[체크] `--field-selector=status.phase!=Running` 한 줄이면 **문제 있는 Pod 만** 남는다. 아침에 클러스터를 살필 때 이것부터 친다

출력을 원하는 대로 뽑기

-o jsonpath와 custom-columns를 알면 kubectl이 조회 도구에서 운영 도구가 된다

- **스크립트에 쓸 값 하나**를 뽑을 때 jsonpath 만 한 게 없다

```
# 전체 정의를 그대로 (필드 이름을 확인할 때)
kubectl get deploy shop-api -o yaml

# 값 하나만 — 지금 도는 이미지 태그는?
kubectl get deploy shop-api \
  -o jsonpath='{.spec.template.spec.containers[0].image}'

# 원하는 열만 표로 — Pod 가 어느 노드에 몇 번 재시작했나
kubectl get pod -o custom-columns=\
'NAME:.metadata.name,NODE:.spec.nodeName,RESTART:.status.containerStatuses[0].restartCount'

# 준비 안 된 Pod 만 골라 내기
kubectl get pod --field-selector=status.phase!=Running

# 이름만 뽑아 다른 명령에 넘기기
kubectl get pod -l app=shop-api -o name | xargs -n1 kubectl logs --tail=5
```

[실무] 장애 대응 스크립트는 대부분 **`-o jsonpath` 한 줄**로 시작한다. 필드 경로는 `-o yaml` 로 먼저 확인한다

jsonpath 로 값 하나만 뽑기

스크립트에서 쓰려면 표가 아니라 값 하나가 필요하다

- **키에 점이 들어 있으면 역슬래시로 escape 해야 한다. 가장 자주 막히는 지점이다**

```
# 기본 — 중첩 필드를 점으로 따라간다
kubectl get pod my-pod -o jsonpath='{.status.podIP}'
kubectl get svc shop-api -o jsonpath='{.spec.clusterIP}'

# 배열 — [*] 는 전부, [0] 은 첫 번째
kubectl get endpoints shop-api -o jsonpath='{.subsets[0].addresses[*].ip}'
kubectl get nodes -o jsonpath='{.items[*].metadata.name}'

# 키에 점이 있으면 escape — ConfigMap 키가 대표적이다
kubectl get cm my-config -o jsonpath='{.data.app\.properties}'

# 조건식 — 여러 컨테이너 중 이름으로 고르기
kubectl get deploy shop-api -o jsonpath=\
'{.spec.template.spec.containers[?(@.name=="app")].image}'

# 표로 뽑기 — 여러 값을 열로
kubectl get pods -o custom-columns=\
'NAME:.metadata.name,NODE:.spec.nodeName,IP:.status.podIP'
```

[주의] `?(@.name=="app")` 에서 `?()` 는 조건 시작, `@` 는 현재 항목이다. **작은따옴표로 감싸야** 셸이 안 건드린다

로그와 컨테이너 안으로

logs로 무슨 일이 있었는지 보고, exec로 직접 들어가 확인한다. 진단의 두 축이다

- 재시작한 Pod 는 **`--previous` 로 죽기 직전 로그**를 봐야 원인이 나온다

```
# 로그 — 기본
kubectl logs shop-api-7d9f8-x2k4l --tail=100
kubectl logs -f deploy/shop-api                # 실시간, Deployment 단위
kubectl logs -l app=shop-api --tail=20 --prefix # 여러 Pod 를 한 번에

# 죽기 직전 로그 — CrashLoopBackOff 진단의 핵심
kubectl logs shop-api-7d9f8-x2k4l --previous

# 컨테이너 안으로 들어가기
kubectl exec -it shop-api-7d9f8-x2k4l -- sh
kubectl exec shop-api-7d9f8-x2k4l -- env | grep SPRING   # 한 줄 실행

# 로컬 포트로 끌어와 브라우저에서 확인
kubectl port-forward deploy/shop-api 8080:8080
```

[주의] CrashLoop 상태에서 그냥 `logs` 를 치면 **새로 뜨는 컨테이너의 빈 로그**가 나온다. `--previous` 를 쓴다

만들고 고치고 지우기

운영에 남는 것은 전부 apply로 한다. 그 전에 diff로 무엇이 바뀔지 먼저 본다

- `apply` 는 **없으면 만들고 있으면 갱신**한다 — 몇 번을 실행해도 결과가 같다
- `kubectl diff` 는 **적용 전에 변경분을 보여 준다.** 운영에서는 필수 습관이다

```
# 적용 전에 무엇이 바뀌는지 본다  ← 습관으로
kubectl diff -f k8s/deployment.yaml

# 적용 (디렉터리 통째로도 된다)
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/                    # 폴더 전체
kubectl apply -k overlays/prod           # Kustomize (심화 과정)

# 급할 때만 — 반드시 파일에도 반영한다
kubectl scale deploy/shop-api --replicas=5
kubectl set image deploy/shop-api app=shop-api:1.0.1

# 지우기 — 파일 기준으로 지우면 빠뜨림이 없다
kubectl delete -f k8s/deployment.yaml
kubectl delete pod --all -n class-6   # 위험: 확인하고 실행
```

[주의] `delete pod` 는 Deployment 가 있으면 **다시 만들어진다.** 정말 멈추려면 replicas=0 또는 Deployment 삭제

파일 없이 바로 적용하기

heredoc 을 쓰면 파일을 만들지 않고도 매니페스트를 그대로 적용할 수 있다

- 실습·데모에 좋다. 다만 **기록이 안 남으므로 운영에는 파일을 쓴다**

```
kubectl apply -f - <<'EOF'
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-config
data:
  app.properties: |
    app.name=MyApp
    app.version=1.0.0
  log.level: INFO
  max.connections: "100"      # 숫자도 반드시 문자열로
EOF

# 여러 리소스를 한 번에 — --- 로 나눈다
# 지울 때도 같은 방식이 편하다
kubectl delete -f - <<'EOF'
...
EOF
```

[주의] `<<'EOF'` 처럼 **따옴표를 붙인다.** 안 붙이면 셸이 `$VAR` 를 먼저 치환해 매니페스트가 망가진다

첫 배포를 손으로 확인한다

아직 매니페스트를 안 배웠지만, 명령형으로 한 번 띄워 보고 3장의 구조를 눈으로 확인한다

- 이번만 명령형으로 한다. **7장부터는 전부 매니페스트**로 진행한다

```
# 1) Deployment 하나 만들기 (명령형 — 이번만)
kubectl create deployment hello --image=nginx:1.27 --replicas=3

# 2) 3장에서 본 것들을 눈으로 확인
kubectl get deploy,rs,pod          # Deployment → ReplicaSet → Pod
kubectl get pod -o wide            # 어느 노드에 흩어졌나

# 3) 조정 루프 실험 — Pod 하나를 지워 본다
kubectl delete pod <이름>
kubectl get pod                    # 이름이 다른 Pod 가 새로 생긴다

# 4) 정리
kubectl delete deployment hello
```

[체크] Pod 를 지웠더니 이름이 다른 Pod 가 생겼다. 이유를 옆 사람에게 한 문장으로 설명해 본다

실습 리소스 정리하기

만든 것을 지우는 것도 실습이다. 클라우드에서는 안 지운 것이 요금이 된다

- **Deployment 를 지워도 Service·Ingress·PVC 는 남는다. 종류마다 따로 지워야 한다**
- Ingress 를 지우면 ALB 도 함께 삭제된다 (11장). 반드시 확인한다

```
# ① 지금 내 네임스페이스에 무엇이 있나
kubectl get all                        # Pod·Service·Deployment·RS
kubectl get ingress,configmap,secret,pvc,hpa   # all 에 안 나오는 것들

# ② 라벨로 한꺼번에 지우기 (라벨을 붙여 두면 정리가 쉽다)
kubectl delete all,ingress,configmap,pvc -l app=shop-api

# ③ 파일 기준으로 지우기 — 빠뜨림이 없다
kubectl delete -f k8s/

# ④ 마지막 확인 — ALB·EBS 가 남아 있지 않은가
kubectl get ingress,pvc
# aws elbv2 describe-load-balancers --query 'LoadBalancers[].LoadBalancerName'
# aws ec2 describe-volumes --filters Name=status,Values=available
```

[주의] `kubectl get all` 은 **이름과 달리 전부를 보여 주지 않는다.** Ingress·ConfigMap·Secret·PVC 는 따로 조회해야 한다

[핵심 요약] 실습 환경과 kubectl

5장에서 남길 것은 손에 붙여야 할 명령과 순서이다

항목	한 줄 정리	실무 포인트
도구	kubectl · awscli v2 · docker · helm	kubectl 은 클러스터와 ±1 마이너 버전
접속	aws eks update-kubeconfig 한 번	토큰은 매번 IAM 으로 새로 만든다
기본 NS	set-context --current --namespace	안 하면 default 를 보며 헤맨다
권한 확인	kubectl auth can-i	RBAC 문제를 추측하지 말고 확인한다
문법	동사 + 종류 + 이름 + 옵션	외우지 말고 문법 + --help
진단 순서	get → describe(Events) → logs	Pending 에는 로그가 없다
재시작 로그	logs --previous	CrashLoop 원인은 여기 있다
출력 가공	-o wide / -o jsonpath	wide 는 노드·IP, jsonpath 는 값 하나
변경	diff 로 보고 apply	급할 때 명령형, 그날 안에 파일 반영

[체크] Pod 가 `CrashLoopBackOff` 이다. **어떤 명령을 어떤 순서로 칠 것인가?**

6. kubectl 실무

- 포트포워딩과 exec · 수정과 삭제 · k9s · API 직접 보기

본 문서는 SK AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 AX

포트 포워딩으로 직접 붙기

Ingress 없이도 내 노트북에서 클러스터 안 서비스에 붙을 수 있다

- 디버깅 전용이다. 터미널을 닫으면 끊긴다. 운영 접근 경로로 쓰지 않는다

```
# Service 로 — 뒤의 Pod 중 하나로 연결된다
kubectl port-forward svc/shop-api 8080:80

# Pod 로 직접 — 특정 Pod 를 찍어서 (부하 분산을 건너뛴다)
kubectl port-forward pod/shop-api-7d9f8 8080:8080

# 관리 포트만 열어 보기 — 외부에 노출되지 않은 것을 확인할 때
kubectl port-forward deploy/shop-api 8081:8081
curl -s localhost:8081/actuator/health | jq .

# 백그라운드로 띄우고 쓰기
kubectl port-forward svc/shop-api 8080:80 >/dev/null 2>&1 &
curl -s localhost:8080/api/info | jq .
kill %1
```

[실무] 특정 Pod 하나만 이상할 때 pod/ 로 직접 붙어 보면 Service 문제인지 그 Pod 문제인지 바로 갈린다

컨테이너 안으로 들어가기

로그로 안 보이는 것은 들어가서 봐야 한다. 다만 들어가기 전에 할 일이 있다

- **들어가기 전에 `describe` 와 `logs` 를 먼저 본다.** 대부분 거기서 끝난다

```
# 명령 한 번만 실행
kubectl exec shop-api-7d9f8 -- ls -al /
kubectl exec shop-api-7d9f8 -- env | grep SPRING

# 셸로 들어가기
kubectl exec -it shop-api-7d9f8 -- /bin/sh      # 경량 이미지는 sh
kubectl exec -it shop-api-7d9f8 -- /bin/bash    # bash 가 있으면

# 컨테이너가 여러 개면 -c 로 고른다
kubectl exec -it my-pod -c sidecar -- sh

# 셸이 아예 없는 이미지(distroless)라면 — 임시 컨테이너를 붙인다
kubectl debug -it shop-api-7d9f8 --image=busybox --target=app

# 클러스터 안에서 네트워크 확인용 Pod 하나 띄우기
kubectl run nettest --rm -it --image=nicolaka/netshoot -- bash
```

[주의] exec 로 고친 것은 **컨테이너가 재시작되면 사라진다.** 원인을 찾는 데만 쓰고, 고치는 것은 매니페스트로 한다

리소스 사용량 보기

requests 를 정할 때도, 느려졌을 때도 먼저 실측한다

- `top` 은 **metrics-server 가 있어야** 동작한다. 없으면 에러가 발생한다

```
kubectl top nodes                       # 노드별 사용량
kubectl top pods                        # 네임스페이스 전체
kubectl top pods -l app=shop-api        # 라벨로 거르기

# 컨테이너 단위로 — 사이드카가 얼마나 쓰는지
kubectl top pod shop-api-7d9f8 --containers

# 정렬 — 많이 쓰는 것부터
kubectl top pods --sort-by=cpu
kubectl top pods --sort-by=memory

# 이 값과 requests/limits 를 비교한다
kubectl get pod shop-api-7d9f8 -o jsonpath=\
'{.spec.containers[0].resources}'
```

[체크] `top` 은 **지금 이 순간**이다. requests 를 정하려면 며칠 치 그래프가 필요하고, 그것이 심화 과정의 모니터링이다

라벨과 애노테이션 다루기

라벨은 고르기 위한 것, 애노테이션은 붙여 두기 위한 것이다

- **라벨로는 검색이 되고 애노테이션으로는 안 된다. 그것이 둘을 가르는 기준이다**

```
# 라벨 — 붙이고, 덮어쓰고, 지운다
kubectl label pod my-pod owner=P000
kubectl label pod my-pod owner=P001 --overwrite     # 있으면 --overwrite 필수
kubectl label pod my-pod owner-                      # 끝에 - 를 붙이면 삭제

# 라벨로 고르기 — 여기가 라벨의 존재 이유다
kubectl get pods -l owner=P000
kubectl delete pods -l owner=P000

# 애노테이션 — 검색용이 아니라 기록·설정 전달용
kubectl annotate deploy shop-api \
  kubernetes.io/change-cause="1.1.0 배포 — 결제 버그 수정"
kubectl annotate ingress shop-api \
  nginx.ingress.kubernetes.io/proxy-body-size=8m

kubectl get pod my-pod -o jsonpath='{.metadata.annotations}' | jq .
```

[정리] Ingress 애노테이션이 컨트롤러에 설정을 전달하는 통로이다. 11장에서 본 그것이 이 문법이다

고치는 네 가지 방법

apply · edit · patch · replace 는 무엇을 언제 사용하는지가 다르다

- **운영에서는 apply 하나만 쓴다.** 나머지는 급할 때나 실습용이다

명령	무엇을 하나	언제	위험
apply	파일 기준으로 맞춘다	**항상 이것**	—
edit	에디터로 즉석 수정	급할 때 · 실습	**파일과 어긋난다**
patch	일부 필드만 명령으로	스크립트	파일과 어긋난다
set image	이미지만 바꾸는 지름길	배포 실습	파일과 어긋난다
replace --force	**지우고 새로 만든다**	고칠 수 없는 필드	**중단이 생긴다**

[주의] `replace --force` 는 **삭제 후 재생성**이다. Pod 가 잠시 사라지므로 운영에서는 트래픽이 끊긴다

지우는 여러 가지 방법

무엇을 지우는지 확인하고 지운다. 되돌릴 수 없다

- **`--all` 과 `-l` 을 칠 때는 먼저 `get` 으로 같은 조건을 확인**한다

```
# ① 파일 기준 — 만든 것과 같은 방식으로 지운다 (가장 안전)
kubectl delete -f deploy.yaml
./render.sh l16-complete/*.yaml | kubectl delete -f -

# ② 이름으로
kubectl delete deploy shop-P000

# ③ 라벨로 — 먼저 get 으로 확인하고 지운다
kubectl get pods,svc -l owner=P000        # ← 반드시 먼저
kubectl delete pods,svc -l owner=P000

# ④ 전부 — 공유 네임스페이스에서는 절대 쓰지 않는다
kubectl delete pod --all                    # 위험

# 즉시 삭제 — graceful shutdown 을 건너뛴다
kubectl delete pod my-pod --now             # 종료 유예 무시
kubectl delete pod my-pod --grace-period=0 --force   # 최후의 수단
```

[주의] 우리 실습은 **반 전체가 한 네임스페이스를 사용한다.** `--all` 은 옆 사람 것까지 지우므로 사용하지 않는다

kubectl cp 로 파일을 주고받는다

컨테이너 안팎으로 파일을 옮긴다. 로그 파일이나 힙 덤프를 꺼낼 때 쓴다

- 컨테이너에 **`tar` 가 있어야** 동작한다. 경량 이미지에서는 실패할 수 있다

```
# 로컬 → Pod
kubectl cp ./data my-pod:/shared-data/data

# Pod → 로컬  (장애 분석에서 이 방향을 더 자주 쓴다)
kubectl cp my-pod:/shared-data/logs ./download

# 네임스페이스를 명시하는 형식
kubectl cp ./data class-6/my-pod:/shared-data/data

# 컨테이너가 여러 개면
kubectl cp ./data my-pod:/tmp/data -c app

# 확인
kubectl exec my-pod -- ls -al /shared-data/data

# 힙 덤프 꺼내기 (자바 앱 장애 분석)
kubectl exec my-pod -- jcmd 1 GC.heap_dump /tmp/heap.hprof
kubectl cp my-pod:/tmp/heap.hprof ./heap.hprof
```

[실무] 꺼내는 방향이 진짜 쓸모이다. OOM 직전 힙 덤프나 스레드 덤프를 받아 로컬 도구로 분석한다

k9s 는 터미널용 대시보드이다 (별도 설치 필요)

kubectl 을 수백 번 치는 대신 화면에서 돌아다닌다. 관찰에 특히 좋다

- **상태가 실시간으로 갱신**된다. `get -w` 를 여러 개 실행할 필요가 없다

키	무엇	kubectl 로는
`k9s -n class-6`	네임스페이스를 열고 시작	—
`:pod` · `:svc` · `:deploy`	리소스 종류 전환	`get <종류>`
`/문자열`	필터	`-l` · grep
`d`	describe	`describe`
`l`	로그	`logs`
`s`	셸로 들어가기	`exec -it`
`Ctrl-d`	삭제	`delete`
`:q` · `Esc`	종료 · 뒤로	—

[주의] `Ctrl-d` **가 삭제**이다. 확인 창이 뜨지만 습관적으로 넘기기 쉬우니 익숙해지기 전에는 공유 네임스페이스에서 조심한다

kubectl 없이 API 직접 보기

kubectl 은 REST API 클라이언트일 뿐이다. 그 아래를 직접 볼 수 있다

- `--v=8` **을 붙이면 kubectl 이 실제로 보낸 요청**이 그대로 보인다. 학습에 좋다

```
# kubectl 이 무슨 요청을 보내는지 그대로 본다
kubectl get pods --v=8 2>&1 | grep -E 'GET|Request|Response'

# API 를 직접 호출 — 인증은 kubectl 이 대신해 준다
kubectl get --raw /api
kubectl get --raw /apis
kubectl get --raw /api/v1/namespaces/class-6/pods | jq -r '.items[].metadata.name'

# 로그도 API 다
kubectl get --raw \
  /api/v1/namespaces/class-6/pods/my-pod/log

# 프록시를 띄우면 브라우저로도 볼 수 있다
kubectl proxy --port=8080 &
curl -s localhost:8080/api/v1/namespaces/class-6/pods | jq -r '.items[].metadata.name'
```

[참고] 이 경로가 곧 **RBAC 의 단위**이다. `resources: ["pods/log"]` 같은 표기가 위 URL 구조에서 나온다

API 리소스 스스로 찾기

필드 이름이 기억 안 날 때 검색하지 말고 클러스터에 물어본다

- **explain 은 지금 이 클러스터 버전 기준이다. 인터넷 문서보다 정확하다**

```
# 어떤 리소스가 있나 — 축약형과 API 그룹까지
kubectl api-resources
kubectl api-resources --namespaced=true      # 네임스페이스에 속하는 것
kubectl api-resources --namespaced=false     # 클러스터 범위 (Node, PV, SC)
kubectl api-resources | grep -i ingress

# 필드가 뭐가 있나 — 계층을 따라 내려간다
kubectl explain deployment
kubectl explain deployment.spec.strategy
kubectl explain deployment.spec.template.spec.containers.livenessProbe
kubectl explain pod --recursive | head -40   # 전체 트리

# API 버전 확인 — 매니페스트의 apiVersion 이 맞는지
kubectl api-versions | grep autoscaling
```

[체크] `no matches for kind` 에러가 나면 **`api-resources` 로 그 종류가 있는지**, `api-versions` 로 그 버전이 맞는지 확인한다

적용 전에 확인하는 습관

무엇이 바뀔지 보고 나서 적용한다. 공유 클러스터에서는 필수이다

- **`diff` 는 클러스터에 실제로 물어본다**. 로컬 파일끼리 비교하는 것이 아니다

```
# ① 무엇이 바뀌는가 — 클러스터의 현재 상태와 비교한다
kubectl diff -f deploy.yaml
kubectl diff -k overlays/dev

# ② 보내 보되 저장은 안 한다 — 스키마·정책 검증까지 받는다
kubectl apply -f deploy.yaml --dry-run=server

# ③ 로컬에서만 문법 확인 (클러스터에 연결 안 함)
kubectl apply -f deploy.yaml --dry-run=client

# ④ 매니페스트를 만들되 적용은 안 하기 — 뼈대 만들 때 편하다
kubectl create deploy demo --image=nginx --dry-run=client -o yaml > deploy.yaml
kubectl create cm my-config --from-literal=K=V --dry-run=client -o yaml | kubectl apply -f -
```

[실무] `--dry-run=client -o yaml` 로 뼈대를 만든 뒤 고쳐 쓰는 것이 매니페스트를 처음부터 타이핑하는 것보다 빠르고 정확하다

[핵심 요약] kubectl 실무

명령을 외우는 게 아니라 '무엇을 알고 싶은가'에서 명령으로 가는 길을 익힌다

- **막히면 `explain` 과 `--v=8` 로 확인한다.** 클러스터에 직접 물어보면 검색보다 정확하다

알고 싶은 것	명령
지금 상태가 어떤가	`get` · `get -o wide` · `--sort-by`
왜 그런가	**`describe` 의 Events**
앱이 뭐라고 하나	`logs` · `logs --previous` · `-c`
값 하나만 뽑고 싶다	`-o jsonpath` · `custom-columns`
문제 있는 것만 보고 싶다	`--field-selector=status.phase!=Running`
안에 들어가 보고 싶다	`exec -it` · `debug` · netshoot
내 노트북에서 붙고 싶다	**`port-forward`**
얼마나 쓰고 있나	`top pods --containers`
적용하면 뭐가 바뀌나	**`diff`** · **`--dry-run=server`**
필드 이름이 뭐였지	`explain` · `api-resources`
권한이 있나	`auth can-i`

[실무] `delete` 앞에는 반드시 `get` 을 한 번 친다. 같은 조건으로 무엇이 지워질지 눈으로 보고 나서 지운다

7. Pod

- Pod 의 정의 · 생명주기 · 프로브 · 디버깅

본 문서는 SK AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 AX

Pod 는 컨테이너가 아니다

Pod는 컨테이너 하나 이상을 묶은 껍데기이다. 네트워크와 저장소를 공유하는 '같이 사는 단위'

- 같은 Pod 안의 컨테이너는 IP 를 공유한다. 서로를 localhost 로 부른다
- 스케줄·재시작·삭제가 **Pod 단위**로 일어난다. 컨테이너 하나만 옮길 수 없다

공유하는 것	무엇을 뜻하나	실무 의미
네트워크 네임스페이스	IP · 포트 공간이 하나	같은 포트를 두 컨테이너가 못 쓴다
localhost	서로를 127.0.0.1 로 호출	사이드카와의 통신이 네트워크를 안 탄다
볼륨	같은 볼륨을 마운트 가능	로그 파일을 사이드카가 읽는 구조
생명주기	같이 뜨고 같이 종료된다	하나가 종료되면 재시작 대상은 Pod
노드	항상 같은 노드에 배치	떨어뜨릴 수 없다
공유 안 하는 것	파일시스템(볼륨 제외) · 프로세스	각자 자기 이미지의 파일을 본다

[정리] Pod = **'반드시 같은 노드에서 함께 살아야 하는 컨테이너들'** 의 묶음. 대부분은 컨테이너 하나짜리다

Pod 매니페스트 읽기

실무에서 Pod를 직접 만들 일은 거의 없다. 그래도 읽을 줄 알아야 한다. 문제는 늘 여기서 난다

- Deployment 의 `template` 안에 **이 내용이 그대로** 들어간다

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: shop-api
  labels: { app: shop-api }
spec:
  containers:
    - name: app
      image: 123456789012.dkr.ecr.ap-northeast-2.amazonaws.com/shop-api:1.0.0
      ports:
        - containerPort: 8080          # 문서용. 안 적어도 통신은 된다
      env:
        - name: SPRING_PROFILES_ACTIVE
          value: prod
      resources:                       # 13장에서 자세히
        requests: { cpu: 200m, memory: 512Mi }
        limits:   { cpu: "1",  memory: 1Gi }
  restartPolicy: Always                # Always | OnFailure | Never
```

[참고] `containerPort` 는 **문서일 뿐**이다. 안 적어도 Pod IP 로 그 포트에 접속된다. 적어 두면 읽는 사람이 편하다

Pod 를 열어서 읽기

get -o yaml 로 열면 내가 안 쓴 필드가 잔뜩 있다. 누가 채웠는지 알면 읽힌다

- **내가 쓴 것 · 기본값 · 클러스터가 채운 것** 셋이 섞여 있다

```
spec:
  containers:
  - image: skala-registry.skala-ai.com/class-6/shop-P000:1.0.0
    imagePullPolicy: IfNotPresent      # ← 기본값이 채워짐
    terminationMessagePath: /dev/termination-log   # ← 기본값
    resources: { requests: { cpu: 200m } }         # ← 내가 씀
  nodeName: ip-172-31-40-45...         # ← 스케줄러가 채움
  serviceAccountName: default          # ← admission 이 채움
status:                                # ← 전부 클러스터가 채움
  phase: Running
  podIP: 172.31.6.2
  containerStatuses:
  - restartCount: 0
    imageID: docker-pullable://...@sha256:d59a78...
    state: { running: { startedAt: "2026-08-24T01:12:03Z" } }
```

[체크] **`status.containerStatuses[].imageID`** 가 지금 실제로 도는 이미지다. 태그만 보고 배포됐다고 믿지 않는다

Pod 의 생명주기

Pending → Running → Succeeded/Failed. 상태 이름만 봐도 어디까지 왔는지 알 수 있다

- **Running 은 '컨테이너가 떴다'는 뜻이지 '서비스 가능'이 아니다**
- 서비스 가능 여부는 **Ready 조건**이 따로 알려 준다

상태(phase)	무슨 뜻	여기서 멈추면 볼 곳
Pending	아직 노드에 배치되지 않았거나 이미지 받는 중	describe 의 Events — 자원·taint·이미지
Running	노드에 배치되고 컨테이너가 최소 하나 실행 중	Ready 가 아니면 readinessProbe 확인
Succeeded	모든 컨테이너가 정상 종료 (Job 등)	정상
Failed	컨테이너가 0 이 아닌 코드로 종료	logs --previous 로 원인
Unknown	노드와 통신 불가	노드 상태 확인 — NotReady 인가
Ready 조건	**프로브를 통과해 트래픽을 받을 수 있다**	**Endpoints 등록 여부와 직결**
CrashLoopBackOff	(상태가 아니라 컨테이너 사유) 재시작 반복	logs --previous · 재시작 간격 증가
ImagePullBackOff	(사유) 이미지를 못 받는다	태그 오타 · ECR 권한 · 플랫폼 불일치

[주의] Running 인데 서비스가 503 이면 거의 항상 Ready 가 아니다. `kubectl get pod` 의 READY 열(1/1)을 본다

Pod 상태 이름 읽는 법

STATUS 열에 나오는 이름이 곧 어느 단계에서 멈췄는지를 말해 준다

- **Pending 이면 컨테이너가 아직 없다.** `logs` 를 쳐도 나올 것이 없다

STATUS	무슨 뜻	먼저 볼 곳
Pending	노드 배정 전 · 자원 부족	`describe` 의 `0/N nodes`
ContainerCreating	볼륨·설정을 준비 중	`describe` Events (ConfigMap·PVC)
ImagePullBackOff	이미지를 못 받음	Events 의 `unauthorized`/`not found`
Running (READY 1/1)	정상	—
Running (READY 0/1)	readiness 실패	`get endpoints` · 프로브 경로
CrashLoopBackOff	떴다 죽기를 반복	`logs --previous`
Completed	정상 종료 (Job 은 정상)	Deployment 면 비정상
Evicted	노드 자원 부족으로 축출	QoS 클래스 · `top nodes`
Terminating (오래)	종료가 안 끝남	finalizer · 볼륨 detach

[정리] BackOff 가 붙으면 재시도 간격이 점점 길어진다는 뜻이다. 5초, 10초, 20초... 최대 5분. 그래서 나중에는 느리게 회복된다

초기화 컨테이너

본 컨테이너보다 먼저, 순서대로, 끝까지 성공해야 다음으로 넘어간다

- **의존성이 준비될 때까지 기다리는 일**을 앱 코드가 아니라 매니페스트가 한다

```
spec:
  initContainers:
  - name: wait-for-db                  # ① 먼저 실행 · 성공해야 다음
    image: busybox:1.36
    command: ['sh','-c','until nc -z db 5432; do sleep 2; done']
  - name: migrate                      # ② 그 다음
    image: skala-registry.skala-ai.com/class-6/migrator:1.0.0
    command: ['./migrate.sh']
  containers:                          # ③ init 이 전부 끝나야 시작
  - name: app
    image: skala-registry.skala-ai.com/class-6/shop-P000:1.0.0

# 관찰
#   kubectl get pod        → STATUS: Init:0/2 → Init:1/2 → Running
#   kubectl logs my-pod -c wait-for-db      # init 로그는 -c 로
```

[주의] init 이 실패하면 **Pod 전체가 재시작**된다. 그래서 init 은 반드시 **여러 번 실행해도 안전(멱등)** 해야 한다

프로브 세 가지는 역할이 다르다

살아 있는가(liveness), 받을 수 있는가(readiness), 다 떴는가(startup). 셋은 하는 일이 다르다

- **readiness 는 트래픽을 끊고, liveness 는 컨테이너를 죽인다. 결과가 완전히 다르다**
- 느리게 뜨는 앱은 **startupProbe** 로 초기 유예를 준다

프로브	실패하면	언제 쓰나	잘못 쓰면
readinessProbe	Endpoints 에서 빠져 트래픽 중단	기동 중·과부하·의존성 장애	없으면 준비 전에 503
livenessProbe	**컨테이너를 죽이고 재시작**	데드락 등 스스로 못 푸는 상태	너무 민감하면 무한 재시작
startupProbe	컨테이너 재시작	기동이 오래 걸리는 앱	없으면 liveness 가 기동 중 죽인다
공통 옵션			
initialDelaySeconds	첫 검사까지 대기	기동 시간 고려	짧으면 기동 중 실패 처리
periodSeconds	검사 주기 (기본 10초)		짧으면 부하, 길면 감지 지연
failureThreshold	몇 번 실패해야 실패로 볼지	기본 3	1 이면 일시적 지연에도 재시작된다
timeoutSeconds	응답 대기 (기본 1초)		**1초는 짧다.** JVM 은 늘려야 한다

[주의] liveness 에 **DB 연결 확인을 넣지 마라.** DB 가 잠깐 흔들리면 모든 Pod 가 동시에 재시작해 장애가 커진다

프로브 설정 예시

Spring Boot 앱 기준으로 세 프로브를 어떻게 사용하는지 본다. 13장에서 그대로 쓸 설정이다

- startup 이 성공할 때까지 **liveness·readiness 는 아예 실행되지 않는다**

```yaml
containers:
  - name: app
    image: shop-api:1.0.0
    startupProbe:                    # 기동 유예: 최대 5×30 = 150초
      httpGet: { path: /actuator/health/liveness, port: 8080 }
      periodSeconds: 5
      failureThreshold: 30
    livenessProbe:                   # 프로세스가 살아 있는가 (가볍게)
      httpGet: { path: /actuator/health/liveness, port: 8080 }
      periodSeconds: 10
      timeoutSeconds: 3
      failureThreshold: 3
    readinessProbe:                  # 트래픽을 받아도 되는가 (의존성 포함)
      httpGet: { path: /actuator/health/readiness, port: 8080 }
      periodSeconds: 5
      timeoutSeconds: 3
      failureThreshold: 3
```

[정리] startupProbe 가 있으면 initialDelaySeconds 를 안 써도 된다. 기동이 빠른 날은 그만큼 빨리 서비스에 들어간다

종료는 어떻게 일어나는가

Pod 삭제는 즉시가 아니다. 트래픽을 끊고, SIGTERM을 보내고, 유예 시간을 준 뒤 강제 종료한다

- **Endpoints 제거와 SIGTERM 이 거의 동시에 일어난다. 그래서 잠깐 502가 날 수 있다**
- `terminationGracePeriodSeconds` (기본 30초) 안에 안 끝나면 SIGKILL 이다

순서	무슨 일	우리가 할 일
①	Pod 에 삭제 표시(deletionTimestamp)	-
②	Endpoints 에서 제거 — 새 트래픽 차단	(비동기라 지연이 있다)
③	preStop 훅 실행 (설정했다면)	**sleep 5** 로 ②의 전파를 기다린다
④	컨테이너에 SIGTERM 전달	앱이 graceful shutdown 시작
⑤	처리 중인 요청을 마무리	Spring Boot: shutdown=graceful
⑥	grace period 종료 시 SIGKILL	긴 작업은 period 를 늘린다

[주의] ②가 **비동기**라서 이미 라우팅된 요청이 남는다. `preStop: sleep 5` 가 502 를 없애는 가장 간단한 해법이다

재시작 정책과 백오프

언제 다시 실행할지는 정책이 정한다. 얼마나 기다릴지는 클러스터가 정한다

- **Deployment 의 Pod 는 항상** `Always` 이다. 다른 값이 필요하면 Job 을 사용한다

restartPolicy	언제 재시작	어디에 쓰나
Always (기본)	종료 코드와 무관하게 항상	Deployment · 서버 앱
OnFailure	**0 이 아닌 코드**로 끝났을 때만	Job · 배치
Never	재시작하지 않음	일회성 · 디버깅

[체크] `RESTARTS` 열에 `(5m ago)` 처럼 시각이 함께 나온다. 방금인지 어제인지가 진단을 가른다

멀티 컨테이너 패턴

Pod에 컨테이너를 여러 개 두는 경우는 정해져 있다. 세 가지 패턴만 알면 충분하다

- '앱 두 개를 한 Pod 에' 는 **거의 항상 잘못된 설계**다. 따로 배포·확장할 수 없어진다

패턴	무엇	예	판단
사이드카	앱을 돕는 보조 컨테이너	로그 수집, 프록시, 인증서 갱신	가장 흔하다
앰배서더	외부 연결을 대신 맡는다	DB 프록시, 서비스 메시 프록시	앱 코드를 안 고쳐도 된다
어댑터	출력 형식을 바꿔 준다	앱 지표를 Prometheus 형식으로	레거시 앱에 유용
안티패턴	**독립적인 두 앱을 한 Pod 에**	API + 배치 워커	**각각 Deployment 로 분리**

[실무] Istio 같은 서비스 메시는 **사이드카를 자동 주입**한다. 그래서 Pod 를 보면 컨테이너가 2/2 로 나온다

Pod 가 자기 정보를 아는 법

Pod 이름·노드 이름·IP를 앱이 알아야 할 때가 있다. 환경변수로 주입할 수 있다

- **로그에 Pod 이름을 찍어 두면** 어느 복제본에서 난 문제인지 즉시 안다

```
env:
  - name: POD_NAME
    valueFrom: { fieldRef: { fieldPath: metadata.name } }
  - name: POD_NAMESPACE
    valueFrom: { fieldRef: { fieldPath: metadata.namespace } }
  - name: NODE_NAME
    valueFrom: { fieldRef: { fieldPath: spec.nodeName } }
  - name: POD_IP
    valueFrom: { fieldRef: { fieldPath: status.podIP } }
  - name: MEM_LIMIT                    # 리소스 값도 가져올 수 있다
    valueFrom:
      resourceFieldRef: { containerName: app, resource: limits.memory }

# Spring Boot 에서 쓰기 (application.yml)
# logging.pattern.level: "%5p [${POD_NAME:local}]"
# management.metrics.tags.pod: ${POD_NAME:local}
```

[실무] 지표에 **Pod 이름 태그**를 붙이면 '특정 Pod 만 느리다' 같은 문제를 그래프에서 바로 본다

Pod 를 직접 만들면 안 되는 이유

Pod는 스스로를 지키지 않는다. 노드에 장애가 나면 그대로 사라지고 아무도 다시 만들지 않는다

- 관리자 없는 Pod 를 네이키드 Pod(naked pod) 라고 부른다. 운영에 두지 않는다

방식	노드 장애 시	복제·확장	롤링 업데이트	쓸 곳
Pod 직접	**사라지고 끝**	불가	불가	일회성 디버깅
ReplicaSet	다시 만든다	가능	불가	직접 쓰지 않는다
Deployment	다시 만든다	가능	**가능**	**상태 없는 앱 전부**
StatefulSet	다시 만든다	가능	가능(순서대로)	DB · 큐 등 상태 보유
DaemonSet	노드마다 하나	노드 수만큼	가능	로그·모니터링 에이전트
Job / CronJob	재시도	병렬 가능	-	배치 작업

[정리] 실무에서 만드는 것은 **Deployment 이고 Pod 는 그 결과**이다. Pod 를 직접 만드는 것은 디버깅용 임시 컨테이너뿐이다

Pod 디버깅 도구 상자

Pod가 이상할 때 순서대로 칠 명령들. 이 순서가 진단 시간을 절반으로 줄인다

- **상태 → 이벤트 → 로그 → 안으로** 순서를 지킨다. 건너뛰면 헛다리를 짚는다

```
# ① 지금 상태 — READY 열과 RESTARTS 열을 본다
kubectl get pod -o wide

# ② 왜 그런지 — Events 가 답을 준다
kubectl describe pod <이름> | tail -30

# ③ 앱이 뭐라고 하는지 (컨테이너가 떴을 때만)
kubectl logs <이름> --tail=100
kubectl logs <이름> --previous          # 재시작했다면 필수

# ④ 안에 들어가 확인 — 설정·연결·파일
kubectl exec -it <이름> -- sh
kubectl exec <이름> -- env | sort

# ⑤ 셸이 없는 이미지(distroless)라면 임시 컨테이너를 붙인다
kubectl debug -it <이름> --image=busybox:1.36 --target=app
```

[실무] `kubectl debug` 는 **원본 컨테이너를 건드리지 않고** 도구 컨테이너를 Pod 에 끼워 넣는다. 1.25 부터 정식 기능이다

[핵심 요약] Pod

7장에서 남길 것은 Pod 를 다룰 때의 판단 기준이다

항목	한 줄 정리	실무 포인트
Pod 란	네트워크·볼륨을 공유하는 컨테이너 묶음	IP 가 하나. 포트를 나눠 쓴다
Running vs Ready	떴다 ≠ 받을 수 있다	READY 0/1 이면 503 의 원인
initContainer	먼저·순서대로·끝나야 다음	마이그레이션은 복제본 수만큼 동시 실행
readinessProbe	실패 → 트래픽만 끊는다	의존성 확인을 여기에
livenessProbe	실패 → 컨테이너를 죽인다	**DB 확인을 넣으면 재시작 폭풍**
startupProbe	기동 유예. 준비되면 즉시 넘어간다	initialDelay 보다 낫다
종료	Endpoints 제거가 비동기라 502 가 난다	preStop sleep 5 로 해결
멀티 컨테이너	사이드카·앰배서더·어댑터만	따로 확장해야 하면 분리한다
직접 만들기	네이키드 Pod 는 노드와 함께 사라진다	운영은 전부 Deployment

[체크] 우리 앱의 liveness 경로가 **DB 를 확인하고 있지는 않은가?** Spring Boot 기본 `/actuator/health` 는 확인한다

8. Deployment와 롤아웃

- ReplicaSet 과의 관계 · 롤링 업데이트 · 롤백 · 무중단 배포

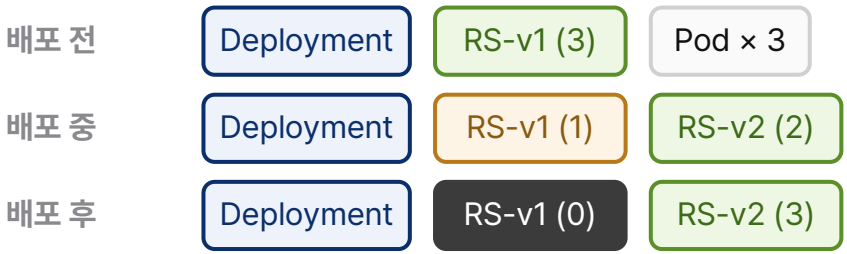
본 문서는 SK AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 AX

Deployment 와 ReplicaSet

Deployment는 버전을 관리하고, ReplicaSet은 개수를 관리한다. 층이 나뉜 이유가 곧 롤백의 원리다

- 새 버전을 배포하면 **새 ReplicaSet 이 생기고 옛것은 0으로 줄어든다**
- 옛 ReplicaSet 이 남아 있기 때문에 **롤백이 가능**하다



Deployment
버전(리비전) 관리
롤아웃 전략 결정
롤백 지시

ReplicaSet
라벨로 Pod 수를 센다
부족하면 만들고
남으면 지운다

Pod
실제 컨테이너
이름이 매번 바뀐다
조정 대상

[정리] 옛 ReplicaSet 은 지워지지 않고 replicas=0 으로 남는다. 그게 곧 롤백 가능한 이력이다

세 계층을 눈으로 확인한다

Deployment 가 ReplicaSet 을 만들고, ReplicaSet 이 Pod 를 만든다. 이름에 그 흔적이 남는다

- Pod 이름의 가운데 토막이 ReplicaSet 해시이다. 버전이 바뀌면 이 값이 바뀐다

```
kubectl get deploy,rs,pod -l app=shop-$ME

# NAME                           READY   UP-TO-DATE   AVAILABLE
# deployment.apps/shop-P000    3/3     3            3
#
# NAME                                      DESIRED  CURRENT  READY
# replicaset.apps/shop-P000-7d9f8c4b     3        3        3      ← 지금 것
# replicaset.apps/shop-P000-5c8a2f1d     0        0        0      ← 이전 버전
#
# NAME                                 READY   STATUS
# pod/shop-P000-7d9f8c4b-x2k9p    1/1     Running
#                    └── ReplicaSet 해시가 Pod 이름에 들어 있다

# 새 버전을 배포하면 ReplicaSet 이 하나 더 생긴다 — 그것이 롤백의 재료다
kubectl rollout history deploy/shop-$ME
```

[정리] 이전 ReplicaSet 을 0 으로 남겨 두는 것이 롤백의 정체이다. `revisionHistoryLimit` 이 몇 개를 남길지 정한다

Deployment 매니페스트

Pod 스펙을 template 안에 넣고, 그 바깥에 개수와 전략을 적는다

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shop-api                    # selector 와 template.labels 는 반드시 일치
spec:
  replicas: 3
  revisionHistoryLimit: 5           # 롤백용 이력 보관 개수 (기본 10)
  selector:
    matchLabels: { app: shop-api } # 불변 — 만든 뒤 못 바꾼다
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1                   # 목표보다 최대 1개 더 만들어도 된다
      maxUnavailable: 0             # 사용 가능한 개수는 줄이지 않는다
  template:                         # 여기부터가 Pod 스펙 (7장)
    metadata:
      labels: { app: shop-api }
    spec:
      containers:
        - { name: app, image: shop-api:1.0.0 }
```

[실무] `maxUnavailable: 0` 이면 **용량이 절대 줄지 않는다.** 트래픽이 많은 서비스의 기본값으로 삼을 만하다

롤링 업데이트는 어떻게 도나

새 Pod를 하나 띄우고 Ready를 기다린 뒤 옛 Pod를 하나 내린다. 이 반복이다

- **Ready 를 기다리는 것이 핵심**이다. 프로브가 없으면 안 뜬 Pod 로 트래픽이 간다

단계	새 RS	옛 RS	서비스 가능	무슨 일
시작	0	3	3	배포 명령
①	1(기동중)	3	3	maxSurge=1 만큼 추가
②	1(Ready)	3	4	**readinessProbe 통과**
③	1	2	3	옛 Pod 하나 종료
④	2(Ready)	2	4	다음 새 Pod
⑤	2	1	3	옛 Pod 종료
⑥	3(Ready)	1	4	마지막 새 Pod
완료	3	0	3	옛 RS 는 0 으로 남는다

[주의] readinessProbe 가 없으면 쿠버네티스는 **컨테이너가 뜨자마자 Ready 로 본다.** 기동 중인 Pod 에 트래픽이 간다

롤아웃 지켜보기

배포는 비동기이다. 명령이 끝나도 배포는 진행 중이다. 끝났는지 확인하는 방법이 따로 있다

- `rollout status` 는 완료될 때까지 기다린다. 배포 스크립트에 반드시 넣는다

```
# 배포 (이미지 태그만 바꾼다)
kubectl set image deploy/shop-api app=shop-api:1.0.1
# 또는 매니페스트를 고치고
kubectl apply -f k8s/deployment.yaml

# 완료까지 기다린다 — 실패하면 0 이 아닌 코드로 끝난다
kubectl rollout status deploy/shop-api --timeout=180s

# 다른 창에서 진행 상황을 지켜본다
kubectl get pod -w
kubectl get rs -w                       # ReplicaSet 개수 변화

# 이력 확인
kubectl rollout history deploy/shop-api
kubectl rollout history deploy/shop-api --revision=3
```

[실무] CI 는 `rollout status` 로 **성공 여부를 판단**해야 한다. apply 만 하고 끝내면 실패한 배포를 성공으로 보고한다

배포가 멈췄을 때

롤아웃이 진행되지 않고 멈추는 경우가 있다. 다행히 이때는 서비스가 아직 안전하다

- 새 Pod 가 Ready 가 안 되면 롤아웃은 거기서 멈춘다. 옛 Pod 는 그대로 남아 있다
- 이것이 `maxUnavailable: 0` 의 두 번째 이점이다. **실패해도 용량이 줄지 않는다**

증상	원인	확인	조치
새 Pod 가 Pending	노드 자원 부족	describe 의 Events	노드 증설 · requests 조정
ImagePullBackOff	태그 오타 · ECR 권한	describe 의 Failed to pull	태그 확인 · 권한 확인
CrashLoopBackOff	앱 기동 실패	logs --previous	설정·환경변수 확인
Running 인데 0/1	readinessProbe 실패	describe 의 Readiness probe failed	경로·포트·유예시간
아주 느리게 진행	startupProbe 유예가 길다	get pod -w	정상. 기다린다
progress deadline 초과	10분 넘게 진행 없음	rollout status 가 실패 반환	**undo 로 되돌린다**

[정리] 멈춘 배포는 장애가 아니다. 옛 버전이 계속 서비스 중이므로, 급하게 손대기보다 원인을 확인하고 되돌린다

되돌릴 때 무엇이 돌아오나

undo 한 줄이면 직전 버전으로 돌아간다. 다만 되돌아가는 것은 매니페스트이지 데이터가 아니다

- 롤백은 옛 ReplicaSet 을 다시 키우는 일이다. 그래서 빠르다
- **DB 스키마 변경은 롤백되지 않는다.** 앱만 옛 버전이 되면 오히려 깨진다

```
# 이력 보기 — CHANGE-CAUSE 가 비어 있으면 무엇이 바뀐지 모른다
kubectl rollout history deploy/shop-api

# 직전 버전으로 되돌리기 (가장 흔한 경우)
kubectl rollout undo deploy/shop-api

# 특정 리비전으로
kubectl rollout undo deploy/shop-api --to-revision=3

# 이력에 이유를 남기려면 애노테이션을 쓴다
kubectl annotate deploy/shop-api \
  kubernetes.io/change-cause="1.0.1 결제 타임아웃 수정" --overwrite

# 진행 중인 롤아웃을 잠깐 세우고 확인하기
kubectl rollout pause deploy/shop-api
kubectl rollout resume deploy/shop-api
```

[주의] 롤백해도 **Git 은 그대로**이다. 다음 배포에서 문제의 버전이 다시 올라간다. 코드도 함께 되돌려야 한다

롤아웃 이력 다루기

무엇이 언제 왜 바뀌었는지 남겨 두면 되돌릴 때 고민이 없다

- **change-cause 를 안 남기면 이력이 <none> 뿐**이라 어느 리비전으로 갈지 알 수 없다

```
# 이력 보기
kubectl rollout history deploy/shop-$ME
# REVISION  CHANGE-CAUSE
# 1         1.0.0 최초 배포
# 2         1.1.0 결제 버그 수정
# 3         <none>          ← 기록을 안 남긴 배포

# 특정 리비전 상세 — 무엇이 달랐는지
kubectl rollout history deploy/shop-$ME --revision=2

# 되돌리기
kubectl rollout undo deploy/shop-$ME                  # 직전으로
kubectl rollout undo deploy/shop-$ME --to-revision=2  # 콕 집어서

# 기록 남기는 습관 — 배포 직후 한 줄
kubectl annotate deploy/shop-$ME \
  kubernetes.io/change-cause="1.2.0 — 재고 조회 캐시 추가" --overwrite
```

[실무] 커밋 메시지와 SHA 를 change-cause 로 넣어 두면 롤백 판단이 몇 초로 줄어든다

스케일 조정

개수를 바꾸는 방법은 세 가지이다. 손으로, 파일로, 자동으로

- HPA 를 쓸 때 매니페스트에 **replicas 를 적어 두면 충돌**한다

```
# ① 손으로 — 급할 때
kubectl scale deploy/shop-api --replicas=6

# ② 파일로 — 운영에 남기는 방법
#    deployment.yaml 의 replicas 를 고치고
kubectl apply -f k8s/deployment.yaml

# ③ 자동으로 — HPA (심화 과정)
kubectl autoscale deploy/shop-api --min=3 --max=10 --cpu-percent=70

# 현재 상태 확인
kubectl get deploy shop-api
# NAME       READY   UP-TO-DATE   AVAILABLE   AGE
# shop-api   6/6     6            6           2d
#            ↑실제/목표  ↑새버전    ↑Ready 개수
```

[주의] HPA 가 붙은 Deployment 의 매니페스트에 `replicas: 3` 이 있으면 **apply 할 때마다 3으로 되돌아간다**

배포 중 트래픽을 잃지 않으려면

무중단 배포는 하나의 설정이 아니라 다섯 가지가 다 맞아야 이루어진다

- 하나라도 빠지면 배포할 때마다 소량의 에러가 난다. 눈에 잘 안 띄어서 더 위험하다

필요한 것	없으면	설정
readinessProbe	기동 중인 Pod 로 트래픽이 간다	7장 — /actuator/health/readiness
maxUnavailable: 0	배포 중 용량이 줄어 지연·오류	strategy.rollingUpdate
preStop sleep	종료 중인 Pod 로 트래픽이 남는다	lifecycle.preStop — sleep 5
graceful shutdown	처리 중이던 요청이 끊긴다	13장 — server.shutdown=graceful
복제본 2개 이상	교체하는 동안 서비스가 빈다	replicas ≥ 2
PodDisruptionBudget	**노드 축소 시 한꺼번에 내려간다**	심화 — minAvailable

[체크] 우리 서비스는 배포 중 **5xx 가 정말 0인가?** 초 단위 지표로 봐야 보인다. 분 단위 평균에는 안 나타난다

배포 명령을 한 줄로 비교

같은 결과를 내는 명령이 여럿이다. 무엇이 기록에 남는지가 다르다

- **운영은 파일 + apply**, 실습·긴급은 나머지. 기준을 이렇게 정한다

하고 싶은 것	빠른 방법	권장 (기록이 남는다)
이미지 교체	`set image deploy/x app=...`	**파일 수정 후 `apply`**
복제본 변경	`scale --replicas=5`	**파일의 `replicas` 수정**
설정만 바꿔 재시작	`rollout restart`	ConfigMap 해시 애노테이션
일부 필드만	`patch -p '{...}'`	파일 수정 후 `apply`
즉석 수정	`edit deploy/x`	**쓰지 않는다**
되돌리기	`rollout undo`	**이전 커밋으로 `apply`**

[정리] 빠른 방법으로 고친 것은 **다음 `apply` 에서 되돌아간다.** 급해서 썼다면 반드시 파일에도 반영한다

[핵심 요약] Deployment와 롤아웃

8장에서 남길 것은 배포를 안전하게 만드는 설정과 판단이다

항목	한 줄 정리	실무 포인트
구조	Deployment → ReplicaSet → Pod	옛 RS 가 0으로 남아 롤백이 가능하다
maxSurge	목표보다 몇 개 더 만들 수 있나	1 이면 잠깐 4개까지
maxUnavailable	용량이 몇 개까지 줄어도 되나	**0 이면 절대 안 줄어든다**
롤링 업데이트	새 Pod Ready 확인 후 옛 Pod 종료	프로브가 없으면 무중단이 아니다
멈춘 배포	옛 버전이 살아 있어 서비스는 정상	당황하지 말고 원인 확인 후 undo
롤백	옛 ReplicaSet 을 다시 키우는 일	Git 도 함께 되돌린다. DB 는 안 돌아간다
rollout status	완료까지 기다리고 결과를 알린다	CI 는 이걸로 성공 판단
Strategy 선택	기본은 RollingUpdate	RWO 볼륨이면 Recreate
스케일	HPA 를 쓰면 replicas 를 빼 둔다	안 그러면 apply 마다 되돌아간다

[체크] 우리 Deployment 에 **readinessProbe · maxUnavailable · preStop** 세 가지가 다 있는가?

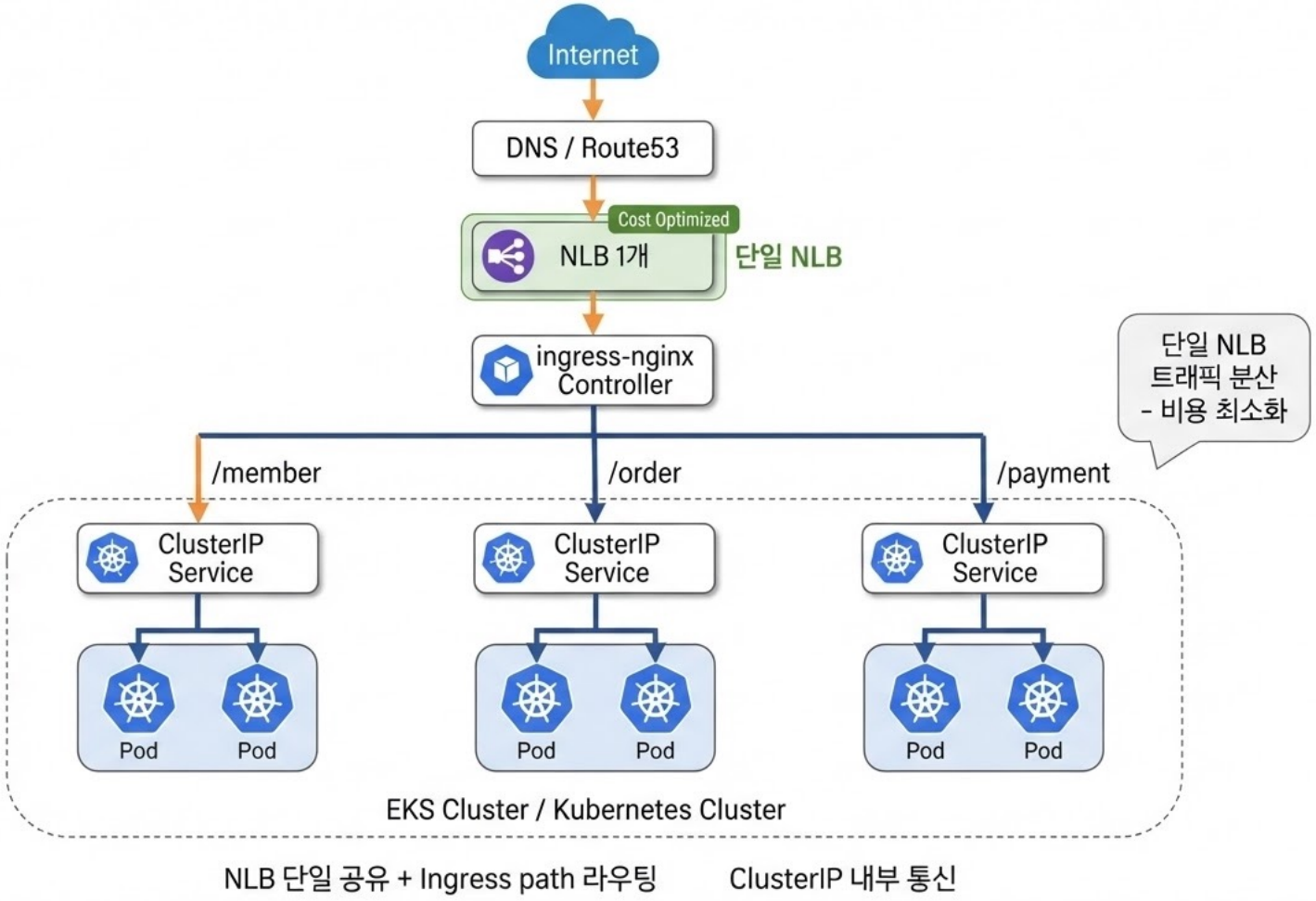
9. Service와 클러스터 네트워킹

- Pod IP 의 문제 · Endpoints · Service 종류 · DNS

본 문서는 SK AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 AX

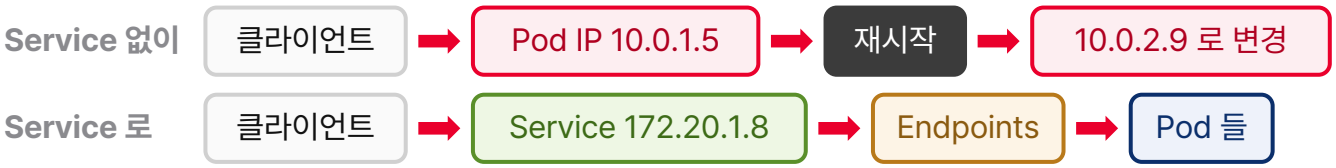
AWS 기반 아키텍처 구성



Pod IP 는 왜 못 쓰나

Pod IP는 Pod가 다시 만들어질 때마다 바뀐다. 그래서 그 앞에 고정된 주소가 필요하다

- Pod 는 재시작·스케일·배포 때마다 IP 가 바뀐다. 설정에 적어 둘 수 없다
- Service 는 **클러스터 수명 동안 안 바뀌는 가상 IP** 를 하나 갖는다



Pod IP

Pod 마다 하나
재생성 시 변경
VPC CNI 는 실제 VPC IP

Service IP(ClusterIP)

생성 시 할당, 삭제까지 유지
어느 노드에도 없는 가상 IP
kube-proxy 규칙으로 동작

Endpoints

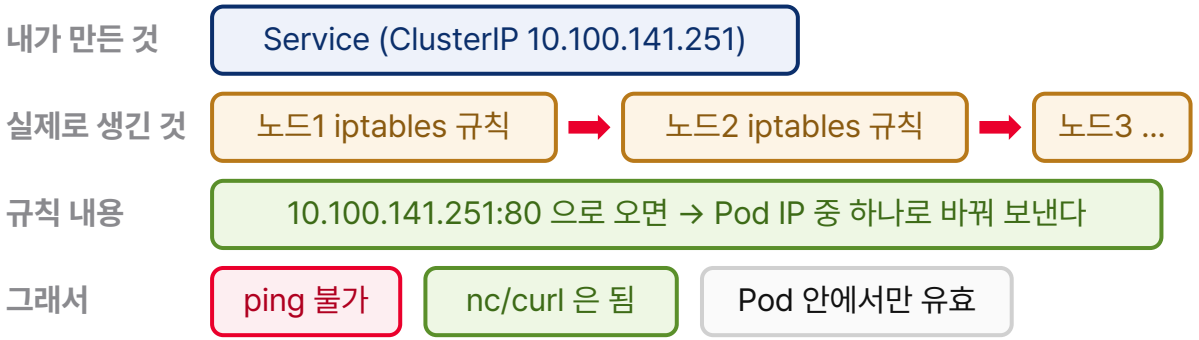
Service 뒤의 실제 Pod 목록
Ready 인 Pod 만 등록
자동으로 갱신

[정리] Service는 Pod의 변경과 무관하게 일관된 접근 주소를 제공하며, 해당 Service에 연결된 여러 Pod로 트래픽을 분산한다.

Service 에는 실체가 없다

ClusterIP 로 ping 이 안 되는 것이 정상이다. 그 IP 로 Listen 하고 있는 프로세스가 없다

- Service 는 각 노드 iptables 에 박힌 규칙이다. 어떤 서버도 그 IP 를 갖고 있지 않다



[체크] 확인은 `ping` 이 아니라 **`curl` 이나 `nc`** 로 한다. `kubectl run` 으로 임시 Pod 를 띄워 클러스터 안에서 친다

Service 매니페스트와 Endpoints

Service는 라벨로 Pod를 찾고, 찾은 결과가 Endpoints에 기록된다. 여기가 비면 연결이 안 된다

- **Endpoints 가 비어 있으면 503** 이다. 원인은 라벨 불일치 아니면 Ready 아님

```yaml
apiVersion: v1
kind: Service
metadata:
  name: shop-api                # 이 이름이 곧 DNS 이름이 된다
spec:
  type: ClusterIP               # 기본값
  selector:
    app: shop-api               # ← 이 라벨을 가진 Ready Pod 를 찾는다
  ports:
    - name: http
      port: 80                  # Service 가 여는 포트
      targetPort: 8080          # Pod 의 포트 (이름도 가능)
---
# 확인 — 여기에 IP 가 나와야 정상
# kubectl get endpoints shop-api
# NAME       ENDPOINTS                                         AGE
# shop-api   10.0.1.5:8080,10.0.2.9:8080,10.0.3.4:8080   5m
```

[실무] 연결이 안 되면 **`kubectl get endpoints` 부터** 본다. 비어 있으면 Service 문제, 차 있으면 앱·네트워크 문제

Endpoints 는 규칙의 오른쪽

Service 를 만든다고 연결되는 게 아니다. Endpoints 에 Pod 가 올라와야 연결된다

- **Endpoints 가 비면 연결이 거부된다. 타임아웃이 아니라 즉시 거부라 구분된다**

```
kubectl get svc shop-$ME
# NAME         TYPE        CLUSTER-IP       PORT(S)
# shop-P000   ClusterIP   10.100.141.251   80/TCP

kubectl get endpoints shop-$ME
# NAME         ENDPOINTS
# shop-P000   172.31.6.2:8080,172.31.6.7:8080,172.31.12.9:8080

# 상세 — 어느 Pod 인지까지 나온다
kubectl get endpoints shop-$ME -o yaml | grep -A4 targetRef

# 비어 있다면 원인은 둘 중 하나다
kubectl describe svc shop-$ME | grep Selector   # ① selector 오타
kubectl get pods -l app=shop-$ME                # ② Ready 인 Pod 가 없다
```

[정리] <none> 이면 Service 문제가 아니라 뒤쪽 문제이다. selector 를 대조하거나 Pod 의 READY 를 본다

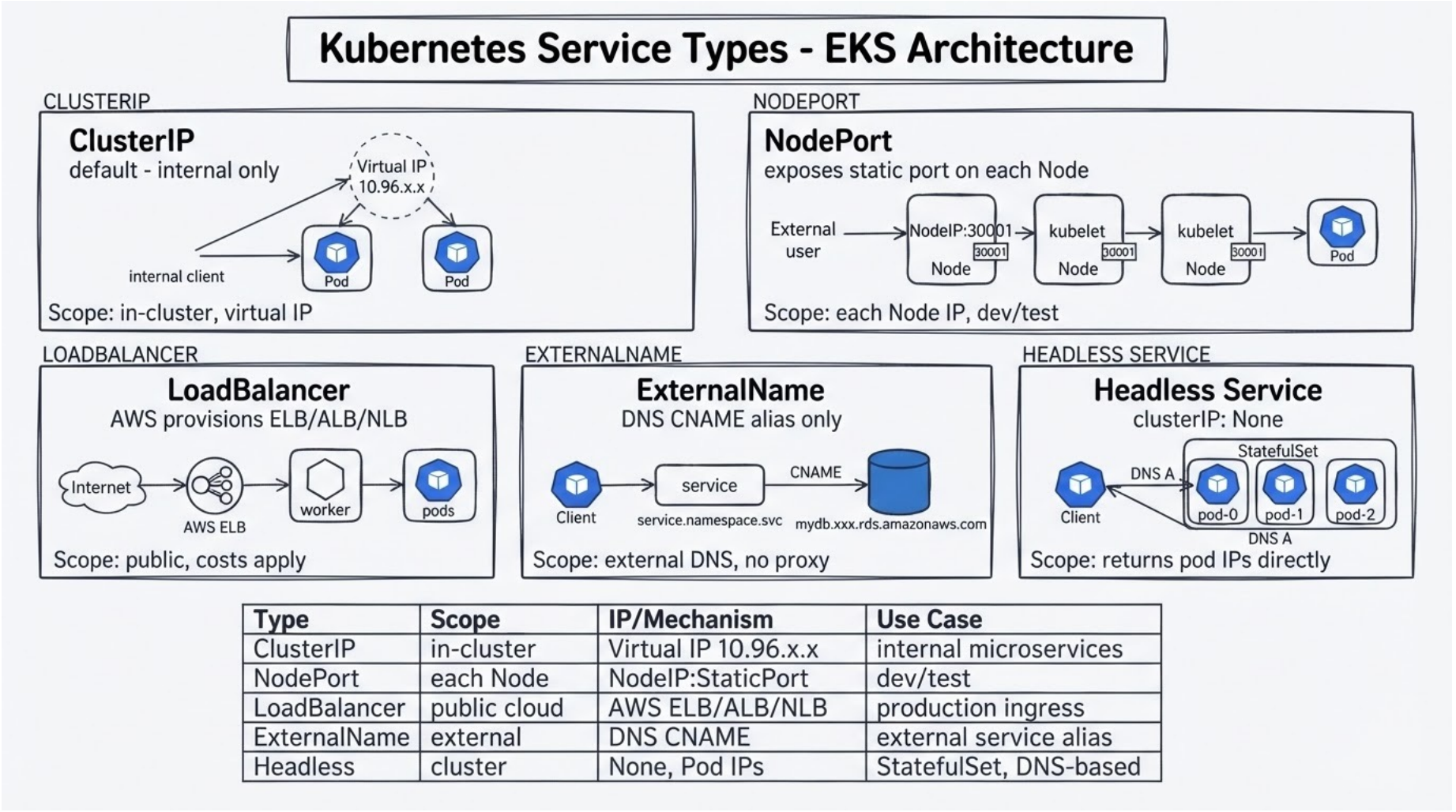
Service 의 종류

어디까지 노출할 것인가에 따라 고른다. 실무에서 보통 사용되는 것은 ClusterIP, LoadBalancer, Headless 이다.

- **NodePort 는 운영에서 잘 안 쓴다.** 포트 범위가 30000~32767 로 제한되고 노드 IP 를 알아야 한다

종류	접근 범위	어떻게 동작	실무
ClusterIP	클러스터 안에서만	가상 IP + kube-proxy 규칙	**기본값. 대부분 이것**
NodePort	노드 IP:30000~32767	모든 노드에 포트를 연다	테스트·온프레미스 진입점
LoadBalancer	외부	클라우드 LB 자동 생성 (EKS: NLB)	Service 하나당 LB 하나 — 비싸다
ExternalName	-	DNS CNAME 을 만들어 준다	외부 RDS 를 이름으로 부를 때
Headless	클러스터 안	clusterIP: None — Pod IP 를 직접 반환	StatefulSet · 클라이언트 LB 주요 저장소 대상

Service 의 종류



Type	Scope	IP/Mechanism	Use Case
ClusterIP	in-cluster	Virtual IP 10.96.x.x	internal microservices
NodePort	each Node	NodeIP:StaticPort	dev/test
LoadBalancer	public cloud	AWS ELB/ALB/NLB	production ingress
ExternalName	external	DNS CNAME	external service alias
Headless	cluster	None, Pod IPs	StatefulSet, DNS-based

NodePort 는 30000번대에서 포트를 받는다

모든 노드의 같은 포트를 열어 둔다. 어느 노드로 들어와도 같은 서비스로 간다

- **클라우드 로드밸런서가 붙는 자리가 여기이다. LoadBalancer 타입은 NodePort 위에 구성된다**

```yaml
apiVersion: v1
kind: Service
metadata: { name: shop-nodeport }
spec:
  type: NodePort
  selector: { app: shop-P000 }
  ports:
    - port: 80          # ClusterIP 의 포트
      targetPort: 8080  # Pod 의 포트
      nodePort: 30975   # 노드에서 열리는 포트 (30000~32767)
---
# 세 포트가 다 다르다 — 순서대로 이어진다
#   <노드IP>:30975  →  <ClusterIP>:80  →  <PodIP>:8080
#
# kubectl get svc shop-nodeport
#   TYPE       CLUSTER-IP       PORT(S)
#   NodePort   10.100.141.251   80:30975/TCP
```

[주의] 모든 노드에서 열린다. Pod 가 없는 노드로 들어와도 iptables 가 다른 노드의 Pod 로 넘겨 준다

클러스터 DNS

Service를 만들면 이름이 자동으로 등록된다. 앱 설정에는 IP가 아니라 이 이름을 쓴다

- 같은 네임스페이스면 **서비스 이름만으로** 부를 수 있다
- 다른 네임스페이스는 **이름.네임스페이스** 를 붙인다

```
# 전체 이름 규칙
<서비스이름>.<네임스페이스>.svc.cluster.local

# 어디서 부르느냐에 따라 줄여 쓸 수 있다
같은 네임스페이스     →  http://shop-api:80
다른 네임스페이스     →  http://shop-api.class-6:80
완전한 이름          →  http://shop-api.class-6.svc.cluster.local

# Spring Boot 설정 예 (application-prod.yml)
spring:
  datasource:
    url: jdbc:mysql://mysql:3306/shop      # ← Service 이름
  data:
    redis: { host: redis }                 # ← Service 이름
# 확인 — 클러스터 안에서 DNS 조회
kubectl run dnstest --rm -it --image=busybox:1.36 -- nslookup shop-api
```

[실무] 앱 설정에는 **반드시 Service 이름**을 쓴다. IP 를 적으면 Service 를 다시 만들 때 전부 고쳐야 한다

CoreDNS 가 이름을 주소로 바꾼다

IP 는 바뀌지만 이름은 안 바뀐다. 그래서 코드에는 이름을 적는다

- 같은 네임스페이스면 **이름만**, 다른 네임스페이스면 이름.네임스페이스

```
# 클러스터 안에서 확인 (netshoot 로 임시 Pod)
kubectl run nettest --rm -it --image=nicolaka/netshoot -- bash

  nslookup shop-P000
  #   Name: shop-P000.class-6.svc.cluster.local
  #   Address: 10.100.141.251        ← ClusterIP 가 나온다

  # 짧은 이름이 되는 이유 — resolv.conf 의 search 목록
  cat /etc/resolv.conf
  #   search class-6.svc.cluster.local svc.cluster.local cluster.local
  #   nameserver 10.100.0.10          ← CoreDNS

  curl -s http://shop-P000/api/info                       # 같은 NS
  curl -s http://shop-P000.class-6/api/info              # NS 명시
  curl -s http://shop-P000.class-6.svc.cluster.local/    # 전체 이름
```

[주의] search 목록 때문에 **외부 도메인 조회가 느려질 수 있다.** `api.github.com` 을 찾을 때 클러스터 도메인부터 붙여 보고 실패한 뒤에야 밖으로 나간다

EKS 에서의 LoadBalancer Service

type: LoadBalancer 하나로 NLB가 만들어진다. 애노테이션으로 동작을 세밀하게 정한다

- **target-type: ip 를 권장한다. 노드를 거치지 않고 Pod 로 바로 간다**

```yaml
apiVersion: v1
kind: Service
metadata:
  name: shop-api-lb
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-type: external
    service.beta.kubernetes.io/aws-load-balancer-nlb-target-type: ip
    service.beta.kubernetes.io/aws-load-balancer-scheme: internet-facing
    service.beta.kubernetes.io/aws-load-balancer-healthcheck-path: /actuator/health
spec:
  type: LoadBalancer
  selector: { app: shop-api }
  ports:
    - { port: 80, targetPort: 8080 }
# kubectl get svc shop-api-lb
# EXTERNAL-IP 에 주소가 나오기까지 2~3분 걸린다
```

[주의] 이 Service 를 지우면 **NLB 도 함께 삭제**된다. 실습 후 정리하지 않으면 요금이 계속 나간다

세션이 필요할 때

기본은 매 요청마다 다른 Pod 로 간다. 그래서 메모리에 세션을 두면 깨진다

- **진짜 해법은 세션을 밖으로 빼는 것이다. 고정은 임시방편이다**

방법	어떻게	문제
세션을 밖으로	Redis · DB 에 저장	**권장** — 인프라가 하나 는다
JWT 등 무상태	토큰에 정보를 담는다	**권장** — 폐기가 어렵다
Service 고정	`sessionAffinity: ClientIP`	Pod 가 종료되면 세션도 사라진다
Ingress 고정	nginx 쿠키 기반 affinity	부하가 한쪽으로 쏠린다

[실무] 커넥션 재사용 때문에 부하 분산이 안 되는 것처럼 보이기도 한다. keep-alive 가 살아 있으면 계속 같은 Pod 로 간다

Service 로 연결이 안 될 때

문제를 층으로 나눠서 아래에서 위로 확인한다. 추측하지 말고 한 층씩 배제한다

- **Pod → Service → DNS → 외부** 순서로 좁힌다. 건너뛰면 원인을 못 찾는다

```
# 진단용 임시 Pod (클러스터 안에서 확인해야 의미가 있다)
kubectl run nettest --rm -it --image=nicolaka/netshoot -- bash

# ① Pod 에 직접 — 앱이 응답은 하는가
curl -s 10.0.1.5:8080/actuator/health

# ② Service IP 로 — kube-proxy 규칙이 도는가
curl -s 172.20.1.8:80/actuator/health

# ③ Service 이름으로 — DNS 가 되는가
nslookup shop-api
curl -s http://shop-api/actuator/health

# ④ Endpoints 확인 — 비어 있으면 여기가 원인
kubectl get endpoints shop-api
kubectl get pod -l app=shop-api -o wide
```

[실무] ①은 되는데 ②가 안 되면 Service 설정(포트·셀렉터), **②는 되는데 ③이 안 되면** DNS 문제이다

Service 관련 흔한 함정

증상별로 원인을 정리했다. 대부분 라벨, 포트, Ready 셋 중 하나이다

- **연결 안 됨의 90%는 Endpoints 가 비어 있는 것**이고, 그 원인은 둘 뿐이다

증상	원인	확인	조치
503 · 연결 거부	Endpoints 가 비었다	get endpoints	라벨 확인 · Ready 확인
타임아웃	targetPort 가 틀렸다	describe svc 의 TargetPort	Pod 의 실제 포트와 맞춘다
가끔만 실패	Ready 아닌 Pod 가 섞였다	get pod 의 READY 열	readinessProbe 조정
트래픽이 한 Pod 로	커넥션 재사용	Pod 별 요청 수 지표	연결 수명 제한 · 메시
EXTERNAL-IP pending	서브넷 태그 · 권한 부족	describe svc 의 Events	서브넷 태그 확인
다른 NS 에서 안 됨	짧은 이름을 썼다	nslookup 결과	`이름.네임스페이스` 로
배포 중에만 502	종료 중 Pod 로 트래픽	-	preStop sleep (7장)

[체크] 지금 `kubectl get endpoints` 를 쳤을 때 **Pod 개수만큼 IP 가 나오는가?** 하나라도 적으면 이유가 있다

[핵심 요약] Service와 네트워킹

9장에서 남길 것은 연결이 안 될 때 볼 곳의 순서이다

항목	한 줄 정리	실무 포인트
왜 필요한가	Pod IP 는 계속 바뀐다	설정에는 Service 이름을 쓴다
Endpoints	Service 뒤의 실제 Ready Pod 목록	**비어 있으면 라벨 아니면 Ready 문제**
ClusterIP	클러스터 안에서만 쓰는 가상 IP	대부분의 서비스가 이것으로
LoadBalancer	Service 하나당 NLB 하나	고비용, Ingress 로 묶는다(11장)
DNS	이름.네임스페이스.svc.cluster.local	다른 NS 는 두 부분을 쓴다
kube-proxy	규칙만 설치, 패킷은 커널이	분배는 연결 단위 — 풀 쓰면 고정
진단 순서	Pod IP → Service IP → 이름	Pod → Service → DNS 계층을 순차적으로 검증해 정상 구간을 배제하고 장애 지점을 특정한다.
세션	Pod 는 언제든 종료된다	세션은 Redis 나 토큰으로
NetworkPolicy	정책이 붙으면 기본 Deny로 바뀐다	NetworkPolicy 적용 시 허용된 통신만 가능하며, EKS에서는 기능 활성화 여부를 반드시 확인한다.

[체크] 앱 설정에 **Pod IP 나 노드 IP 를 적어 둔 곳이 없는가?** 있으면 지금이 고칠 때이다

10. 호출 흐름 추적

- 여섯 구간 · 구간마다의 운영 권장 · 접근 경로 정하기 · 장애 때의 순서

본 문서는 SK AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 AX

요청 하나가 지나는 길

바깥에서 컨테이너 안 프로세스까지 여섯 구간이다. 구간마다 주소가 바뀌고, 미리 걸어 둘 것도 다르다

색이 위치를 뜻한다. ①② 는 클러스터 밖에서 Pod 까지, ③④ 는 클러스터 안 호출, ⑤⑥ 은 Pod 안이다



외부 유입은 LB 에서 Pod 로 바로 간다. ③④ 는 클러스터 안에서 부를 때 주소가 두 번 바뀌는 구간이다

[정리] 운영에서 가장 자주 끊기는 곳은 ④ 이다. readiness 가 떨어지면 Endpoints 에서 빠지고, 그 순간 트래픽이 멎는다

① 도메인에서 로드밸런서까지

이름을 주소로 바꾸는 구간이다. 클러스터 밖의 일이라 kubectl 로는 안 보인다

- `*.skala-ai.com` 이 **전부 같은 LB 를 가리킨다**. 등록하지 않은 호스트도 LB 까지는 도달한다

```
# 내 호스트가 어디를 가리키나
dig +short shop-P000.skala-ai.com
#   a372be0564d58424b9f503dd696e766a.ap-northeast-2.elb.amazonaws.com.
#   43.201.xx.xx        ← 여러 IP (LB 는 여러 대다)

# 클러스터 쪽에서 본 LB 주소 — 이 둘이 같아야 한다
kubectl get ingress shop-$ME
#   HOSTS                       ADDRESS
#   shop-P000.skala-ai.com     a372be05...elb.amazonaws.com

# 운영에서 이 구간에 걸어 두는 것
#   인증서 만료를 감시한다. 갱신이 실패하면 배포와 무관하게 어느 날 끊긴다
#   레코드 TTL 을 짧게 둔다. LB 를 교체할 때 전환이 빨리 퍼진다
#   미등록 호스트는 Ingress 가 404 로 끊게 둔다. 와일드카드라 LB 까지는 온다
#   밖에서 정기적으로 호출한다. 안에서만 확인하면 ① 구간 장애를 못 본다

# 안 되면 — ADDRESS 가 비었다 (1~2분 기다린다) · dig 결과가 없다 (DNS)
```

[주의] 인증서와 DNS 는 배포와 무관하게 끊긴다. 배포 점검표가 아니라 상시 감시에 둔다

② 로드밸런서에서 Pod 까지

LB 대상그룹에 Pod IP 가 직접 등록된다. 노드를 거치지 않는다

- **readiness 를 통과한 Pod 만 대상에 남는다. 배포 중에도 목록이 자동으로 갱신된다**



Target-type	대상	클라이언트 IP	운영 권장
`ip` **(권장)**	Pod IP : targetPort	NLB 는 보존, ALB는 XFF	기본으로 이것을 사용
`instance`	노드IP:NodePort	노드 IP로 바뀐다.	IP 대상을 사용하지 못할 때만

[실무] instance 모드는 NodePort 를 거친다. 예전 구성이거나 IP 대상을 못 쓸 때만 고른다

③④ 보이지 않는 두 번의 변환

클러스터 안에서 부를 때 이름 → ClusterIP → PodIP 로 두 번 바뀐다. 커널이 조용히 바꾼다

- 프로세스가 없는 구간이라 **로그가 없다**. 확인은 `get endpoints` 뿐이다

```
# 실제 값으로 따라가 보기
kubectl get svc shop-$ME -o wide
#   TYPE        CLUSTER-IP       PORT(S)
#   ClusterIP   10.100.141.251   80/TCP

kubectl get endpoints shop-$ME
#   ENDPOINTS   172.31.6.2:8080,172.31.12.9:8080

# 클러스터 안에서 부를 때 — 이름에서 Pod 까지
#   http://shop-$ME:80        ← CoreDNS 가 ClusterIP 로
#     ↓ iptables DNAT (③)   →  10.100.141.251:80   ClusterIP
#     ↓ iptables DNAT (④)   →  172.31.6.2:8080     Pod (확률 선택)
#   외부 유입은 ② 에서 LB 가 PodIP:8080 으로 바로 온다

# Service 를 건너뛰고 Pod 에 직접 — 기존 Pod 에 붙어서 확인한다
kubectl debug -it shop-$ME-xxxxx --image=curlimages/curl \
  --target=app -- curl -s 172.31.6.2:8080/api/info
```

[체크] 운영에서는 `kubectl debug` 로 기존 Pod 에 붙는다. 임시 Pod 가 아무 데서나 떠진다면 그것대로 점검할 일이다

⑤⑥ 컨테이너 안까지

Pod 에 도착했는데 안 되는 경우다. 포트가 틀렸거나 바인딩 주소가 틀렸다

- 앱이 127.0.0.1 에만 바인딩하면 컨테이너 밖에서 못 붙는다. 가장 흔한 마지막 함정

```
# 컨테이너가 실제로 무슨 포트를 Listen 하고 있나
kubectl exec shop-$ME-xxxxx -- ss -ltnp
#   LISTEN  0.0.0.0:8080     ← 정상. 밖에서 붙을 수 있다
#   LISTEN  127.0.0.1:8080   ← 문제. 컨테이너 안에서만 붙는다

# 운영 이미지에는 ss · netstat · curl 이 없다 (distroless · scratch)
kubectl debug -it shop-$ME-xxxxx --image=nicolaka/netshoot \
  --target=app -- ss -ltnp

# 매니페스트의 targetPort 와 실제 컨테이너 포트를 대조한다
kubectl get svc shop-$ME -o jsonpath='{.spec.ports[0].targetPort}'
kubectl get pod shop-$ME-xxxxx \
  -o jsonpath='{.spec.containers[0].ports[*].containerPort}'

# 운영에서 이 구간을 미리 막는 두 가지
#   앱이 기동 로그에 바인딩 주소를 남기게 한다
#   readinessProbe 를 Service 의 targetPort 와 같은 포트로 둔다
```

[정리] 운영 이미지는 도구가 없는 쪽이 옳다. 진단은 `kubectl debug` 로 붙여서 하고, 이미지에는 넣지 않는다

복구를 먼저 하고 원인은 그다음에 찾는다

장애 중에 할 일은 두 가지이다. 서비스를 세우는 것과 끊긴 구간을 좁히는 것

- **최근 배포가 있었다면 조사보다 `rollout undo` 가 먼저이다**

순서	무엇을 해 본다	되면	안 되면
0	**최근 롤아웃 · 설정 변경 확인**	변경 없음 → 1 로	변경 있음 → 먼저 되돌린다
1	**Pod IP 로 직접** `curl`	④~⑥ 정상 → 앞을 본다	⑤⑥ 문제 → 컨테이너로
2	`get endpoints`	④ 정상	selector · readiness
3	**Service 이름으로** `curl`	③④ 정상	DNS 또는 규칙
4	`get ingress` **ADDRESS**	②③ 정상	컨트롤러 · 애노테이션
5	`dig` **도메인**	① 정상	DNS 레코드
6	**밖에서** `curl`	전 구간 정상	—

[정리] 되돌려서 서비스가 살아나면 원인은 천천히 본다. 조사와 복구를 같이 하면 둘 다 늦는다

[핵심 요약] 호출 흐름 추적

구간을 외우는 것이 목적이 아니다. 어디를 먼저 볼지 정해 두면 원인을 찾는 시간이 줄어든다

- **서비스를 먼저 살리고 원인은 그다음에 찾는다. 미리 준비한 것이 없으면 둘 다 늦어진다**

운영에서 지킬 것	왜
문제가 나면 되돌리기부터	`rollout undo` 로 서비스를 먼저 살리고 원인은 그다음에 본다
Endpoints 개수를 지켜본다	이 숫자가 0 이 되는 순간 트래픽이 끊긴다
readiness 를 제대로 넣는다	준비된 Pod 만 Endpoints 에 남기는 유일한 장치다
진단은 kubectl debug 로	새 Pod 를 띄우지 않고 문제의 Pod 에 도구만 붙인다
이미지에 도구를 넣지 않는다	ss·curl 없는 이미지가 더 안전하다. 필요할 때만 붙인다
앱은 0.0.0.0 으로 Listen 하게 한다.	127.0.0.1 로 두면 컨테이너 밖에서 못 붙는다
DB 는 포트 포워딩으로만	포트를 상시로 열면 인터넷에 DB 를 공개하는 셈이다
externalTrafficPolicy	서비스마다 LB 를 만들면 요금과 관리 지점이 늘어난다
인증서와 DNS 를 감시한다	① 구간은 kubectl 로 안 보이고, 배포와 무관하게 끊긴다

[체크] 아홉 줄 중 우리 서비스에 지금 없는 것 하나를 고른다. 그 자리가 다음 장애에서 시간을 잡아먹는다

11. Ingress와 외부 노출

- Ingress 의 역할 · 컨트롤러 · 경로와 호스트 · HTTPS

본 문서는 SK AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

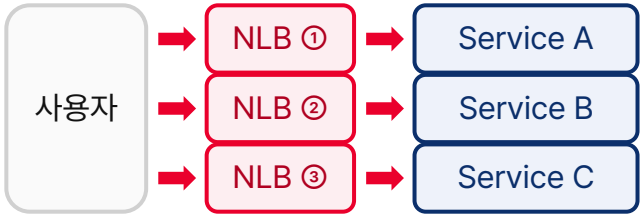
SK 주식회사 AX

Ingress 가 푸는 문제

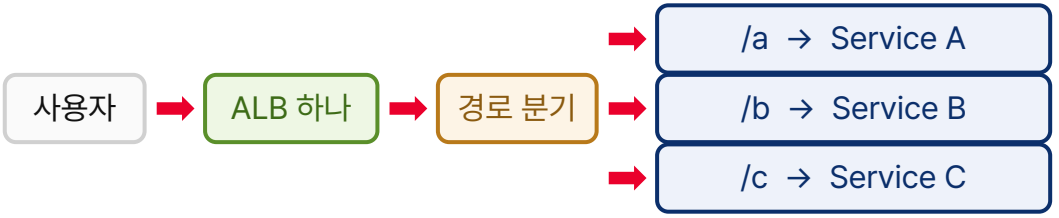
서비스마다 로드밸런서를 만들면 비용도 관리도 늘어난다. 하나로 묶어 경로로 나눈다

- LoadBalancer Service 5개 = **NLB 5개**. 월 요금이 다섯 배이다
- Ingress 는 **L7(HTTP)** 이라 경로·호스트·헤더로 분기할 수 있다

Service 만 사용하면 LB 가 서비스 수만큼 생긴다



Ingress 로 묶으면 LB 하나가 경로로 나눈다



LoadBalancer Service

L4 (TCP)
서비스당 LB 하나
HTTPS 를 직접 처리 못 함

Ingress

L7 (HTTP)
LB 하나에 여러 서비스
경로·호스트로 분기

EKS 에서

ALB 자동 생성
ACM 인증서 연동
WAF·인증 붙이기 쉬움

[정리] Ingress = 규칙, Ingress Controller = 그 규칙을 실제 LB 로 만드는 프로그램. 둘은 다른 것이다

Ingress Controller 고르기

EKS에서는 AWS Load Balancer Controller가 표준이다. 다른 선택지도 이유가 있을 때 쓴다

- 컨트롤러마다 지원하는 애노테이션이 다르다. 옮길 때 전부 다시 써야 한다

컨트롤러	만드는 것	장점	약점
AWS LB Controller	ALB / NLB	AWS 관리형, ACM·WAF 연동	AWS 종속, ALB 요금
ingress-nginx	클러스터 안 nginx Pod	이식성, 기능 풍부, 무료	Pod 를 직접 운영·확장해야
Traefik	클러스터 안 Pod	설정이 간결, 대시보드	생태계가 상대적으로 작다
Istio Gateway	Envoy	서비스 메시와 통합	학습·운영 부담이 크다

[실무] 두 개를 같이 쓸 수도 있다. 외부는 ALB, 내부 서비스 간 라우팅은 ingress-nginx 식으로 나누는 구성도 흔하다

Ingress Controller 안쪽

Ingress 리소스는 설정일 뿐이다. 실제로 트래픽을 받는 것은 컨트롤러의 Pod 이다

- **모든 Ingress 가 nginx 설정 파일 한 벌로 합쳐진다. 그래서 서로 간섭할 수 있다**



[주의] 호스트가 겹치면 먼저 만든 쪽이 이긴다. 실습에서 이름을 자기 것으로 유일하게 만들어야 하는 이유이다

Ingress 매니페스트

경로별로 어느 Service로 보낼지 적는다. 애노테이션이 ALB의 실제 동작을 결정한다

- `ingressClassName` 이 **어느 컨트롤러가 처리할지**를 정한다

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip        # Pod 로 직접
    alb.ingress.kubernetes.io/healthcheck-path: /actuator/health
    alb.ingress.kubernetes.io/group.name: team-a   # ALB 를 공유한다
spec:
  ingressClassName: alb
  rules:
    - http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service: { name: shop-api, port: { number: 80 } }
```

[실무] `group.name` 을 같게 주면 **여러 Ingress 가 ALB 하나를 공유**한다. 안 주면 Ingress 마다 ALB 가 생겨 요금이 는다

경로와 호스트로 나누기

같은 ALB에서 경로로도 나누고 도메인으로도 나눈다. 마이크로서비스를 하나의 주소로 묶는 방법

- **규칙 순서가 중요하다.** ALB 는 더 구체적인 규칙에 높은 우선순위를 준다

```
spec:                                  # ingressClassName: alb 는 생략
  rules:
    - host: api.shop.example.com         # 도메인으로 분기
      http:
        paths:
          - path: /orders
            pathType: Prefix
            backend: { service: { name: order-api, port: { number: 80 } } }
          - path: /users
            pathType: Prefix
            backend: { service: { name: user-api, port: { number: 80 } } }
          - path: /                      # 나머지 전부 (가장 마지막)
            pathType: Prefix
            backend: { service: { name: web, port: { number: 80 } } }
    - host: admin.shop.example.com       # 다른 도메인 → 다른 서비스
      http:
        paths: [ { path: /, pathType: Prefix,
                   backend: { service: { name: admin, port: { number: 80 } } } } ]
```

[주의] `path: /` 규칙을 **위쪽에 두면 아래 규칙이 절대 안 걸린다.** 넓은 규칙일수록 아래로 내린다

경로 규칙이 겹칠 때

어느 규칙이 이기는지 정해져 있다. 모르면 '왜 저기로 가지' 하고 헤맨다

- **정확한 경로(Exact) > 긴 접두사 > 짧은 접두사** 순으로 이긴다

pathType	규칙	`/api/v1/orders` 요청이
`Exact`	완전히 같아야	`/api/v1/orders` 만 매칭
`Prefix /api/v1`	경로 조각 단위로 시작	**매칭 (더 김 → 우선)**
`Prefix /api`	경로 조각 단위로 시작	매칭하지만 위에 밀린다
`Prefix /`	전부	**catch-all — 최후순위**
`Prefix /ap`	**매칭 안 됨**	조각 단위라 `/ap` ≠ `/api`

[주의] host 없는 Ingress 는 catch-all 이다. 우리 클러스터의 Grafana 가 그래서 아무 호스트나 받는다. 운영에서는 반드시 host 를 지정한다

HTTPS 붙이기

ACM 인증서를 애노테이션으로 연결하면 끝이다. 인증서 갱신도 AWS가 알아서 한다

- **TLS 종료는 ALB 에서 일어난다. 앱은 평문 HTTP 를 받는다**
- HTTP 로 들어온 요청은 **리다이렉트로 HTTPS 로** 넘긴다

```yaml
metadata:
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP":80},{"HTTPS":443}]'
    alb.ingress.kubernetes.io/certificate-arn: >-
      arn:aws:acm:ap-northeast-2:123456789012:certificate/abcd-1234
    alb.ingress.kubernetes.io/ssl-redirect: '443'    # HTTP → HTTPS
    alb.ingress.kubernetes.io/ssl-policy: ELBSecurityPolicy-TLS13-1-2-2021-06
spec:
  rules:
    - host: api.shop.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend: { service: { name: shop-api, port: { number: 80 } } }
```

[주의] 앱은 평문을 받으므로 `request.isSecure()` 가 false 이다. **X-Forwarded-Proto 헤더**를 봐야 원래 프로토콜을 안다

Ingress 가 안 될 때

ALB가 안 생기거나, 생겼는데 502/504가 발생하거나. 원인은 대개 정해져 있다

- **ADDRESS 열이 비어 있으면** 컨트롤러가 ALB 를 못 만든 것이다. 컨트롤러 로그를 본다

증상	원인	확인
ADDRESS 가 계속 비어 있음	컨트롤러 없음 · IAM 권한 부족	controller 로그, describe ingress
	서브넷 태그 누락	퍼블릭 서브넷에 kubernetes.io/role/elb
502 Bad Gateway	타깃이 비정상 — Pod 가 응답 안 함	타깃 그룹 상태, get endpoints
503 Service Unavailable	등록된 타깃이 없다	Service·Endpoints 확인
504 Gateway Timeout	앱 응답이 ALB 유휴 타임아웃(60초) 초과	타임아웃 조정 또는 앱 개선
헬스체크 실패	경로가 인증을 요구한다	/actuator/health 를 permitAll 로
일부 경로만 404	path 순서 · pathType 문제	규칙 순서 확인
HTTPS 인데 리다이렉트 루프	앱이 X-Forwarded-Proto 를 안 본다	forward-headers-strategy

[실무] `kubectl logs -n kube-system deploy/aws-load-balancer-controller` 가 **ALB 문제의 1차 정보원**이다

Ingress 없이 노출하는 법들

Ingress 가 늘 정답은 아니다. 무엇을 노출하느냐에 따라 고른다

- **HTTP 가 아니면 Ingress 를 못 쓴다. gRPC·TCP·DB 는 다른 방법이 필요하다**

방법	무엇에	비용·주의
Ingress	HTTP/HTTPS · 여러 서비스	**LB 하나를 공유** — 권장
LoadBalancer Service	TCP · gRPC · 단일 서비스	**서비스마다 LB** — 비싸다
NodePort	임시 · 테스트	인터넷에 포트가 열린다
port-forward	**개발·디버깅**	터미널이 살아 있어야
Gateway API	차세대 표준	심화 과정

[실무] LoadBalancer Service 를 서비스마다 만들면 LB 요금이 서비스 수만큼 붙는다. HTTP 면 Ingress 로 모으는 것이 정석이다

[핵심 요약] Ingress와 외부 노출

11장에서 남길 것은 외부에 내보낼 때의 판단과 함정이다

항목	한 줄 정리	실무 포인트
왜 Ingress	LB 하나에 여러 서비스를 경로로 붙인다	LoadBalancer Service 는 개수만큼 요금
구성 요소	Ingress(규칙) + Controller(실행)	컨트롤러 없으면 아무 일도 안 일어난다
IRSA	ServiceAccount 에 IAM 역할을 연결	액세스 키를 Secret 에 넣지 않는다
group.name	여러 Ingress 가 ALB 하나를 공유	안 주면 Ingress 마다 ALB 가 생긴다
target-type: ip	노드를 안 거치고 Pod 로 직접	홉이 줄고 클라이언트 IP 보존
path 순서	넓은 규칙(/)은 아래로	위에 두면 아래가 절대 안 걸린다
HTTPS	ACM ARN 을 애노테이션에	앱은 평문 — X-Forwarded-Proto 필요
헬스체크	ALB 가 200 을 받아야 타깃이 산다	Security 가 막으면 전체 503
정리	Ingress 삭제 = ALB 삭제	실습 후 반드시 정리 — 요금이 나간다

[체크] HTTPS 로 접속했는데 **로그인 후 http:// 로 튕긴다면** 무엇을 고쳐야 하는가?

12. 설정과 스토리지

- ConfigMap 과 Secret · 주입과 반영 · PV · PVC · 볼륨 선택

본 문서는 SK AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 AX

설정을 왜 밖으로 빼나

같은 이미지가 모든 환경에서 돌아야 한다. 환경마다 이미지를 다시 빌드하면 '테스트한 것'과 '배포한 것'이 달라진다

- 이미지에 설정을 넣으면 환경마다 다른 이미지가 된다. 검증의 의미가 사라진다
- 12-Factor App 의 3번 원칙: **설정은 환경에 저장**한다

방식	동작	문제	판단
이미지에 포함	application-prod.yml 을 빌드에 넣는다	환경마다 재빌드. 비밀값이 이미지에 남는다	쓰지 않는다
환경변수 직접	Deployment 에 env 를 나열	매니페스트가 길어지고 중복된다	값이 두세 개면 가능
ConfigMap	설정을 별도 오브젝트로	비밀값에는 부적합	**일반 설정은 이것**
Secret	비밀값 전용 오브젝트	기본은 base64 — 암호화가 아니다	**비밀값은 이것 + 추가 보호**
외부 저장소	Secrets Manager · Vault	구성이 늘어난다	운영 환경 권장

[정리] 빌드는 한 번, 배포는 여러 환경. 이 원칙이 깨지면 '스테이징에서는 됐는데'가 반복된다

ConfigMap 만들기

키-값 쌍이거나 파일을 통째로 넣는 형태이다. 두 형태를 다 지원하고 쓰임이 다르다

- 값은 **문자열**이다. 숫자·불린도 따옴표로 감싸야 안전하다

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: shop-config
data:
  # ① 키-값 — 환경변수로 주입하기 좋다
  SPRING_PROFILES_ACTIVE: "prod"
  LOG_LEVEL: "INFO"
  ORDER_TIMEOUT_MS: "3000"          # 숫자도 문자열로
  # ② 파일 통째로 — 볼륨으로 마운트하기 좋다
  application-prod.yml: |
    server:
      shutdown: graceful
    spring:
      datasource:
        url: jdbc:mysql://mysql:3306/shop
    logging:
      level: { com.skala.shop: INFO }
```

[주의] 값을 따옴표 없이 `true` 나 `3000` 으로 쓰면 **YAML 파서가 불린·숫자로 읽어** apply 가 거부된다

주입 방식 ① 환경변수

개별 키를 골라 넣거나 전부 한꺼번에 넣는다. 편한 대신 갱신이 안 된다

- **환경변수는 컨테이너 시작 시점에 고정**된다. ConfigMap 을 고쳐도 안 바뀐다

```yaml
containers:
  - name: app
    image: shop-api:1.0.0
    env:
      # ① 키 하나를 골라서 (이름을 바꿔 넣을 수 있다)
      - name: SPRING_PROFILES_ACTIVE
        valueFrom:
          configMapKeyRef: { name: shop-config, key: SPRING_PROFILES_ACTIVE }
      - name: DB_PASSWORD
        valueFrom:
          secretKeyRef: { name: shop-secret, key: db-password }
    envFrom:
      # ② 전부 한꺼번에 (키 이름이 그대로 환경변수 이름이 된다)
      - configMapRef: { name: shop-config }
      - secretRef: { name: shop-secret }
```

[실무] `envFrom` 은 편하지만 **어떤 변수가 들어가는지 매니페스트만 봐서는 모른다.** 중요한 값은 개별로 명시한다

주입 방식 ② 볼륨 마운트

ConfigMap을 디렉터리로 마운트하면 파일이 된다. 이 방식은 갱신이 반영된다

- 볼륨 마운트는 **ConfigMap 을 고치면 파일 내용이 자동으로 바뀐다** (약 1분 이내)
- 다만 앱이 파일을 다시 읽어야 의미가 있다. Spring Cloud Kubernetes 등이 필요

```yaml
spec:
  volumes:
    - name: config
      configMap:
        name: shop-config
        items:                              # 원하는 키만 고를 수도 있다
          - { key: application-prod.yml, path: application-prod.yml }
  containers:
    - name: app
      volumeMounts:
        - name: config
          mountPath: /config                # /config/application-prod.yml 이 생긴다
          readOnly: true
      env:
        - name: SPRING_CONFIG_ADDITIONAL_LOCATION
          value: /config/
```

[주의] `mountPath` 에 **기존 파일이 있으면 디렉터리가 통째로 덮인다.** /app 에 마운트하면 앱 JAR 가 사라진다

설정 우선순위

같은 값이 여러 곳에 있으면 누가 이기는지 알아야 한다

- **나중에 주입된 것이 이긴다. 컨테이너의 env 가 이미지의 ENV 를 덮는다**

순위	어디서	예
1 (가장 셈)	**컨테이너 `env`**	매니페스트에 직접 쓴 값
2	`envFrom` (ConfigMap·Secret)	묶어서 주입한 값
3	이미지의 `ENV`	Dockerfile 에 쓴 기본값
4	앱 내부 기본값	`application.yml` · `@Value` 기본
—	볼륨 마운트 파일	**환경변수와 별개 경로**

[체크] 설정이 이상하면 **매니페스트를 읽지 말고 `exec -- env` 를 친다.** 추측이 아니라 실물을 본다

설정이 바뀌면 다시 뜨게

ConfigMap 을 고쳐도 Pod 는 모른다. 알게 만드는 방법이 있다

- **Pod 템플릿이 바뀌어야 롤아웃이 일어난다. ConfigMap 변경은 템플릿이 아니다**

방법	어떻게	평가
손으로	`kubectl rollout restart`	실습에는 충분
해시 애노테이션	**ConfigMap 내용의 해시를 템플릿에**	**표준 관용구**
이름에 버전	`shop-config-v2` 로 새로 만들기	옛것이 쌓인다
볼륨 마운트	파일은 자동 갱신 (~1분)	**앱이 다시 읽어야**
Reloader 등	변경 감지 후 자동 롤아웃	도구를 하나 더 운영

[정리] 볼륨 마운트는 파일만 갱신된다. 앱이 파일 변경을 감시하지 않으면 여전히 옛 값을 쓴다. 두 단계다

Secret 은 암호화가 아니다

base64는 인코딩이다. 누구나 한 줄로 되돌릴 수 있다. 이 사실을 모르면 사고가 난다

- Secret 의 값은 **etcd 에 그대로 저장**된다 (기본 설정에서는 암호화 없이)
- **RBAC 으로 Secret 읽기 권한을 제한**하는 것이 첫 번째 방어선이다

```
# Secret 을 만들면 base64 로 보인다 — '암호화된 것 같은' 착시
kubectl create secret generic shop-secret \
  --from-literal=db-password='P@ssw0rd!'

kubectl get secret shop-secret -o jsonpath='{.data.db-password}'
# UEBzc3cwcmQh

# 되돌리는 데 1초 걸린다
kubectl get secret shop-secret -o jsonpath='{.data.db-password}' | base64 -d
# P@ssw0rd!

# 그래서 확인해야 할 것
kubectl auth can-i get secrets --as=system:serviceaccount:class-6:default
kubectl get secret -o yaml > backup.yaml   # ← 이 파일은 평문이나 다름없다
```

[주의] Secret 을 Git 에 커밋하지 마라. base64 는 난독화조차 아니다. 커밋 이력에 남으면 회수도 어렵다

Secret 만들고 쓰기

stringData를 쓰면 base64 인코딩을 직접 안 해도 된다. 사람이 읽고 쓰기 쉬워진다

- `data` 는 base64 값, `stringData` 는 **평문을 쓰면 쿠버네티스가 인코딩**해 준다

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: shop-secret
type: Opaque
stringData:                          # 평문으로 쓴다 (data 는 base64 필요)
  db-password: "P@ssw0rd!"
  jwt-secret: "very-long-random-string-here"
---
# 볼륨으로 마운트하면 파일이 된다 (환경변수보다 안전한 편)
spec:
  volumes:
    - name: secret
      secret: { secretName: shop-secret, defaultMode: 0400 }
  containers:
    - name: app
      volumeMounts:
        - { name: secret, mountPath: /etc/secret, readOnly: true }
```

[실무] 환경변수는 **크래시 덤프·로그·`/proc` 에 노출**될 수 있다. 민감도가 높으면 파일 마운트가 낫다

볼륨의 세 가지 수명

컨테이너와 함께 죽는 것, Pod와 함께 죽는 것, Pod보다 오래 남는 것. 목적에 따라 선택한다

- **컨테이너 재시작만으로도 파일시스템은 초기화된다. 볼륨 없이는 아무것도 안 남는다**

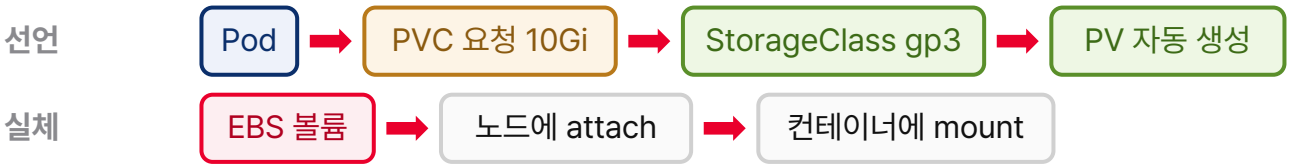
종류	수명	쓸 곳	주의
(볼륨 없음)	컨테이너 재시작까지	임시 계산	재시작하면 사라진다
emptyDir	Pod 수명	사이드카와 파일 공유, 임시 작업	Pod 가 옮겨 가면 사라진다
emptyDir(memory)	Pod 수명	빠른 임시 저장	메모리를 쓴다. limit 에 포함
PVC	**Pod 와 무관하게 유지**	**업로드 파일 · DB 데이터**	EBS 는 AZ 에 묶인다
hostPath	노드 수명	노드 로그 수집 등	**보안 위험. 앱에는 쓰지 않는다**
ConfigMap/Secret	오브젝트 수명	설정 파일 (12장)	읽기 전용

[주의] `hostPath` 는 **노드 파일시스템을 그대로 연다.** /var/run/docker.sock 같은 걸 열면 컨테이너 탈출로 이어진다

PV · PVC · StorageClass

앱은 '이만큼 필요하다'만 말하고(PVC), 실제 디스크는 자동으로 만들어진다(StorageClass)

- PVC 는 **요청서**, PV 는 **실제 볼륨**, StorageClass 는 **만드는 방법**이다
- 동적 프로비저닝 덕분에 **PV 를 사람이 미리 만들 필요가 없다**



PVC (요청)
크기와 접근 모드
네임스페이스에 속함
앱 팀이 작성

PV (실체)
실제 볼륨 하나
클러스터 범위
대개 자동 생성

StorageClass
어떤 디스크를 어떻게
gp3 · io2 · EFS
인프라 팀이 정의

[정리] 앱은 PVC 만 쓴다. 어떤 디스크인지, 어떻게 만드는지는 StorageClass 가 숨겨 준다

PVC 를 만들어 Pod 에 붙이기

PVC 하나와 volumeMounts 두 줄이면 된다. 실제 EBS 볼륨이 자동으로 만들어진다

- PVC 를 만들어도 **Pod 가 생기기 전에는 볼륨이 안 만들어진다** (WaitForFirstConsumer)

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: uploads }
spec:
  accessModes: [ReadWriteOnce]     # EBS 는 RWO 만 가능
  storageClassName: gp3
  resources: { requests: { storage: 10Gi } }
---
apiVersion: apps/v1
kind: Deployment
spec:
  template:
    spec:
      volumes:
        - { name: uploads, persistentVolumeClaim: { claimName: uploads } }
      containers:
        - name: app
          volumeMounts: [ { name: uploads, mountPath: /data/uploads } ]
```

[주의] RWO 볼륨을 쓰는 Deployment 는 replicas 를 2 이상으로 못 올린다. 두 번째 Pod 가 Pending 에 머문다

PVC 의 상태 전이

Pending 이 정상인 경우가 있다. 언제 그런지 알면 헤매지 않는다

- `ebs-sc` 는 **WaitForFirstConsumer** 이다. Pod 가 정해지기 전에는 일부러 만들지 않는다

상태	무슨 뜻	다음
Pending	아직 볼륨이 없다	Pod 를 붙이면 진행 (정상)
Pending (오래)	SC 가 없거나 용량·모드 불가	`describe pvc` 의 Events
Bound	볼륨과 연결됨	정상
Terminating (오래)	쓰는 Pod 가 남아 있다	Pod 를 먼저 지운다
Lost	PV 가 사라졌다	복구 불가 — 백업에서

[주의] PVC 를 지워야 EBS 가 삭제된다. `reclaimPolicy: Delete` 면 PVC 를 지울 때 볼륨과 데이터가 함께 사라진다

접근 모드와 스토리지 선택

여러 Pod가 같이 쓸 수 있는가가 첫 갈림길이다. 그다음이 성능과 비용이다

- **EBS 는 한 노드, EFS 는 여러 노드**에서 동시에 붙일 수 있다

접근 모드	뜻	EBS	EFS	S3
ReadWriteOnce	한 노드에서 읽기·쓰기	**가능**	가능	-
ReadOnlyMany	여러 노드에서 읽기만	불가	가능	-
ReadWriteMany	여러 노드에서 읽기·쓰기	**불가**	**가능**	-
특성				
성능	-	낮은 지연, 높은 IOPS	네트워크 지연 있음	객체 단위
비용(GB·월)	-	약 $0.09 (gp3)	약 $0.36	약 $0.025
쓸 곳	-	DB · 단일 Pod 데이터	공유 파일 · 미디어	**업로드 · 백업**

[실무] 파일 업로드는 **S3 를 먼저 검토하라.** PVC 는 용량 관리·백업·확장을 전부 우리가 책임진다

어떤 볼륨을 고를까

대부분의 앱은 볼륨이 필요 없다. 필요하다면 무엇이 필요한지부터 정한다

- **먼저 물어볼 것이 있다. 정말 파일로 남겨야 하나?** 로그는 stdout, 업로드는 S3 가 대개 낫다

용도	권장	왜
로그	**stdout · stderr**	볼륨 불필요 · 수집기가 가져간다
업로드 파일	**S3 등 오브젝트 스토리지**	무한 확장 · Pod 와 무관
임시 작업 파일	`emptyDir`	Pod 와 함께 사라져도 된다
캐시	`emptyDir` 또는 Redis	재생성 가능하면 볼륨 불필요
DB 데이터	**`ebs-sc` (RWO)**	빠름 · 한 노드 전용
여러 Pod 공유	**`efs-sc` (RWX)**	느리고 비싸지만 동시 접근
설정 파일	ConfigMap · Secret	볼륨이 아니라 설정이다

[실무] 볼륨이 붙는 순간 Pod 는 그 AZ·그 노드에 묶인다. 이동성이 줄어드는 대가를 치르는 것이니 정말 필요한지 먼저 따진다

[핵심 요약] 설정과 스토리지

설정은 밖에서 주입하고, 데이터는 Pod 밖에 둔다. 둘 다 '컨테이너는 사라진다'는 전제에서 나온다

- 이 장에서 가장 많이 막히는 것은 '고쳤는데 왜 그대로인가' 하나이다

주제	핵심	안 하면
설정 분리	이미지는 하나, 설정은 환경마다	환경마다 재빌드
환경변수 주입	**시작 시 고정** — 바꾸려면 롤아웃	고쳤는데 그대로
볼륨 주입	파일은 갱신되지만 앱이 다시 읽어야	설정 반영 지연
Secret	**인코딩일 뿐 암호화가 아니다**	Git 에 평문 유출
PV · PVC	요청(PVC)과 실물(PV)을 분리	앱이 인프라를 알게 된다
StorageClass	`ebs-sc` 는 WaitForFirstConsumer	엉뚱한 AZ 에 볼륨 생성
접근 모드	**EBS 는 RWO** · EFS 는 RWX	두 번째 Pod 가 Pending
정리	**PVC 를 지워야** 볼륨이 삭제된다	요금이 계속 나간다

[체크] `kubectl exec <pod> -- env` 로 **지금 실제로 들어간 값**을 본다. 매니페스트가 아니라 실물을 확인하는 습관

13. 리소스와 배포 실전

- requests 와 limits · 완성 매니페스트 · 종료 처리 · 진단과 다음 단계

본 문서는 SK AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 AX

requests 와 limits

requests는 스케줄러가 보는 값이고, limits는 커널이 강제하는 값이다. 둘의 쓰임이 완전히 다르다

- **스케줄러는 requests 만 본다.** limits 는 배치 결정에 아무 영향이 없다
- requests 를 안 적으면 스케줄러는 **'자원을 안 쓴다'고 판단**한다

	requests	limits
누가 쓰나	kube-scheduler	노드 커널(cgroup)
의미	이만큼은 보장받는다	이 이상은 못 쓴다
CPU 초과 시	-	**스로틀링 — 느려지지만 종료되지는 않는다**
메모리 초과 시	-	**OOMKilled. 즉시 종료된다**
안 적으면	스케줄러가 0으로 본다 → 노드 과밀	무제한 → 노드 전체를 위협
과하게 크면	노드를 낭비한다 (비용 증가)	-
권장	**실사용 p50~p70**	**requests 의 2~4배 (CPU) / 1.2~1.5배 (메모리)**

[주의] CPU 와 메모리는 초과 시 동작이 다르다. CPU 는 느려지고, 메모리는 종료된다. 그래서 메모리 limit 이 더 위험하다

단위와 실제 의미

CPU는 밀리코어, 메모리는 바이트이다. Mi와 M을 헷갈리면 5% 차이가 난다

- `1000m` = **vCPU 1개**. `500m` 은 vCPU 절반을 쓸 수 있다는 뜻이다
- `Mi` **는 1024 기반,** `M` **은 1000 기반**이다. 메모리는 항상 `Mi`/`Gi` 를 사용한다

```
resources:
  requests:
    cpu: 200m            # 0.2 vCPU — 스케줄러가 이만큼 자리를 잡아 준다
    memory: 512Mi        # 512 × 1024 × 1024 바이트
  limits:
    cpu: 1000m           # 1 vCPU 까지 (초과하면 스로틀링)
    memory: 1Gi          # 1Gi 초과 시 OOMKilled

# 단위 참고
#   cpu:  1 = 1000m = vCPU 1개 (하이퍼스레드 1개 기준)
#   mem:  1Gi = 1024Mi = 1,073,741,824 B
#         1G  = 1000M  = 1,000,000,000 B   ← 약 7% 작다
#
# 노드의 실제 여유 확인
# kubectl describe node <노드>   → Allocatable / Allocated resources
# kubectl top node               # metrics-server 필요
```

[실무] 노드의 전체 메모리 ≠ 사용 가능 메모리. 예약분을 뺀 값이 Allocatable 이다. `describe node` 로 확인한다

QoS 클래스와 축출 순서

QoS(Quality of Service) 는 AWS 고유 기능이 아니라 **Kubernetes가 Pod의 CPU/Memory requests와 limits 설정을 기준으로 Pod의 자원 우선순위를 분류하는 기능이며,** 노드의 CPU·메모리가 부족해졌을 때 어떤 Pod를 우선 보호하고, 어떤 Pod 를 먼저 종료할지 판단하는 기준을 제공함.

노드에 자원이 부족하면 누군가는 쫓겨난다. 그 순서는 우리가 적은 requests/limits로 이미 정해져 있다

- **BestEffort 가 가장 먼저 쫓겨난다. requests 를 안 적은 Pod 가 여기 해당한다**
- 중요한 서비스는 **Guaranteed 로 만들어** 축출 대상에서 뒤로 밀어 둔다

QoS	조건	축출 순서	쓸 곳
Guaranteed	모든 컨테이너에 requests = limits	**가장 나중**	DB · 핵심 API
Burstable	requests < limits (또는 일부만 지정)	중간	일반 웹 서비스
BestEffort	**아무것도 안 적음**	**가장 먼저**	배치·실험 워크로드
축출 기준			
1순위	BestEffort	-	즉시 대상
2순위	Burstable 중 requests 초과 사용량이 큰 것	-	많이 초과할수록 먼저
3순위	Guaranteed	-	노드가 정말 위험할 때만

[실무] QoS 는 **설정하는 값이 아니라 결과**이다. requests/limits 를 어떻게 적었는지가 자동으로 클래스를 정한다

컨테이너에 올릴 앱의 조건

12-Factor App의 항목 중 쿠버네티스에서 특히 중요한 여섯 가지. 하나씩 확인하며 시작한다.
(클라우드 환경에서 애플리케이션을 **배포하기 쉽고, 확장하기 쉽고, 운영하기 쉽게 만들기 위한 12가지 설계 원칙)**

- **상태를 프로세스 안에 두지 않는 것**이 가장 중요한 조건이다

조건	왜	안 지키면	다룬 장
설정을 환경에서 읽는다	이미지 하나로 모든 환경	환경마다 재빌드	12장
로그를 stdout 으로	Pod 가 사라져도 로그는 남는다	장애 원인이 함께 사라진다	2장
상태를 밖에 둔다	Pod 는 언제든 종료된다	확장·재시작에서 데이터 유실	9장
빠르게 뜨고 우아하게 종료된다	롤아웃·스케일 속도	배포 때마다 오류	이 장
헬스 엔드포인트 제공	프로브의 판단 근거	무중단 배포 불가	7장 · 이 장
단일 프로세스	컨테이너 하나에 하나	시그널 전달·모니터링이 꼬인다	2장

[체크] 우리 앱은 **재시작해도 잃는 것이 없는가?** 세션·업로드 파일·캐시·스케줄러 상태를 하나씩 짚어 보라

Actuator 로 헬스 엔드포인트 만들기

Spring Boot는 liveness/readiness 그룹을 기본 제공한다. 다만 꺼져 있으므로 켜야 한다

- **`/actuator/health` 는 DB 까지 확인**한다. liveness 에 사용하면 재시작 폭풍이 발생한다 (7장)

```yaml
# application-prod.yml
management:
  endpoints:
    web: { exposure: { include: health, info, prometheus } }
  endpoint:
    health:
      probes: { enabled: true }        # liveness/readiness 그룹 활성화
      show-details: never              # 외부에 내부 정보를 노출하지 않는다
  health:
    livenessState: { enabled: true }
    readinessState: { enabled: true }
  server:
    port: 8081                         # 관리 포트 분리 — 외부 노출을 막기 쉽다

server:
  shutdown: graceful                   # 종료 시 처리 중 요청을 마무리
spring:
  lifecycle: { timeout-per-shutdown-phase: 25s }
```

[실무] 관리 포트를 **8081 로 분리**하면 Ingress 는 8080 만 노출하므로 `/actuator` 가 인터넷에 열리지 않는다

종료를 우아하게 만들기

SIGTERM을 받고 처리 중인 요청을 마무리한 뒤 종료된다. 세 곳의 시간이 맞아야 한다

- **preStop sleep → Spring 종료 유예 → grace period** 순으로 시간이 커져야 한다

설정	값	역할	어긋나면
`preStop` **sleep**	5초	Endpoints 제거 전파를 기다린다	종료 중 Pod 로 트래픽 → 502
`server.shutdown`	graceful	새 요청은 거부, 진행 중은 완료	처리 중 요청이 끊긴다
`timeout-per-shutdown-phase`	25초	Spring 이 기다리는 최대 시간	긴 요청이 잘린다
`terminationGracePeriodSeconds`	40초	K8s 가 기다리는 최대 시간	**작으면 SIGKILL**
합계 검산	5 + 25 < 40	**여유가 있어야 한다**	SIGKILL 로 강제 종료
ENTRYPOINT	exec 형식	SIGTERM 이 앱에 도달	셸 형식이면 전달 안 됨 (2장)

[주의] `terminationGracePeriodSeconds` 가 **preStop + Spring 유예보다 작으면** 앱이 정리를 마치기 전에 SIGKILL 이 온다

완성 매니페스트 ① Deployment

지금까지 배운 것이 전부 들어 있다. 프로브·리소스·설정·종료·분산

- 각 줄이 어느 장에서 나왔는지 확인하며 읽으면 **전체가 하나로 이어진다**

```
spec:
  replicas: 3                                    # 8장
  strategy: { rollingUpdate: { maxSurge: 1, maxUnavailable: 0 } }
  template:
    spec:
      terminationGracePeriodSeconds: 40          # 9장 · 이 장 (분산 제약 생략)
      containers:
        - name: app
          image: <레지스트리>/shop-api:1.0.0-a3f9c21    # 2장 (커밋 SHA 태그)
          ports: [ { name: http, containerPort: 8080 },     # 서비스
                   { name: mgmt, containerPort: 8081 } ]     # 관리
          envFrom: [ { configMapRef: { name: shop-config } },
                     { secretRef: { name: shop-secret } } ]   # 12장
          resources:                             # 이 장 앞부분
            requests: { cpu: 300m, memory: 768Mi }
            limits:   { cpu: "1",  memory: 1Gi }
          lifecycle:
            preStop: { exec: { command: ["sh","-c","sleep 5"] } }
```

[정리] 이 한 장이 **2·7·8·13장과 이 장의 결론**이다. 각 설정에는 그것이 막아 주는 구체적인 장애가 하나씩 있다

완성 매니페스트 ② 프로브

관리 포트를 보고, 세 프로브가 각자 다른 일을 한다

- 프로브가 **관리 포트(8081)** 를 보므로 서비스 포트와 독립적이다

```
        startupProbe:                       # 기동 유예 최대 150초
          httpGet: { path: /actuator/health/liveness, port: mgmt }
          periodSeconds: 5
          failureThreshold: 30
        livenessProbe:                      # 프로세스가 살아 있는가
          httpGet: { path: /actuator/health/liveness, port: mgmt }
          periodSeconds: 10
          timeoutSeconds: 3
          failureThreshold: 3
        readinessProbe:                     # 트래픽을 받아도 되는가
          httpGet: { path: /actuator/health/readiness, port: mgmt }
          periodSeconds: 5
          timeoutSeconds: 3
          failureThreshold: 3
        securityContext:                    # 보안 기본값
          runAsNonRoot: true
          allowPrivilegeEscalation: false
          readOnlyRootFilesystem: true
```

[주의] `readOnlyRootFilesystem: true` 를 켜면 **`/tmp` 도 못 쓴다.** emptyDir 을 `/tmp` 에 마운트해야 한다

완성 매니페스트 ③ Service 와 Ingress

서비스 포트만 노출하고, 경로로 붙인다. HPA 는 심화 과정에서 다루므로 샘플 프로젝트를 참고한다

- Service 는 8080 만 노출한다. 8081 을 노출하면 관리 포트 분리가 무의미해진다

```
kind: Service                       # apiVersion 은 지면상 생략
metadata: { name: shop-api }
spec:
  selector: { app: shop-api }
  ports: [ { name: http, port: 80, targetPort: http } ]   # mgmt 제외
---
kind: Ingress
metadata:
  name: shop-api
  annotations:                                            # 11장
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/healthcheck-port: "8081"    # 관리 포트로
    alb.ingress.kubernetes.io/healthcheck-path: /actuator/health/readiness
    alb.ingress.kubernetes.io/group.name: team-a
spec:
  ingressClassName: alb
  rules: [ { http: { paths: [ { path: /api, pathType: Prefix, backend:
    { service: { name: shop-api, port: { number: 80 } } } } ] } } ]
```

[실무] ALB 헬스체크를 readiness 로 잡으면 앱이 준비된 뒤에 타깃이 살아난다. 배포 중 502 가 줄어든다

배포 전 최종 점검표

운영에 처음 올리기 전에 이 열 가지를 확인한다. 하나라도 빠지면 첫 장애에서 만난다

- 실습용 설정이 **그대로 운영에 올라가는 일**이 실제로 자주 일어난다

항목	확인	관련 장
이미지 태그가 불변인가	latest 가 아닌 커밋 SHA	2장
non-root 로 도는가	USER 지정 + runAsNonRoot	2장 · 이 장
requests/limits 가 있는가	BestEffort 가 아닌지	이 장
세 프로브가 다 있는가	startup · liveness · readiness	7장
liveness 가 DB 를 안 보는가	`/health/liveness` 경로	7장 · 이 장
graceful shutdown 이 되는가	preStop + shutdown + grace	이 장
설정이 밖에 있는가	ConfigMap · Secret	12장
비밀값이 Git 에 없는가	이력까지 확인	12장
복제본이 흩어져 있는가	`get pod -o wide` 의 NODE	**심화**
PDB 가 있는가	노드 작업 시 보호	**심화**

[체크] 이 열 개 중 **지금 우리 서비스가 못 지키는 것은 무엇인가?** 그것이 다음에 겪을 장애의 후보이다

장애 대응의 순서

원인을 찾기 전에 영향을 줄인다. 순서를 정해 두면 새벽에도 흔들리지 않는다

- **복구가 먼저, 원인은 그다음이다. 원인을 찾다가 장애 시간이 길어지는 경우가 많다**

단계	무엇을	명령·행동
① 범위 확인	전체인가 일부인가	`kubectl get pod -A | grep -v Running`
② 최근 변경	배포·설정 변경이 있었나	`kubectl rollout history` · Git 로그
③ 되돌리기	**최근 변경이 있으면 먼저 롤백**	`rollout undo` · `helm rollback`
④ 영향 축소	안 되면 스케일 아웃·트래픽 차단	`kubectl scale` · Ingress 수정
⑤ 원인 조사	서비스가 안정된 뒤에	logs · describe · 지표
⑥ 기록	타임라인과 조치 내용	포스트모템 작성
⑦ 재발 방지	알람·프로브·테스트 추가	같은 장애를 두 번 겪지 않게

[실무] 3번을 망설이지 마라. '원인을 알고 되돌리자'는 생각이 장애를 30분에서 3시간으로 늘린다

종합 진단표 ① 워크로드

Pod 상태별 원인과 확인 명령. 현업에서 가장 자주 펴 보게 될 표

- **상태 이름이 곧 어느 단계에서 막혔는지**를 알려 준다 (3장)

증상	원인 후보	확인	조치
Pending	노드 자원 부족	describe pod 의 Events	requests 조정 · 노드 증설
	taint · affinity · AZ 불일치	같은 Events	제약 완화 · WaitForFirstConsumer
ImagePullBackOff	태그 오타 · ECR 권한	describe 의 Failed to pull	태그·권한 확인
	플랫폼 불일치 (arm64)	exec format error	`--platform linux/amd64`
CrashLoopBackOff	앱 기동 실패	**logs --previous**	설정·의존성 확인
	liveness 가 너무 민감	describe 의 Liveness failed	startupProbe · 임계 완화
OOMKilled	메모리 limit 초과	describe 의 Last State	limit 상향 · 힙 비율 조정
Running 0/1	readinessProbe 실패	describe 의 Readiness failed	경로·포트·의존성
Evicted	노드 자원 부족으로 축출	get events	requests 설정 · QoS 상향
ContainerCreating	볼륨 attach 실패	describe 의 Events	AZ · 볼륨 개수 상한

[실무] describe 의 Events 를 먼저 본다. 위 열 가지 중 아홉 가지가 거기에 원인이 그대로 적혀 있다

종합 진단표 ② 네트워크·설정

워크로드는 정상인데 안 될 때 보는 표이다

- **증상에서 시작해 확인 명령으로 간다. 원인 추측은 마지막이다**

증상	원인 후보	확인
연결 즉시 거부	**Endpoints 가 비었다**	`get endpoints`
연결 타임아웃	포트·바인딩 주소	`exec -- ss -ltnp`
Service 는 되는데 Ingress 만	class · host · path	`describe ingress` Rules
Ingress ADDRESS 없음	**ingressClassName**	`get ingressclass`
이름 해석 안 됨	네임스페이스 · CoreDNS	`nslookup` (netshoot)
외부 API 호출이 느림	**DNS search 목록**	도메인 끝에 `.` 을 붙여 비교
설정이 안 바뀜	**환경변수는 시작 시 고정**	`exec -- env`
`CreateContainerConfigError`	ConfigMap·Secret 없음	`describe pod` Events
`ImagePullBackOff (이미지)`	이미지 경로·태그 오타	`describe pod` 의 `Events`

[체크] 표의 순서가 곧 확인 순서이다. 위에서부터 짚어 내려가면 대개 세 번 안에 걸린다

운영에 올리기 전 마지막

실습 환경과 운영은 다르다. 무엇을 바꿔야 하는지 목록으로 남긴다

항목	실습환경에서는	실제 운영에서는
저장소	인메모리	**실제 DB**
이미지 태그	`1.0.0`	**커밋 SHA**
비밀값	Secret 평문	External Secrets
실습용 API	`/api/load` 열림	**제거**
리소스 값	추측	**`top` 실측 기반**
권한	반 공용 계정	최소 권한 RBAC
관측	`kubectl logs`	로그 수집 · 지표 · 알림
배포	손으로 `apply`	GitOps
가용성	replicas 2	PDB(PodDisruptionBudget) · 노드 분산

[주의] **`/api/load` 와 `/api/unready` 는 실습 전용**이다. 외부에 열리면 그대로 공격 수단이 되므로 운영 이미지에서 반드시 제외한다
[참고] PDB →"노드 점검이나 교체를 하더라도 최소 몇 개의 Pod는 반드시 살아 있어야 한다"를 지정하는 정책



수고하셨습니다.

본 문서는 SK AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 AX